# פרויקט הנדסת תוכנה

## תיק פרויקט בחלופה הגנת סייבר ומערכות הפעלה

# תוכן עניינים

**מבוא** ..... 4
**רקע ותקציר כללי:** ..... 4
**רציונל הפרויקט:** ..... 4
**מה המוצר המוגמר אמור לבצע:** ..... 5
**אילו אתגרים אני צופה:** ..... 5
**הגדרת הלקוח:** ..... 5
**הגדרת יעדים ומטרות** ..... 5
**בעיות, תועלות וחסכונות** ..... 6
**סקירות פתרונות קיימים** ..... 6
**סקירת טכנולוגיות הפרויקט:** ..... 7
**מגבלות טכנולוגיות:** ..... 7
**תיחום הפרויקט:** ..... 7
**תיאור מפורט של המערכת:** ..... 8
**פירוט היכולות לפי סוג משתמש** ..... 9
**פירוט הבדיקות:** ..... 10
**לוח זמנים:** ..... 11
**ניהול הסיכונים בפרויקט:** ..... 13
**מבנה / ארכיטקטורה של הפרויקט** ..... 21
**תיאור חומרה** ..... 21
**תיאור הטכנולוגיה הרלוונטית** ..... 23
**תיאור זרימת המידע במערכת** ..... 25
**תיאור האלגוריתמים המרכזיים בפרויקט:** ..... 28
**תיאור סביבת הפיתוח** ..... 31
**תיאור מסכי המערכת** ..... 36
- **מסך בית** ..... 36
- **מסך אודות** ..... 37
- **מסך הרשמה** ..... 38
- **מסך התחברות** ..... 40
- **מסך חיפוש טיסה** ..... 40
- **עמוד ההזמנות שלי** ..... 42
- **מסך הצ'אט** ..... 44
- **מסך השרת** ..... 44

**פירוט מבני הנתונים** ..... 47
**סקירת האיומים שיתכנו למערכת** ..... 50
**מימוש הפרויקט** ..... 52
**מודולים/מחלקות מיובאים** ..... 52

**מחלקות/מודולים שפותחו במסגרת הפרויקט** ..... 52
**About** ..... 52
**Contact** ..... 54
**Flights** ..... 56
**Home** ..... 60
**Login** ..... 63
**Signup** ..... 66
**MyBookings** ..... 69
**Server** ..... 73
**ClientHandler** ..... 75
**City** ..... 79
**Seat** ..... 80
**BookedFlight** ..... 81
**User** ..... 82
**Flight** ..... 83
**ActiveFlight** ..... 85
**FlightSearchData** ..... 86
**ChatMessage** ..... 87
**Users** ..... 88
**ConnectionManager** ..... 91
**OnlineUsersManager** ..... 92
**SecurityManager** ..... 93
**UDP** ..... 96
**UI** ..... 97
**קטעי קוד של האלגוריתמים** ..... 99
**מסמך הבדיקות:** ..... 109
**מדריך למשתמש** ..... 110
**פירוט כלל קבצי המערכת:** ..... 110
- **סביבה נדרשת:** ..... 110
- **כלים נדרשים:** ..... 111
- **מיקומי הקבצים:** ..... 112
- **רשת:** ..... 112
- **משתמשי המערכת:** ..... 112

**סיכום אישי / רפלקציה** ..... 113
**ביבליוגרפיה:** ..... 114

## מבוא

**רקע ותקציר כללי:**

חברות תעופה בכל העולם משתמשות במערכות להזמנת טיסות כדי לנהל את כל תהליך ההזמנה של הנוסעים בצורה מסודרת, מהירה ומדויקת. בעבר אנשים היו צריכים ללכת לסוכנות נסיעות בשביל להזמין כרטיס, אבל היום כמעט הכול נעשה דרך האינטרנט, דרך מערכות מתקדמות של החברות עצמן, באופן עצמאי על ידי הנוסע.

המטרה של המערכות האלו היא לאפשר לנוסעים לבצע הזמנה בקלות, לבחור יעד, לראות תאריכים, לבחור מושב, לשלם ולקבל כרטיס - והכול בצורה אוטומטית. המערכות האלה גם עוזרות לחברות לנהל את המקומות במטוס, לעקוב אחרי כמות הנוסעים, ולעדכן את הנתונים ברגע.

בנוסף, היום יש תחרות גדולה מאוד בין חברות התעופה,מנועי חיפוש ואתרי ההזמנות, וכל אחת מנסה להציע מערכת נוחה יותר, ברורה יותר ואטרקטיבית יותר למשתמש. החברות משקיעות בעיצוב, באינטראקטיביות, כלים מתקדמים כמו בחירת מושבים בזמן אמת, מפה אינטראקטיבית, התראות חכמות ועוד. כל זה נועד למשוך את המשתמש לבחור דווקא בהן ולא במתחרות. לכן, מערכות כאלה הפכו לחלק חשוב ומשמעותי מהשירות שחברת תעופה נותנת ללקוח, לא פחות מהטיסה עצמה.

המערכת CybAir ,מאפשרת למשתמשים להזמין טיסות בצורה פשוטה ונוחה, דרך ממשק ידידותי. CybAir משלבת מאפיינים של מגוון חברות תעופה רשמיות במטרה ליצור את החוויה הטובה ביותר בעת הזמנת כרטיסי טיסה.

CybAir בנויה כך שיש שרת מרכזי שמנהל את המידע, ולקוחות שמתחברים אליו כדי לבצע פעולות שונות. הפרויקט מאפשר ללמוד על תקשורת ברשת, סנכרון מידע בזמן אמת ואבטחת מידע בתהליכי הזמנה. בנוסף, הפרויקט משלב תכנות של ממשק משתמש גרפי עם ניהול חיבורים ברשת. הפרויקט מדמה מערכת הזמנות אמיתית ועוזר להבין איך חברות תעופה מנהלות את ההזמנות שלהן כיום.

**רציונל הפרויקט:**

תחום הזמנת הטיסות רלוונטי מאוד והוא ממשיך להתפתח מיום ליום. הפרויקט נבחר מתוך עניין וסקרנות בפיתוח מערכות מורכבות המשלבות בין ניהול נתונים, חוויית משתמש ונוחות שימוש. מערכות להזמנת כרטיסי טיסה דורשות תכנון מדויק של מבנה הנתונים, תקשורת בין רכיבים שונים, ואינטראקציה נוחה עם המשתמש תוך שמירה על אבטחה גבוהה מאוד. מדובר בתהליך שכולל מידע משתנה כמו זמינות מושבים, פרטי משתמשים, טיסות, תפקידים ברשת ועוד, וכל אלה צריכים לעבוד יחד. מערכת מורכבת כזו ניתן לבנות

במגוון דרכים יצירתיות ולשלב בה מגוון מאפיינים לפי העדפות אישיות, ולכן בגלל האפשרויות הרבות לפיתוח, והרלוונטיות של התחום כיום, הפרויקט נבחר.

**מה המוצר המוגמר אמור לבצע:**

המערכת מאפשרת יצירת חשבון, התחברות, בחירת טיסות ליעדים רבים בכל העולם בזמן אמת מתוך מאגר טיסות, בחירת מושבים בטיסה, תקשורת עם נציגים ותמיכה, הפקת כרטיסי טיסה רשמיים והיכרות בין הנוסעים עם מפה אינטראקטיבית. כל הפעולות של המערכת מתבצעות תוך שמירה על פרטיות הנוסעים ואבטחת המידע הרגיש.

**אילו אתגרים אני צופה:**

האתגרים המרכזיים בפרויקט הם שמירה על תקשורת תקינה ויציבה בין השרת ללקוחות, ניהול סנכרון של הנתונים (כמו זמינות מושבים) בזמן אמת, טיפול בתקלות ובעיות תקשורת, אבטחת המידע של המשתמשים. בנוסף, מערכת מהסוג הזה פונה למשתמשים אמיתיים ולכן יש לה חשיבות רבה בהצגת ממשק נוח, זרימה חלקה של הפעולות, ומענה ברור לכל מצב. השילוב בין רמה טכנית גבוהה לבין שירות נגיש למשתמש הופך את העבודה על הפרויקט לאתגר טכנולוגי מעמיק ומעניין.

**הגדרת הלקוח:**

המערכת מיועדת למשתמשים שרוצים להזמין טיסות בצורה נוחה, אינטואיטיבית ומהירה ונמצאים ברשת מקומית. בנוסף, היא מתאימה גם לגורמים בתוך חברת התעופה עצמה, כמו מנהלי טיסות או נציגים שאחראים על סידור המושבים והזמנות. בפועל, המשתמשים העיקריים במערכת הם לקוחות שמתחברים כדי לבצע הזמנה, לבחור יעד ומושב, לנהל את תהליך הטיסה ולהכיר נוסעים אחרים מכל העולם. מצד שני, יש גם משתמש אחד שמשמש כשרת ומנהל את כל המידע - הוא זה שמחזיק את הנתונים המרכזיים, מטפל בבקשות, ודואג שהמערכת תתפקד בצורה תקינה מול כל הלקוחות האחרים.

**הגדרת יעדים ומטרות**

המערכת נועדה לספק פתרון פשוט, מהיר ונוח להזמנת טיסות וניהול מקומות ישיבה. המטרה המרכזית היא לאפשר למשתמש לבצע את כל שלבי ההזמנה, החל מהרשמה והתחברות, בחירת טיסה ומושב עד להצגת הכרטיס הדיגיטלי או המודפס בשדה התעופה והיכרות עם נוסעים אחרים במטוס דרך ממשק אחד ברור ויעיל.

מטרות נוספות של המערכת כוללות:

- מתן חוויה משתמש איכותית, הכוללת ממשק גרפי נוח וברור.
- יצירת סביבת עבודה שמבוססת על מודל של שרת ולקוחות, כדי לדמות מצב מציאותי של מערכת טיסות.

- ניהול מידע בזמן אמת וסנכרון בין המשתמשים, כך שכל שינוי במצב הטיסה (כמו תפיסת מושב) מתעדכן מיד אצל כולם.

**בעיות, תועלות וחסכונות**

אחת הבעיות המרכזיות בעולם התעופה היא יצירת מערכת שתאפשר לנוסעים להזמין טיסות בצורה נוחה, מהירה וללא תקלות. פעמים רבות מערכות כאלה מסובכות לשימוש או שהן לא מציגות את כל המידע בצורה ברורה, מה שמקשה על המשתמש לבצע החלטה. בנוסף, קיים אתגר טכני של שמירה על מידע מעודכן בזמן אמת, במיוחד כאשר מספר משתמשים מבצעים פעולות במקביל.

המערכת CybAir נועדה להתמודד עם בעיות אלה על ידי יצירת מערכת פשוטה להפעלה, עם ממשק משתמש ברור, וסנכרון מהיר של הנתונים בין כל המשתמשים. המערכת מבטיחה שכל שינוי שמתבצע, כמו תפיסת מושב, יתעדכן אצל כל שאר המשתמשים.

התועלות המרכזיות מהמערכת הן:

- נוחות בשימוש וביצוע פעולות בצורה ברורה ומהירה
- חיסכון בזמן למשתמש בזכות תהליך פשוט
- עקרון מקורי של היכרות בין הנוסעים בטיסה
- מניעת תקלות בזכות בדיקות תקינות ואימות נתונים
- הענקת ביטחון למשתמשים באמצעות הגנה על מידע רגיש

השירותים שהמערכת מספקת כוללים:

- הרשמה והתחברות
- הצגת טיסות קיימות
- בחירת מושב בטיסה
- הצגת הזמנות של המשתמש וכרטיסי טיסה רשמיים
- הצגת יתר הנוסעים בטיסה ואת עיר המוצא של כל אחד מהם על מפה אינטראקטיבית (למשתמש יש בחירה אם המידע ייחשף או לא)
- שיחוח עם השרת וקבלת שירות

**סקירות פתרונות קיימים**

בעולם קיימות מערכות רבות להזמנת טיסות, בעיקר של חברות תעופה מסחריות ואתרי הזמנות גדולים. מערכות אלו כוללות מגוון רחב של אפשרויות, ביניהן ניהול דרכונים, תהליכי אימות זהות, שדות מותאמים לפי מדינות, אפשרויות ביטוח, ועוד תהליכים בירוקרטיים מורכבים. לעומת זאת, המערכת CybAir מתמקדת בהשגת המטרה המרכזית של המשתמש: הזמנת מקום בטיסה בצורה פשוטה, נגישה וברורה יותר. המערכת נבנתה כך שכל משתמש

יוכל להבין במהירות את השלבים הדרושים, בלי להסתבך בפרטים טכניים ובמסכים עמוסים. היא מתאימה במיוחד למצבים שבהם הפשטות והיעילות חשובים יותר מהמורכבות.
מה שמייחד את המערכת לעומת מערכות מסחריות הוא האפשרות ליצירת היכרות אינטראקטיבית בין הנוסעים. כל משתמש יכול לראות את יתר הנוסעים שמעוניינים בהיכרות בטיסה, מה שמם, מה גילם, ומהי עיר המוצא שלהם. תכונה זו יוצרת חוויה אישית ומחברת יותר בין המשתמשים, באופן שלא קיים ברוב מערכות ההזמנות הרגילות.

**סקירת טכנולוגיות הפרויקט:**
מערכת CybAir פותחה בסביבת Windows Forms בשפת התכנות #C תוך שימוש בעקרונות של מודל שרת-לקוח עם תקשורת באמצעות פרוטוקול UDP לבחירת השרת ויצירת תשתית החיבור בינו לבין שאר הלקוחות ותקשורת באמצעות פרוטוקול TCP ליצרית החיבור עצמו והעברת מסרים ישירה בין השרת ללקוח. מאגר הטיסות בפרויקט נוצר באמצעות עיבוד של מאגר נתונים אמיתי של חברת תעופה משנת 2013 והתאמתו למערכת CybAir בשפת התכנות Python בספריית Pandas. הבחירה בטכנולוגיות אלו נבעה מהרצון לבנות מערכת עם ממשק גרפי נוח וגמיש, שמאפשר אינטראקציה בזמן אמת בין מספר משתמשים תוך יצירת תקשורת ביניהם ברשת מקומית באופן מאובטח. המערכת אינה דורשת חיבור לאינטרנט או שרתים חיצוניים, אלא מתבסת על רשת מקומית ,(LAN) מה שהופך אותה לנגישה גם ללא תשתיות מתקדמות. בנוסף, נעשה שימוש בספריות וטכנולוגיות רבות למטרות שונות (הרחבה בהמשך)

הטכנולוגיות שבהן נעשה שימוש אינן חדשות או לא מוכרות אך השילוב ביניהן נעשה באופן מעניין ויצירתי.

**מגבלות טכנולוגיות:**
המערכת פועלת רק בסביבה מקומית ולכן לא מחוברת לבסיסי נתונים חיצוניים. בנוסף, היא מוגבלת למספר משתמשים שפועלים באותו הזמן דרך הרשת המקומית, ולא תומכת בתקשורת אינטרנטית גלובלית. במערכת לא קיים שילוב עם מערכות תשלום אמיתיות, אימות זהות ממשלתית וזאת מתוך בחירה להתמקד במטרה המרכזית של הזמנת טיסה וסנכרון בין נוסעים.

**תיחום הפרויקט:**
הפרויקט עוסק בעיקר בתחום הרשות, אבטחת מידע וניהול נתונים יעיל במערכת ההפעלה. בפרויקט נעשה שימוש במספר פרוטוקולים של תקשורת:

- פרוטוקול UDP – מהיר, אין חיבור קבוע, המידע לא בהכרח מגיע, המידע לא בהכרח מגיע בסדר הנכון.
- פרוטוקול TCP – איטי יותר מ-UDP, מצריך חיבור יציב ודו-כיווני, מוודא שהנתונים מגיעים בשלמותם ובסדר הנכון, בודק ומתקן שגיאות לפי הצורך.

- HTTPS – מתבסס על פרוטוקול HTTP בשילוב עם אבטחה והצפנה, שניהם מבוססים על פרוטוקול TCP אך מטרתם ליצור תקשורת בין דפדפן או אפליקציה לשרת אינטרנט.

על מנת להגן על התקשורת מפני תוקפים חשוב לאבטח אותה בעזרת הכלים האלו:

- AES (Advanced Encryption Standard) – אלגוריתם נפוץ להצפנת מידע בצורה סימטרית, כלומר שני הצדדים שמקבלים את המידע משתמשים במפתח זהה כדי להצפין ולפענח אותו.
- RSA – אלגוריתם נפוץ נוסף להצפנת מידע, אך בצורה א-סימטרית, כלומר לכל אובייקט RSA יש מפתח פומבי שחשוף לכולם ומפתח פרטי שלא חשוף לאף אחד.
- Hashing – מנגנון של הצפנת מידע רגיש מאוד באמצעות הפיכת כל פיסת מידע למחרוזת אקראית לחלוטין באורך קבוע, כך שלא ניתן לפענח את המספר על פי אורך התוצאה.
- Salting – פעולת הוספת מחרוזת אקראית באורך מסוים למסר לפני פעולת Hashing כך שלא תהיה דרך לפענח את המידע שנשלח בסיום ההצפנה.

בפרויקט נעשה שימוש במערכת ההפעלה Windows אשר מריצה את התוכנה ומאפשרת פתיחת החלון הוויזואלי של Windows Forms. המערכת משתמשת ביכולות של Windows לניהול תהליכים רבים באותו הזמן (multithreading) כך שכל חיבור של לקוח לשרת מנוהל בתהליך נפרד, מה שמאפשר ריבוי פעולות בו זמנית במקביל וללא חסימה. כך למשל ההאזנה של השרת והלקוחות להודעות שנשלחות אחד לשני יכולה להתבצע תוך כדי פעולות אחרות שקשורות לעדכון העיצוב בחלון.

מערכת CybAir אינה פועלת על מערכות הפעלה אחרות כגון Linux או macOS, מה שמגביל אותה לסביבת Windows בלבד. בנוסף התקשורת של המערכת היא רק מקומית ואין שימוש בפרוטוקול HTTPS למטרת הרחבת התקשורת לאינטרנט הבינלאומי.

**תיאור מפורט של המערכת:**

עבור כל משתמש שמפעיל את המערכת מתבצעת תחילה ההחלטה האם הוא זכאי להיות השרת או לא. ההחלטה לא תלויה במשתמש אלא מתבצעת על בסיס מקום פנוי. המערכת מציעה לעיין בעמוד הבית ולהתרשם מתמונות ומקומות יפים בעולם שמהווים יעדים של טיסות קיימות במאגר, ניתן לעיין בעמוד "אודות" ולקרוא על החברה CybAir במטרה ללמוד ולהכיר אותה. ניתן להירשם למערכת וליצור חשבון חדש במידה ואין, וניתן להתחבר לחשבון קיים במידה ויש. לאחר הירשמות או התחברות באמצעות הזנת פרטים, נפתחות אפשרויות

רבות: ניתן למלא את הטופס של חיפוש הטיסה ולקבל את כל הטיסות הזמינות במאגר אשר מתאימות לנתונים שנבחרו. לאחר בחירת הטיסה ניתן לבחור את המושבים הפנויים בהתאם למחלקה ומספר הנוסעים שנבחרו ולבסוף ניתן לבצע פעולת דמוי תשלום בהתאם למחיר הטיסה. המערכת מאפשרת לגשת לכרטיסי טיסה שהוזמנו על ידי המשתמש לעיין בהם לבטל אותם ולהוריד למחשב אם יש צורך. דרך הכרטיסים ניתן להיכנס למפת העולם האינטראקטיבית שבה מוצגים המשתמשים האחרים, שהסכימו להופיע, ליד עיר מגוריהם. הדבר מאפשר לשאר הנוסעים להכיר מעט את יתר הנוסעים לפני הטיסה כך שהיא יכולה להפוך גם למחברת ומגבשת. בכל שלב שבו המשתמש מחובר הוא יכול להתנתק בביטחון כך שכל הנתונים שלו נשמרים. במידה ויש צורך המערכת מציעה לפנות לתמיכה ולשוחח עם השרת בשיח דיגיטלי.

**פירוט היכולות לפי סוג משתמש**

**שרת:**

- ניהול חיבורים עם לקוחות
- בדיקה ואישור הרשמה
- בדיקה ואישור התחברות
- שליחת טיסות בהתאם לחיפוש הלקוח
- בדיקה ואישור הזמנת טיסה
- שליחת ההזמנות של לקוח שמבקש
- שיחוח עם כל אחד מהמשתמשים הרשומים באופן מוצפן בצ'אט
- סינון בקשות לא חוקיות או שגויות

**לקוח:**

- עיון ומעבר על תמונות בעמוד הבית
- קריאת המידע על החברה והמערכת CybAir
- הרשמה למערכת
- התחברות למערכת
- חיפוש טיסות זמינות באמצעות מילוי טופס
- בחירת המושבים בהתאם לנתוני הטופס
- תשלום בהתאם לטיסה שנבחרה (לא מאומת)
- צפייה בכרטיסי הטיסה
- ביטול הזמנה
- שמירת הכרטיס כקובץ

- צפייה בפרטים לא רגישים של הנוסעים האחרים בטיסה על מפה אינטראקטיבית

**פירוט הבדיקות:**

1. בדיקת זכאות להיות שרת:

הבדיקה שתתבצע אמורה לבדוק שלא יכול להיווצר מצב שבו שני משתמשים הם השרת. הבדיקה תתבצע כך שיהיו מספר ניסיונות להפעיל את המערכת, גם לאחר זמן קצר וגם לאחר זמן ארוך, באותו הזמן כדי לדמות מצב של כמה משתמשים פעילים ולוודא שהמערכת מאפשרת רק שרת אחד.

2. בדיקת תקינות רישום – הבדיקה מתבצעת גם בלקוח וגם בשרת כדי לוודא שמדובר בנתונים תקינים. בצד לקוח הבדיקה תהיה למלא את הטופס במגוון דרכים כך שבכל פעם שדה אחד אינו תקין, רק לאחר מכן הטופס ימולא באופן תקין והבדיקה תעבור לצד שרת. כדי לבדוק את צד השרת הטופס ימולא מספר פעמים עם נתונים של משתמש זהה ופעם אחת עם פרטים של משתמש חדש.
3. בדיקת תקינות התחברות – הבדיקה שתתבצע אמורה לבדוק שמשתמש שמזין פרטי משתמש תקינים מצליח להתחבר רק אם הוא לא מחובר כבר ממחשב אחר. אם הפרטים שהמשתמש מזין אינם מתאימים לאף משתמש קיים הוא לא יוכל להתחבר. הבדיקה תתבצע כך שבמספר הרצות של המערכת יוזנו פרטים שגויים, בהרצה אחת תהיה הזנה של פרטים נכונים שתאפשר התחברות ואז יוזנו אותם פרטים במספר הרצות כדי לוודא שהמערכת לא מאפשרת לאותו משתמש להתחבר יותר מפעם אחת.
4. בדיקת תקינות התנתקות מהחשבון – הבדיקה שתתבצע אמורה לבדוק שלאחר התנתקות מהחשבון החיבור TCP לא פעיל יותר וניתן להירשם ולהתחבר מחדש. הבדיקה תתבצע בכך שיהיו מספר התחברויות והתנתקויות ממספר הרצות שונות וחלונות שונים במערכת.
5. בדיקת תקינות הזמנת טיסה – הבדיקה שתתבצע אמורה לבדוק שמשתמש מצליח להזמין את המקומות שבחר והם מתעדכנים במערכת ושמשתמש לא יכול להזמין את אותה הטיסה אם כבר הזמין אותה. הבדיקה תתבצע כך שיהיו מספר הזמנות של טיסות שונות ואז הזמנות של אותן טיסות, כל הזמנה תהיה מפוקחת ותיבדק בקובץ ההזמנות.
6. בדיקת תקינות עמוד יצירת הקשר – הבדיקה שתתבצע אמורה לבדוק שאין תקלות ביצירת הקשר עם השרת, ההודעות נשמרות, המידע מועבר בזמן אמת, הלקוח יכול לעבור בין חלונות ולחזור לצ'אט ושמספר משתמשים יכולים לתקשר עם השרת בו זמנית. הבדיקה תתבצע כך שבמספר הרצות שונות יהיו ניסיונות לשלוח לשרת הודעות, לסגור את החלון, להתנתק, לעבור חלון, לחזור לחלון הצ'אט, להתחבר מחדש.
7. בדיקת תקינות ההצפנה – הבדיקה שתתבצע אמורה לבדוק שהמידע שעובר ברשת בחיבור TCP אכן מוצפן ולא ניתן לראות את המידע החשוף. הבדיקה תתבצע כך שבזמן

הרצה ההודעות יודפסו לפני הצפנה ואחרי פענוח כדי לראות שיש תיאום ויהיה שימוש בתוכנת WireShark שמאפשרת לראות את המידע שעובר ברשת.

**לוח זמנים:**

| חודש | בפועל | תכנון |
|---|---|---|
| **ספטמבר** | חקרתי על פרוטוקולים של תקשורת, הבנתי איך שרתים מתקשרים עם לקוחות והחלטתי לבנות מערכת של הזמנת טיסות. | ללמוד רקע תאורטי על רשתות ,TCP/UDP להבין איך מערכות כאלה עובדות ולחשוב על רעיון לפרויקט. |
| **אוקטובר** | התחלתי להתנסות בשפת התכנות #C ולבדוק אפשרויות יצירתיות בשפה להרחבת הפרויקט | להכיר את שפת התכנות #C, לחשוב על רעיונות ספציפיים בתוך הפרויקט |
| **נובמבר** | בניתי את עמוד הבית, ועמוד אודות | לבנות צד לקוח שלא תלוי בחיבור לשרת |
| **דצמבר** | בניתי עמוד רישום ללא שליחת נתונים ועמוד רישום ללא שליחת נתונים | להמשיך לפתח את צד הלקוח ולחבר אותו לשרת |
| **ינואר** | יצרתי לולאות תקשורת ב - TCP בין השרת ללקוח, והתחלתי לעבוד גם על שליחה ובקרה | לעבוד על תקשורת רציפה בין הלקוח לשרת |
| **פברואר** | אפשרתי לכל לקוח להתחבר, ליצור ולהתנתק מהחשבון באמצעות TCP וניהול המידע בשרת | לאפשר התחברות והתנתקות מחשבון ויצירת חשבון באמצעות השרת |

| חודש | בפועל | תכנון |
|---|---|---|
| **מרץ** | מצאתי מאגרי נתונים מתאימים של טיסות ושדות תעופה עבדתי ושיניתי חלק מהמידע כדי שיתאים למערכת. בניתי את עמוד הזמנת הטיסה ועמוד "ההזמנות שלי" | לטפל בנושא הזמנת הטיסות בצד לקוח ובצד שרת |
| **אפריל** | הוספתי את המפה האינטראקטיבית לעמוד "ההזמנות שלי" והוספתי צ'אט בין הלקוח לשרת | להמשיך להוסיף כלים יצירתיים ומיוחדים למערכת |
| **מאי** | כתבתי את ספר הפרויקט ביצעתי תיקונים בדקתי מקרי קיצון ושיפרתי חלקים מסוימים | לכתוב את כל ספר הפרויקט ולהתכונן להגשה. |
| **יוני** | ווידאתי שהפרויקט עובד כמו שצריך והגשתי אותו | לוודא שהפרויקט עובד כראוי ולהגיש אותו |

**ניהול הסיכונים בפרויקט:**

| סיכון | התמודדות מתוכננת | מה קרה בפועל |
|---|---|---|
| החיבור נקטע בצורה לא מסודרת | ליצור מנגנון ניתוק מסודר שמבוסס על בדיקות קבועות על כל לקוח באמצעות אחד מהפרוטוקולים TCP/UDP | השתמשתי בפעימות קבועות של פרוטוקול UDP ותקשורת קבועה בין השרת ללקוח כדי לזהות שהלקוח לא מגיב ומנותק |
| שגיאות בקריאה או שליחה של המידע דרך TCP | שימוש בטכניקות Error Handling | עשיתי שימוש ב-try ו-catch בהתאם למסר והצגתי הודעות מתאימות |
| תקיפות man in the middle שבהן ניתן לשלוף את המידע ואף לשנות אותו בזמן המעבר בין השרת ללקוח | הצפנת המידע שנשלח ואימות ובדיקה של ההודעות שמתקבלות | השתמשתי בשילוב של הצפנה סימטרית AES וא-סימטרית RSA וביצעתי בדיקות רבות כדי לוודא שלא בוצעו שינויים להודעה |
| פריצה למאגר הנתונים של השרת וקבלת גישה לסיסמאות האישיות של המשתמשים | הצפנה מיוחדת עבור הסיסמאות ושמירתן מוצפנות | לכל סיסמה הוספתי Salt (מחזרות אקראית) וביצעתי פעולת Hash כדי שלא תהיה אפשרות לפענח את הסיסמה ושמרתי אותה מוצפנת במאגר הנתונים |

**צד שרת**

1. **שם היכולת:** ניהול חיבורים עם לקוחות

**מהות:** טיפול בכל לקוח שמתחבר למערכת באמצעות יצירת חיבור TCP להעברת מידע באופן ישיר כל עוד הלקוח מחובר

**אוסף פעולות נדרשות:**

- קבלת חיבור באמצעות TcpListener
- יצירת thread נפרד לכל לקוח
- שמירת הלקוח ברשימת חיבורים פעילים
- בדיקה קבועה שהחיבור בין הלקוח לשרת עדיין פעיל ותקין

**אובייקטים נחוצים:** TcpListener, TcpClient, רשימת חיבורים פעילים, UdpClient

2. **שם היכולת:** בדיקה ואישור הרשמה

**מהות:** אימות הנתונים של משתמש חדש, הצפנת סיסמה, ושמירה במערכת

**אוסף פעולות נדרשות:**

- קבלת פרטי הרשמה דרך חיבור ה-TCP עם המשתמש ופענוח
- בדיקת תקינות נתונים
- הצפנת הסיסמה עם Salt ו-Hash
- שמירת המשתמש במבנה נתונים
- עדכון רשימת החיבורים הפעילים (אם ההתחברות מוצלחת)
- הצפנת תגובת הצלחה/כשל ושליחה

**אובייקטים נחוצים**: users.json, Validator, Hashing (SHA256), Salt StreamReader, StreamWriter, רשימת חיבורים פעילים, פענוח והצפנה

3. **שם היכולת:** בדיקה ואישור התחברות

**מהות:** אימות ובדיקת פרטי משתמש מול הנתונים השמורים

**אוסף פעולות נדרשות:**

- קבלת פרטי התחברות דרך חיבור ה-TCP עם המשתמש
- חיפוש והשוואה במאגר נתוני המשתמשים
- הצפנת הסיסמה שהתקבלה עם Salt ו-Hash והשוואה
- עדכון רשימת החיבורים הפעילים (אם ההתחברות מוצלחת)
- הצפנת תגובת הצלחה/כשל ושליחה

**אובייקטים נחוצים:** users.json, Validator, Hashing (SHA256), Salt StreamReader, StreamWriter, רשימת חיבורים פעילים, פענוח והצפנה

4. **שם היכולת:** שליחת טיסות בהתאם לחיפוש הלקוח

**מהות:** סיפוק המידע שמאפשר למשתמש להזמין את הטיסה

**אוסף פעולות נדרשות:**

- קבלת פרטי חיפוש הטיסה דרך חיבור ה-TCP עם המשתמש ופענוח
- חיפוש והשוואה במאגר נתוני הטיסות הפעילות
- בדיקה האם יש מספיק מקום בטיסה למספר התושבים הרצוי
- הצפנת רשימת הטיסות המתאימות לחיפוש המשתמש ושליחה

**אובייקטים נחוצים**:ActiveFlights.json StreamReader ,Flights.csv, StreamWriter, פענוח והצפנה

5. **שם היכולת:** בדיקה ואישור הזמנת טיסה

**מהות:** בדיקה שהמושבים שנבחרו אכן תקינים ופנויים ושניתן לעדכן את נתוני הטיסה והזמנות המשתמש

**אוסף פעולות נדרשות:**

- קבלת פרטי המושבים שנבחרו ושמות הנוסעים דרך חיבור ה-TCP עם המשתמש ופענוח
- חיפוש במאגר הטיסות של <u>המשתמש</u> ובדיקה שהטיסה שנבחרה לא הוזמנה כבר
- בדיקה האם המושבים שנבחרו בטיסה עדיין פנויים ולא נתפסו בזמן ההזמנה
- עדכון מאגר הטיסות של המשתמש ומאגר הטיסות הכללי
- הצפנת תגובת הצלחה/כשל ושליחה

**אובייקטים נחוצים:** StreamReader ,ActiveFlights.json , StreamWriter, users.json, פענוח והצפנה

6. **שם היכולת:** שליחת ההזמנות של לקוח שמבקש

**מהות:** העברת המידע הדרוש למשתמש על כל ההזמנות שהוא ביצע כדי שיוכל להציג את כרטיסי הטיסה כשצריך

**אוסף פעולות נדרשות:**

- הצפנת הנתונים ממאגר הטיסות של המשתמש ושליחה

**אובייקטים נחוצים:** StreamReader , StreamWriter, users.json, הצפנה

7. **שם היכולת:** ביטול כרטיס טיסה

**מהות:** הוצאת המשתמש ממאגר הנוסעים של הטיסה שביקש

**אוסף פעולות נדרשות:**

- הוצאת המשתמש ממאגר הנוסעים של הטיסה
- הוצאת הטיסה ממאגר הטיסות של המשתמש

**אובייקטים נחוצים:** StreamReader , StreamWriter, users.json, הצפנה

8. **שם היכולת:** שיחוח עם כל אחד מהמשתמשים בצ'אט מוצפן

**מהות:** שליחת הודעות מהשרת ללקוח כדי לעזור ללקוח או לענות על שאלות באופן מוצפן, ושמירה על סדר ההודעות

**אוסף פעולות נדרשות:**

**שליחה הודעה ללקוח:**

- לחיצה על המשתמש שאליו יש לשלוח הודעה
- הקלדת הודעה בתיבת הטקסט textbox
- לחיצה על כפתור השליחה
- הצגת ההודעה על חלון ההודעות של השרת
- שמירת ההודעה במאגר ההודעות של המשתמש
- חיפוש במאגר החיבורים הפעילים את חיבור ה-TCP וההצפנה הסימטרית AES עם המשתמש
- הצפנת ההודעה ושליחה

**קבלת הודעה מהלקוח:**

- קבלת ההודעה מהמשתמש דרך חיבור ה-TCP עם המשתמש ופענוח
- הצגת ההודעה על חלון ההודעות של השרת
- שמירת ההודעה במאגר ההודעות של המשתמש

**אובייקטים נחוצים:** StreamWriter, users.json, רשימת חיבורים פעילים, פענוח/הצפנה, כפתור שליחה, תיבת טקסט, חלון הודעות

**צד לקוח**

1. **שם היכולת:** עיון ומעבר על תמונות בעמוד הבית

**מהות:** אפשרות לעבור בין תמונות ולקבל טעימה מהיעדים שהמערכת מציעה

**אוסף פעולות נדרשות:**

- טעינת תמונות

- שליטה על מעבר בין תמונות

**אובייקטים נחוצים:** חלון לעמוד בית, כפתורי ניווט, קבצי התמונות

2. **שם היכולת:** קריאת מידע על החברה והמערכת

**מהות:** הצגת הסבר על CybAir בעמוד מידע מיוחד.

**אוסף פעולות נדרשות:**

- יצירת מסך מידע
- הצגת טקסטים ותמונות

**אובייקטים נחוצים:** חלון לעמוד אודות, קבצי התמונות, טקסט

3. . **שם היכולת:** הרשמה למערכת

**מהות:** הכנסת פרטי משתמש לטופס ושליחתם לשרת כדי ליצור משתמש חדש

**אוסף פעולות נדרשות:**

- מילוי טופס
- ולידציה
- הצפנת הנתונים ושליחתם לשרת
- קבלת תשובה ופענוח

**אובייקטים נחוצים:** חלון לעמוד הרשמה, טופס עם שדות למילוי, כפתור שליחה, הצפנה ופענוח, StreamReader, StreamWriter

4. **שם היכולת:** התחברות למערכת

**מהות:** הכנסת פרטי משתמש לטופס ושליחתם לשרת כדי להתחבר לחשבון קיים

**אוסף פעולות נדרשות:**

- מילוי טופס
- ולידציה
- הצפנת הנתונים ושליחתם לשרת
- קבלת תשובה ופענוח

**אובייקטים נחוצים:** חלון לעמוד התחברות, טופס עם שדות למילוי, כפתור שליחה, הצפנה ופענוח, StreamReader, StreamWriter

5. **שם היכולת:** התחברות למערכת

**מהות:** הכנסת פרטי משתמש לטופס ושליחתם לשרת כדי להתחבר לחשבון קיים

**אוסף פעולות נדרשות:**

- מילוי טופס
- ולידציה
- הצפנת הנתונים ושליחתם לשרת
- קבלת תשובה ופענוח

**אובייקטים נחוצים:** חלון לעמוד התחברות, טופס עם שדות למילוי, כפתור שליחה, הצפנה ופענוח, StreamReader, StreamWriter

6. **שם היכולת:** חיפוש טיסות

**מהות:** הכנסת פרטי טיסה לטופס ושליחתם לשרת כדי לקבל אפשרויות הזמנה שונות

**אוסף פעולות נדרשות:**

- מילוי טופס
- ולידציה
- הצפנת הנתונים ושליחתם לשרת
- קבלת תשובה ופענוח

**אובייקטים נחוצים:** חלון לעמוד חיפוש טיסות, טופס עם שדות למילוי, כפתור שליחה, הצפנה ופענוח, StreamReader, StreamWriter

7. **שם היכולת:** בחירת מושבים

**מהות:** המשתמש בוחר את המושבים בטיסה לפי מספר הנוסעים בהזמנה, מתוך המושבים הפנויים במבנה דמוי מטוס בחלון להזמנת טיסות

**אוסף פעולות נדרשות:**

- לחיצה על כפתורי המושבים

**אובייקטים נחוצים:** חלון לעמוד הזמנת טיסות, מבנה של מטוס, כפתורים המדמים מושבים

8. **שם היכולת:** תשלום (לא מאומת)

**מהות:** המשתמש ממלא את שמות שאר הנוסעים בהזמנה ואת פרטי התשלום

**אוסף פעולות נדרשות:**

- מילוי טופס השמות של שאר הנוסעים בהזמנה
- מילוי פרטי התשלום בטופס
- ולידציה של השמות ופרטי התשלום
- הצפנת המידע של ההזמנה (טיסה, מושבים ושמות) ושליחה לשרת

- קבלת תשובה ופענוח

**אובייקטים נחוצים:** חלון לעמוד הזמנת טיסות, טופס עם שדות למילוי, כפתור שליחה, StreamWriter, StreamReader, הצפנה ופענוח

9. **שם היכולת:** צפייה בכרטיסי טיסה

**מהות:** המשתמש יכול להסתכל על פרטי ההזמנות שלו שהן גם כרטיסי הטיסה שאותם ניתן להציג בשדה התעופה

**אוסף פעולות נדרשות:**

- הצפנת בקשה לקבלת הזמנות ושליחה לשרת
- קבלת הזמנות מהשרת ופענוח
- הצגת הכרטיסים בחלון ההזמנות

**אובייקטים נחוצים:** חלון לעמוד "ההזמנות שלי", StreamWriter, StreamReader, הצפנה ופענוח

10. **שם היכולת:** ביטול הזמנה

**מהות:** המשתמש יכול לבטל את הכרטיסים שהוא הזמין ובכך לפנות מקום במטוס

**אוסף פעולות נדרשות:**

- לחיצה על כפתור הביטול שעל הכרטיס
- הצפנת בקשה לביטול הטיסה ושליחה לשרת
- קבלת הזמנות מהשרת ופענוח
- עדכון והצגת הכרטיסים בחלון ההזמנות

**אובייקטים נחוצים:** חלון לעמוד "ההזמנות שלי", כרטיס טיסה, StreamWriter, StreamReader, הצפנה ופענוח, כפתור ביטול

11. **שם היכולת:** שמירת כרטיס כקובץ

**מהות:** המשתמש יכול לשמור את הכרטיס בקובץ ולאחר מכן להדפיס אותו אם יש צורך

**אוסף פעולות נדרשות:**

- לחיצה על כפתור השמירה שעל הכרטיס
- פתיחת חלון שמירת הקובץ ושמירה

**אובייקטים נחוצים:** חלון לעמוד "ההזמנות שלי", SaveFileDialog, הצפנה ופענוח, כפתור שמירה, כרטיס טיסה

12. **שם היכולת:** צפייה בפרטים לא רגישים של נוסעים בטיסה

**מהות:** הצגת מפת העולם אינטראקטיבית שבה מופיע מידע של נוסעים אחרים בטיסה, שהסכימו שייחשף למטרות היכרות. המידע כולל שם גיל ועיר מגורים. בנוסף במפה מופיע מסלול הטיסה מעיר המוצא לעיר היעד.

**אוסף פעולות נדרשות:**

- לחיצה על כרטיס הטיסה
- פתיחת חלון המפה האינטראקטיבית
- הצפנת הבקשה לקבלת הנתונים ושליחה לשרת
- קבלת המידע מהשרת ופיענוח
- הצגת המידע על המפה

**אובייקטים נחוצים:** כרטיס טיסה, StreamWriter, StreamReader, הצפנה ופענוח, כפתור ביטול, GMap.Net

## מבנה / ארכיטקטורה של הפרויקט

**תיאור חומרה**

במערכת CybAir רכיבי החומרה של המחשב ממלאים תפקיד מרכזי בביצועים. כל רכיב תורם בצורה שונה להרצת המערכת, להצגת המידע בזמן אמת ולעיבוד התקשורת ברקע. להלן מוצגים רכיבי החומרה, שהקשר ביניהם מאפשר למערכת לפעול כראוי:

- המעבד (CPU - Central Processing Unit): רכיב חומרה במחשב שמבצע את הפקודות המאוחסנות בזיכרון המחשב. הפקודות אותן מקבל המעבד הן פקודות אלמנטריות ביותר, כשכל פקודה בנויה מרצף קצר של ביטים. רצף זה קרוי שפת מכונה. כל דגם של מעבד מתאפיין בסט הפקודות שלו. יכולותיו של המעבד כוללות קריאת מידע מהזיכרון או מהתקנים שונים, ביצוע פעולות מתמטיות ולוגיות על מידע זה וכתיבת תוצאות החישוב בחזרה לזיכרון או לחלופין שליחתו להתקנים חיצוניים. יכולת זו מאפשרת למעבד גמישות רבה מספיק כדי לשמש למגוון רחב ביותר של יישומים. במערכת CybAir, המעבד אחראי על הרצת כל הפעולות, ניהול תהליכים מקבילים (Threads), ניהול התקשורת בחיבורי TCP/UDP , קריאת קבצים מהזיכרון ואינטראקציה עם רכיבים נוספים.

- RAM – Random Access Memory : זיכרון אחסון מהיר שבו המעבד משתמש כדי לאחסן קבצים זמניים על מנת שיהיו מוכנים לשליפה מהירה לעומת הכוננים הקשיחים. כל המידע שנאגר מעבודה עם תוכנות או קבצים מסוימים במחשב, גלישה באינטרנט ואפילו צפייה בסרטונים מאוחסן בזיכרון ה RAM -לשימוש בעת הצורך. הגישה למידע והנתונים שנמצאים בדיסק הקשיח לוקחת הרבה זמן יחסית, אז כדי לזרז תהליכים יש את זיכרון ה- RAM שהוא בעצם "זיכרון לטווח קצר". במערכת CybAir, זיכרון ה-RAM אחראי על שמירת הנתונים בזמן ההרצה, עבודה מול קבצים מעודכנים, ועבודה עם אובייקטים שונים כגון משתמש חדש או טיסה חדשה.

- כרטיס מסך (GPU - Graphics Processing Unit) : כרטיס המסך במחשב אחראי לייצר את הפלט הוויזואלי מהמחשב על גבי המסך. למעשה, כמעט כל התוכניות מייצרות פלטים ויזואליים וכרטיס המסך הוא רכיב החומרה האחראי לקחת פלט זה ולהורות למסך אילו נקודות להאיר על הצג (ובאיזה צבע) כדי שנוכל לראות את הפלט. כרטיס המסך מקבל כקלט מהמעבד נתונים בינאריים והופך אותם לאות תצוגה (בעבר אנלוגי בלבד, כיום גם ספרתי). במערכת CybAir כרטיס המסך אחראי על ניהול ועדכון הממשק הוויזואלי באנימציות, תמונות או אלמנטים גרפיים מורכבים כגון מפת העולם בעמוד ההזמנות

.

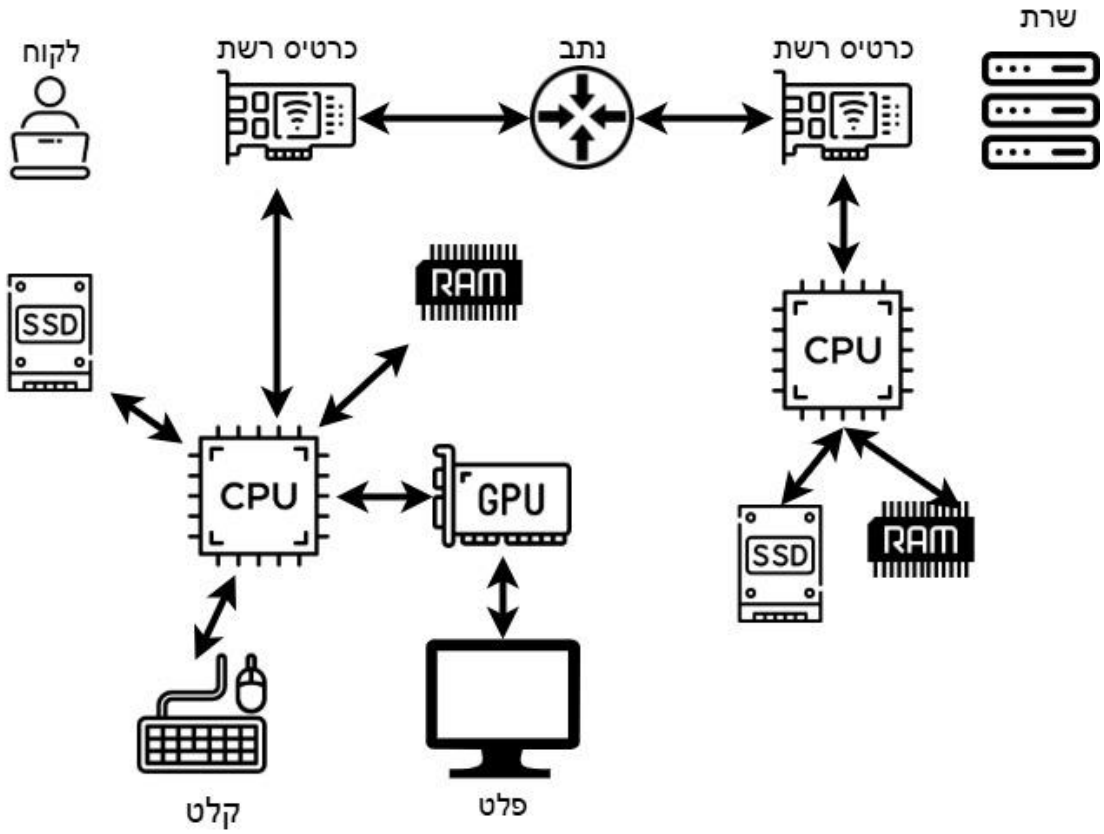
- כרטיס רשת (Network Interface Card – NIC): ידוע גם כמתאם רשת או מתאם LAN, הוא רכיב מרכזי במחשב המאפשר לו להתחבר לרשתות ולמכשירים אחרים- באמצעות Ethernet(חיבור קווי) או Wi-Fi (חיבור אלחוטי). לכל כרטיס רשת יש כתובת MAC ייחודית ,שבעזרתה ניתן לזהות את המחשב ברשת. זה חשוב במיוחד לניהול נכון של כתובות, אבטחה ותקשורת. במערכת CybAir ה- NIC מאפשר את כל התקשורת בין השרת ללקוחות באמצעות פרוטוקולים כמו TCPאו UDP. ללא כרטיס רשת תקין המערכת לא יכולה לתקשר עם השרת.

- נתב (Router): הנתב מכוון את תנועת כל משאבי המידע ברשת ממקום למקום. הוא מאפשר העברת נתונים בין רשתות בכלל והעברת נתונים מרשת האינטרנט אל הרשת הביתית. המערכת CybAir, שבה יש שרת אחד שמתקשר עם לקוחות רבים דרך רשת מקומית (LAN), הנתב הוא רכיב מרכזי שמאפשר את כל התקשורת הזו להתבצע בצורה חלקה ומדויקת.

- התקני קלט ופלט (I/O – Input/Output): אמצעים פיזיים שמאפשרים למשתמש לתקשר עם המחשב, ולקבל ממנו תגובה. האמצעים הפיזיים כוללים: מקלדת, עכבר, מסך מגע, מצלמה ומיקרופון לקלט, מסך ורמקולים לפלט. במערכת CybAir, התקני קלט ופלט הם הגשר בין המשתמש לבין התוכנה. הם מאפשרים להזין מידע, לבצע בחירות, ולראות את תוצאות הפעולות.

- אחסון (SSD/HDD): המקום שבו המחשב שומר קבצים, תוכנות, תמונות, סרטים, מערכת הפעלה ועוד. הם כמו הספרייה המרכזית של המחשב. כל המידע שצריך להישמר גם אחרי כיבוי נשמר שם. במערכת CybAir, לא כל המידע נשמר רק ברגע שהמחשב דולק, אלא גם צריך לשמור מידע קבוע שנשמר גם לאחר כיבוי כמו מאגר הכתובות ומאגר המידע על הטיסות.

להלן שרטוט המציג את הקשרים בין רכיבי החומרה השונים:




**תיאור הטכנולוגיה הרלוונטית**

במהלך פיתוח הפרויקט השתמשתי בשתי שפות תכנות:

1. #C - שפת תכנות מונחית עצמים שפותחה על ידי חברת Microsoft כחלק מפלטפורמת NET. היא מיועדת בעיקר לפיתוח יישומים על מערכת ההפעלה Windows, ומתאימה במיוחד ליישומים בעלי ממשק גרפי. הבחירה ב-#C נובעת מהתמיכה הרחבה שהיא מציעה בפיתוח יישומי ממשק משתמש גרפי (GUI) באמצעות Windows Forms, ובגלל היכולות המתקדמות שלה בניהול תקשורת ברשת. גרסת ה-#C שבה נעשה שימוש היא: NET 9.
2. Python - שפת תכנות עילית, שמתמקדת בפשטות התחביר ובמהירות הפיתוח. היא שפת קוד פתוח, המתאימה למגוון רחב של יישומים. Python מצטיינת ביכולת שלה להתמודד עם קבצים ונתונים בצורה נוחה מאוד בזכות ספריות כמו Pandas. הבחירה בשפה זו הייתה למטרת עיבוד נתונים על טיסות אמיתיות שנלקחו מהאינטרנט ויצירת מאגר טיסות בהתאם לדרישות המערכת. גרסת ה-Python שבה נעשה שימוש היא: Python 3.10.0.

בפרויקט זה נעשה שימוש ב-Windows כמערכת ההפעלה הראשית, מכיוון שהיא תומכת ביישומי Windows Forms, מאפשרת עבודה נוחה עם קבצים, תיקיות ותקשורת רשת הכוללת אבטחה. גרסת ה-Windows שבה נעשה שימוש היא: Windows 10. השילוב בין מערכת ההפעלה לבין השפות שנבחרו אפשר פיתוח יעיל ומיטבי של מערכת ההזמנות CybAir.

במערכת CybAir נעשה שימוש רחב ברשתות על מנת ליצור תקשורת בין השרת ללקוחות:

UDP – פרוטוקול תקשורת קל ומשקל, שמעביר חבילות נתונים בלי להבטיח סדר או אמינות, מתאים למשלוח מהיר של מידע. בפרויקט זה הוא שימש לקבלת ההחלטה של מי זכאי להיות השרת ולביסוס התשתית של החיבור הישיר והמאובטח בינו לבין השרת באופן מהיר ויעיל תוך שיתוף מפתחות הצפנה. בנוסף פרוטוקול תקשורת זה אפשרת לשרת לבצע בדיקות בזמנים קבועים כדי לוודא שהלקוח עדיין מחובר ולפעול בהתאם לתגובה שלו. אם הוא מגיב סימן שהחיבור קיים אך אם הוא לא מגיב בזמן מוקצב סימן שהייתה התנתקות פתאומית ויש לסגור את החיבור בהתאם.

TCP – פרוטוקול תקשורת אמין שמאפשר העברת נתונים בסדר נכון בין מחשבים דרך רשת, ומוודא שכל החבילות מגיעות בלי אובדן או שגיאות. פרוטוקול זה איטי יותר מ-UDP בגלל הבדיקות שהוא מבצע עבור כל הודעה שנשלחה או התקבלה. בפרויקט, הוא שימש כחיבור ישיר ומאובטח בין השרת והלקוח להעברת מידע ומסרים שאחראים על הפעלת כל הפעולות במערכת. עבור כל פעולה או בקשה של הלקוח לקבלת מידע רגיש שנמצא בשרת הוא משתמש בחיבור היציב שהוגדר בשבילו כדי לשלוח ולקבל הודעות.

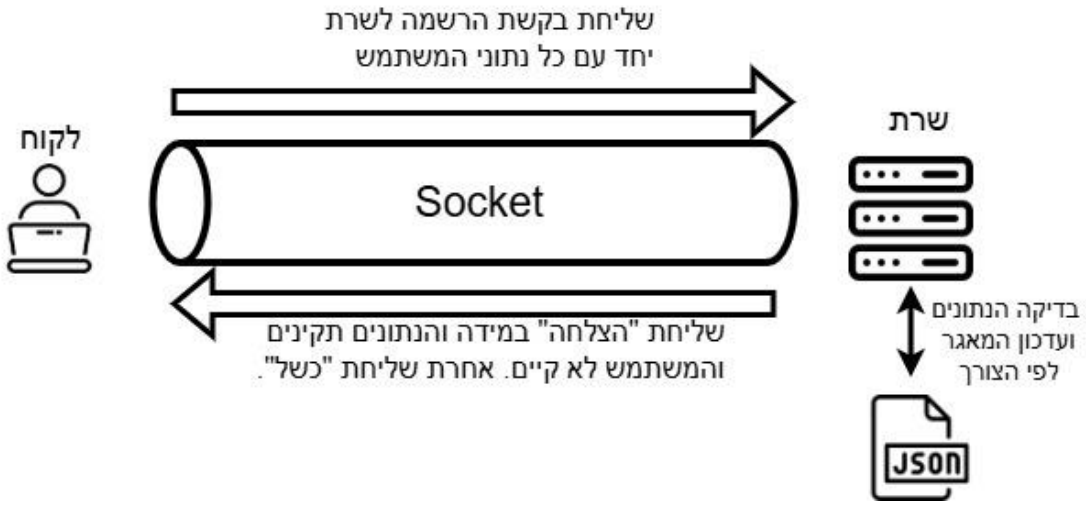
HTTPS – מתבסס על פרוטוקול תקשורת HTTP אך משלב גם הצפנה ואבטחה. הוא פועל באינטרנט המאפשר העברת מידע בצורה מאובטחת בין מחשב לשרת, על ידי הצפנת הנתונים כך שהם מוגנים מפני גישה או שינוי על ידי צד שלישי. בפרויקט הוא שימש לקבלת מידע מהמקור הפתוח של Google Maps ו-OpenStreetMap והצגת מפת העולם האינטראקטיבית באמצעות ספריית GMap.NET.

על מנת להגן על המידע העובר דרך הרשת המקומית והמידע שנשמר במאגר הנתונים נעשה שימוש בהצפנה מסוגים שונים:

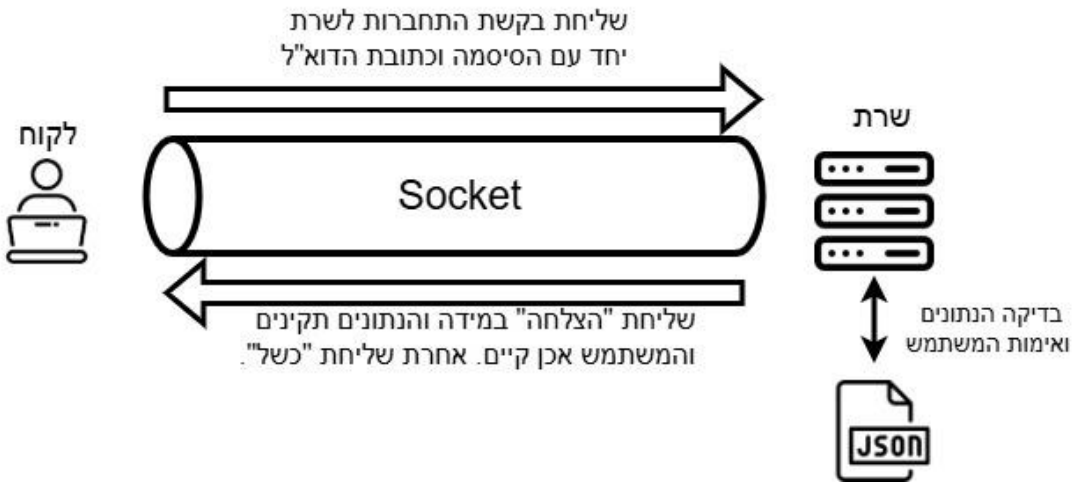
נעשה שימוש במנגנון הצפנה שמטרתו להגן על תקשורת ברשת על ידי הצפנת המידע, אימות זהות הצדדים והבטחת שלמות הנתונים. נעשה שימוש בהצפנה סימטרית AES בשילוב עם הצפנה א-סימטרית RSA כדי לאבטח את הנתונים שעוברים ברשת בצורה היעילה והמהירה ביותר. בנוסף נעשה שימוש בעקרונות Hashing ו-Salting שמטרתם להצפין את הסיסמאות של המשתמשים במיוחד. עקרון Hashing משתמש באלגוריתם מתמטי מורכב כדי לעוות לחלוטין כל ערך שהוא מקבל תוך שמירה על אורך קבוע. Salting הוא תהליך של הוספת סיומת של מחרוזת רנדומלית לחלוטין באורך מסוים לערך המקורי לפני אלגוריתם Hash כך שבסיום ההצפנה אין שום דרך לפענח את המידע המקורי.

**תיאור זרימת המידע במערכת**

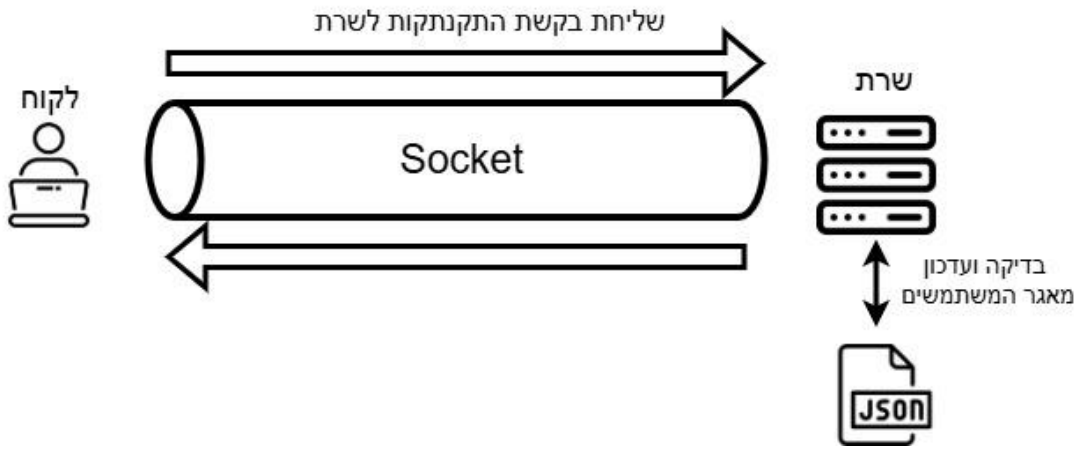
**הרשמה למערכת:**



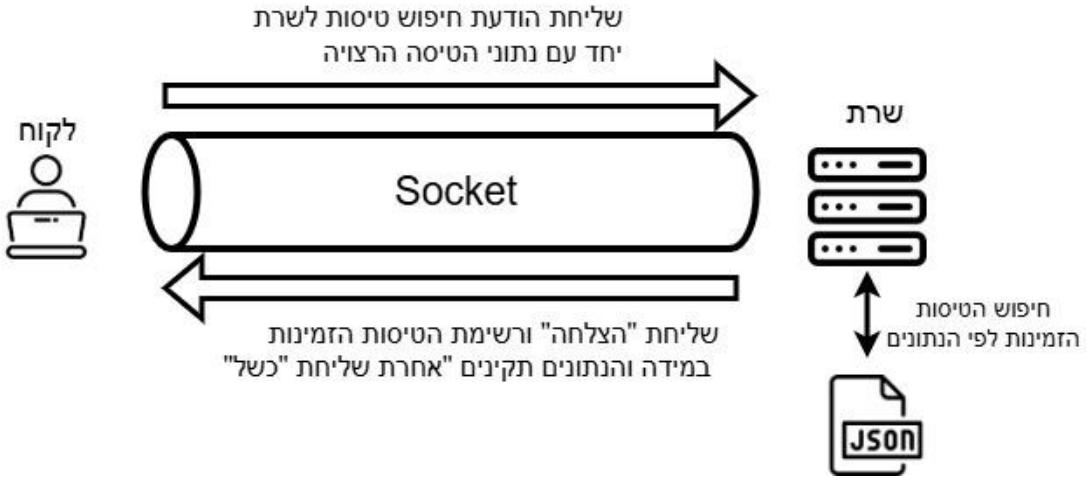

**התחברות למערכת:**



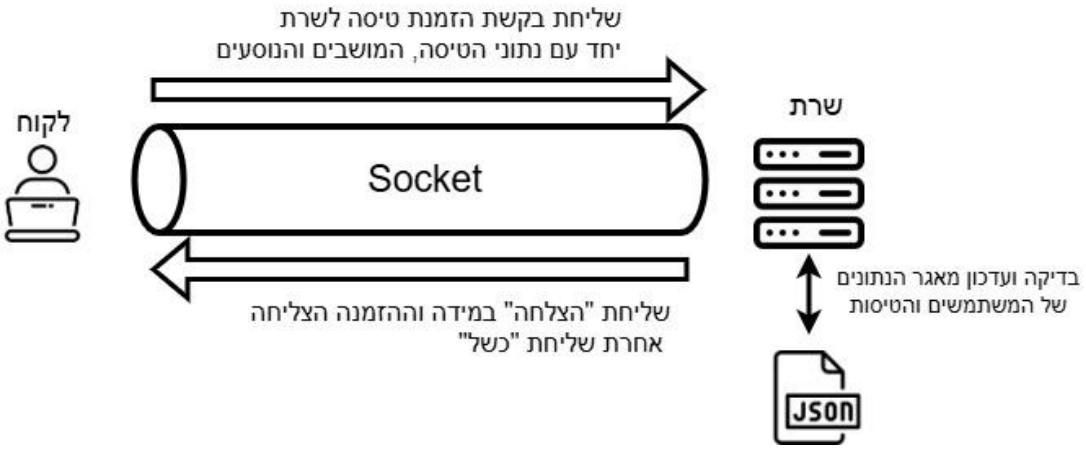
**התנתקות מהמערכת:**



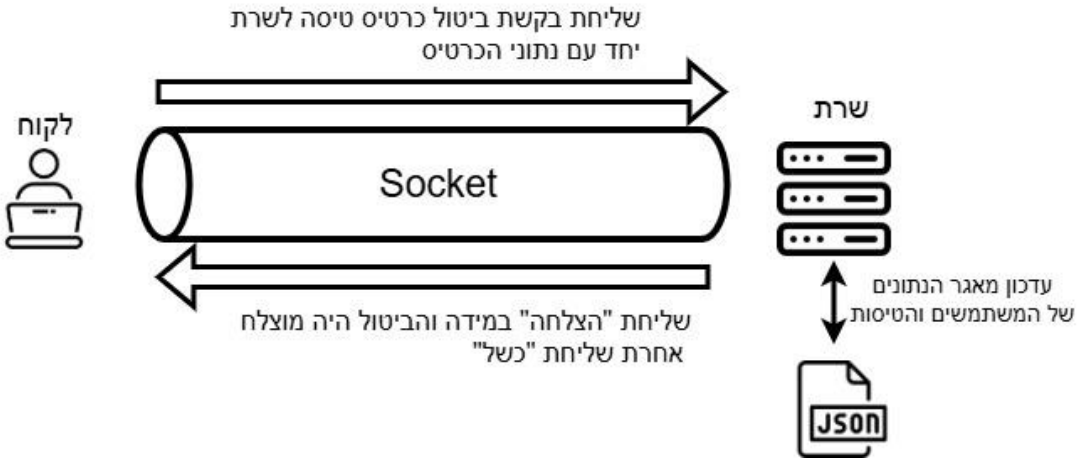

**חיפוש טיסות:**



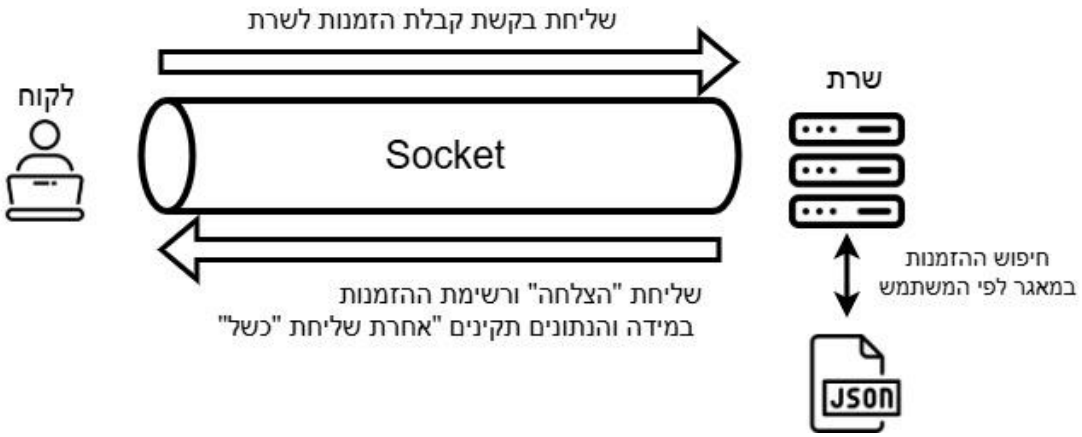

**הזמנת טיסה:**



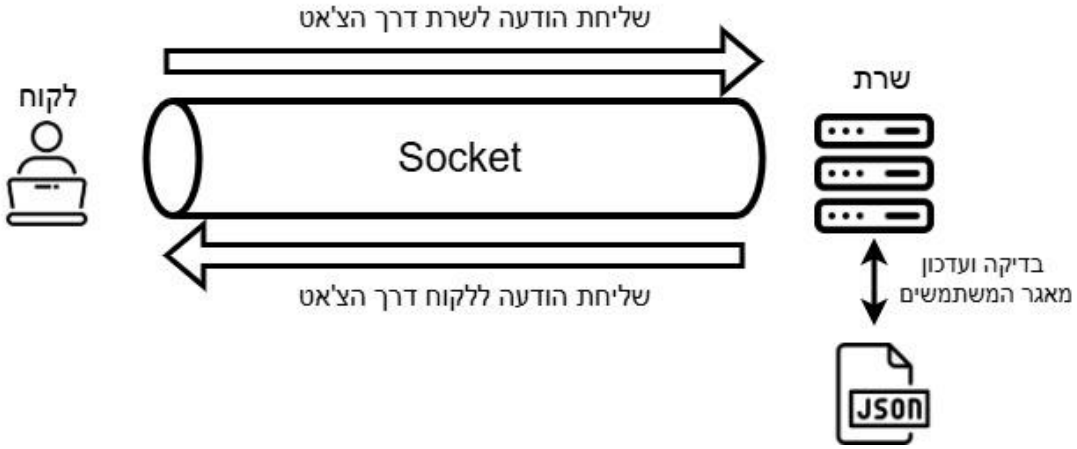
**ביטול טיסה:**



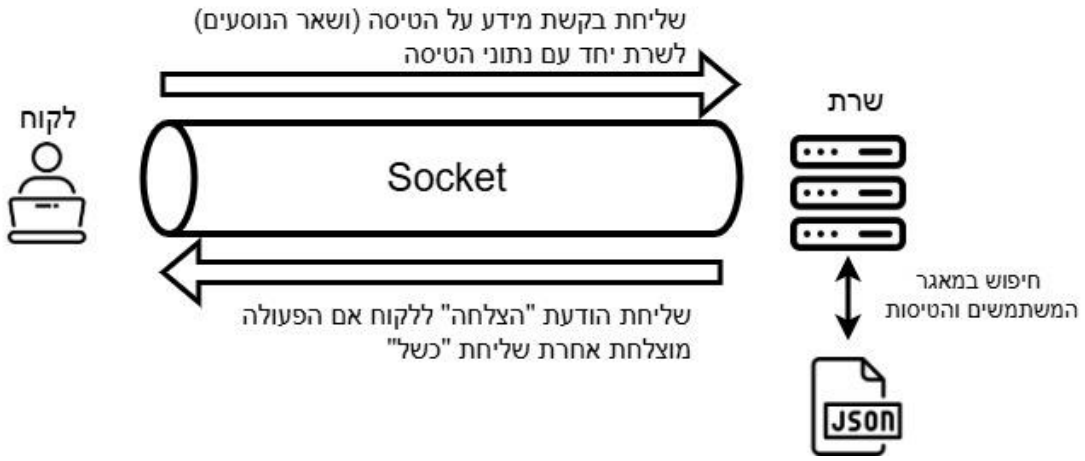

**קבלת הזמנות:**




**שיחה בין השרת ללקוח דרך הצ'אט:**



**קבלת מידע על הטיסה:**




**תיאור האלגוריתמים המרכזיים בפרויקט:**

בעיה אלגוריתמית: בחירת השרת ומניעת לקוחות אחרים מלקבל זכאות לתפקיד

אלגוריתמים קיימים לפתרון הבעיה:

1. הפתרון הפשוט ביותר, בו השרת מוגדר לפני הרצת המערכת לפי כתובת IP, סיסמה או שם משתמש. כך חלוקת התפקידים ברורה מראש ואין צורך בבקשה להיות השרת, אלא ניתן ישירות לבקש להתחבר אליו באמצעות פרוטוקול TCP.
2. פתרון נוסף הוא שימוש בקובץ משותף או בסיס נתונים לכל המשתמשים. הקובץ או בסיס הנתונים נמצא על מחשב נפרד. כל משתמש שמפעיל את המערכת מנסה לעדכן את הקובץ השיתופי, כך שרק המשתמש הראשון שמפעיל את המערכת שיכול לעדכן את הקובץ השיתופי ולכתוב את כתובת ה-IP. באמצעות כתובת ה-IP שלו כל הלקוחות שיבואו אחריו יוכלו להתחבר אליו באמצעות פרוטוקול TCP.
3. הפתרון המקובל האחרון הוא שימוש בפרוטוקול UDP כדי לבקש להיות השרת. פרוטוקול UDP לא מחייב חיבור יציב עם לקוח או שרת נוסף ולכן הוא מאפשר שליחת הודעות לכל המחשבים המחוברים ל-Port מסוים באמצעות "שידור". כל לקוח "משדר" את הבקשה שלו לשאר המחשבים ומחכה לתשובה. אם הוא לא מקבל תשובה שתפקיד השרת תפוס, בטווח זמן מסוים, הוא בעצמו הופך לשרת ומתחיל למנוע מאחרים לקבל גם את התפקיד.

מקורות רלוונטיים: אין

סקירת הפתרון שנבחר: הפתרון שנבחר הוא פתרון מספר 3 מתוך הפתרונות הקיימים לבעיה. פתרון זה מאפשר דינמיות והרצה על מחשבים שונים מכיוון שהשרת לא נבחר מראש אלא בזמן ההרצה, בניגוד לפתרון 1. בנוסף, אין צורך בשמירת קובץ שיתופי על מחשב קבוע נפרד וטיפול בבעיות גישה של משתמשים. פתרון זה נבחר מכיוון שהוא פשוט מאוד ומשתמש ביתרונות של UDP . שימוש בפרוטוקול UDP בהקשר הזה הוא יעיל מאוד ואין סיבה לדאוג לאיבוד מידע מכיוון שהמידע שנשלח קטן מאוד.

בעיה אלגוריתמית: יצירת חיבור מאובטח להעברת הודעות בתקשורת מוצפנת בין הלקוח לשרת

אלגוריתמים קיימים לפתרון הבעיה:

1. האלגוריתם הראשון לפתרון הבעיה הוא שימוש בפרוטוקול TLS. TLS הוא שכבת תקשורת מאובטחת הבנויה מעל TCP שבה כאשר מתבצע החיבור, יש אימות זהות בין השרת ללקוח ומתבצע תהליך ששמו "לחיצת יד". בתהליך זה השרת מעניק ללקוח תעודה דיגיטלית ובה המפתח הפומבי של הצפנת RSA א-סימטרית. הלקוח משתמש במפתח שקיבל להצפנת מפתח חדש שיוצר מסוג AES הצפנה סימטרית ושולח בחזרה לשרת. השרת מפענח את מפתח ה-AES שקיבל מהלקוח באמצעות המפתח הפרטי שלו וכך מתרחשת החלפת מפתחות שמאפשרת הצפנה ופענוח מצד הלקוח והשרת. תהליך לחיצת היד מתרחש בפרוטוקול TLS באופן אוטומטי.
2. אלגוריתם נוסף לפתרון הבעיה הוא שימוש בפרוטוקול Diffie-Hellman. בפרוטוקול זה הלקוח והשרת מייצרים מפתח פרטי, שהוא מספר אקראי, ומחשבים את המפתח הפומבי שלהם באמצעות אלגוריתם מתמטי. הם משתמשים ב-TCP או UDP להחליף את המפתחות הפומביים. ממצב שבו לכל צד יש את המפתח הפרטי שלו ואת המפתח הפומבי של אחר, שני הצדדים מבצעים חישוב מתמטי שמאפשר להם להגיע למפתח משותף. המפתח המשותף מחושב בזכות התלות המתמטית של המפתחות שכל צד שלח וקיבל. כך נוצר המפתח המשותף שמאוד קשה לפענח ומאפשר תקשורת מאובטחת בין השרת ללקוח.
3. האלגוריתם האחרון לפתרון הבעיה נקרא "הצפנה היברידית" והיא משתמשת בעקרון "לחיצת היד" של TLS אך בצורת העברה ידנית ושונה. השרת בלבד יוצר מפתח RSA ושולח ב-UDP רק את המפתח הפומבי. בעת הצורך, כשנוצר חיבור TCP , לאחר הרשמה או התחברות, הלקוח מייצר מפתח AES מצפין את ההודעה שהוא צריך לשלוח, מצפין את המפתח באמצעות המפתח הפומבי שהשרת הפיץ ושולח יחד עם ההודעה המוצפנת לשרת. השרת משתמש במפתח הפרטי שלו כדי לפענח את מפתח ה-AES ולאחר מכן גם

את ההודעה באמצעות המפתח המפוענח. כך מתבצעת החלפת המפתחות שמאפשרת הצפנה ופענוח מצד הלקוח והשרת.

מקורות רלוונטיים: ראה מאמר [1], ראה מאמר [2], ראה מאמר [3]

סקירת הפתרון שנבחר: הפתרון שנבחר הוא "הצפנה היברידית" מכיוון שזהו אלגוריתם יעיל מאוד שמנצל את היתרונות של כל אחד מסוגי ההצפנה: מורכבות המפתחות של RSA ומהירות הצפנת המידע של AES עבור מידע בכל הגדלים. פרוטוקול Diffie-Hellman הוא דרך טובה להחלפת מפתחות אך הוא לא מתמקד ביעילות ומהירות ההצפנה. TLS משתמש בתעודה דיגיטלית – תהליך מורכב יחסית לפרויקט בסדר גודל של CybAir, לכן לא נעשה בו שימוש. לכן אלגוריתם 3 נבחר.

בעיה אלגוריתמית: אלגוריתם חיפוש הטיסה

אלגוריתמים קיימים לפתרון הבעיה:

1. האלגוריתם הראשון לפתרון הבעיה סינון דינמי. המשמעות של שימוש בסינון דימני היא לתת למשתמש אפשרות לא למלא שדות בטופס חיפוש הטיסה ולבצע התאמות בהתאם לשדות שכן מולאו. לדוגמה כאשר המשתמש בוחר למלא את שדה תאריך היציאה בלבד, השרת יחפש במאגר את כל הטיסות שיוצאות בתאריך שנבחר מכל יעד בעולם לכל יעד אחר בעולם, במחלקה רנדומלית, וישלח את התוצאות חזרה ללקוח.
2. האלגוריתם השני לפתרון הבעיה הוא סינון בגישה מחמירה. המשמעות של סינון בגישה מחמירה היא שהמשתמש מחויב למלא את כל השדות בטופס שהם: תאריך יציאה, תאריך חזרה, מוצא, יעד, מספר נוסעים, ומחלקת טיסה. השרת מקבל את הנתונים ובוחר את הטיסות שמתאימות **לכל** הנתונים בלבד.

מקורות רלוונטיים: ראה מאמר [4]

סקירת הפתרון שנבחר: הפתרון שנבחר הוא שימוש בסינון בגישה מחמירה, מכיוון שכשמדובר בטיסות, יש הרבה מידע לייחס לו חשיבות, כגון: תאריכים, יעדים, שעות, מחירים ועוד. המטרה של פרויקט CybAir היא להקל על המשתמש ולהפוך את הממשק לאינטואיטיבי. כאשר משתמשים בסינון דינמי, יכולות להתקבל מאות תוצאות שיכולות ליצור עומס מידע על המשתמש, בלבול ואף תחושת חוסר אונים. בנוסף כשיש מאות תוצאות הדבר מקשה על הממשק הגרפי עצמו ומאט אותו בצורה משמעותית. אומנם סינון בגישה מחמירה מצריך מעט יותר מאמץ והחלטיות מצד המשתמש אך הוא מאפשר הצגה מסודרת של מספר מצומצם של תוצאות. לכן נעשה שימוש באלגוריתם סינון בגישה מחמירה.

**תיאור סביבת הפיתוח**

הפרויקט פותח באמצעות שפת התכנות C# בסביבת הפיתוח Visual Studio, אחת מסביבות הפיתוח המשולבות המובילות והמומלצות לעבודה עם שפות מבוססות .NET. Visual Studio מספקת כלים מתקדמים לכתיבה, עריכה, מציאת שגיאות, ניהול פרויקטים מורכבים, ותמיכה בממשק משתמש גרפי Windows Forms, אשר מאפשר יצירת יישומים ויזואליים דינמיים המותאמים למשתמשי Windows. בחירה בסביבה זו נתנה אפשרות לממש את כל הפונקציות הדרושות בפרויקט, החל מניהול תצוגות גרפיות ועד לתקשורת בזמן אמת בין לקוחות לשרת. במהלך הפיתוח נעשה שימוש במגוון ספריות .NET חיצוניות למטרות שונות.

לצורך יצירת מאגר הנתונים והטיפול במידע על הטיסות, נעשה שימוש בשפת Python, שהיא שפת תכנות דינמית ופופולרית מאוד בתחום ניתוח נתונים וניהול קבצים. באמצעות במהלך הפיתוח נעשה שימוש בספריית Pandas של Python. Pandas היא ספריית קוד פתוח שנועדה לעיבוד וניתוח נתונים בצורה נוחה ויעילה. היא מספקת מבנים מתקדמים כמו DataFrame שמאפשרים לאחסן נתונים בצורת טבלה, בדומה לגיליונות Excel או לטבלאות במסדי נתונים ולבצע עליהם פעולות כמו מיון, סינון, קיבוץ, מיזוג וייצוא לקבצים (כגון JSON או CSV). בפרויקט CybAir, נעשה שימוש ב־Pandas לצורך יצירת מאגר נתונים של טיסות, עיבוד המידע בצורה מסודרת, וסידורו בקובץ CSV אשר שימש את מערכת ההזמנות.

לצד סביבת הפיתוח, אחד האתגרים המרכזיים בפרויקט היה תקשורת בטוחה ויציבה בין השרת ללקוחות. Visual Studio מאפשר ניתוח של התקשורת דרך Debugger מובנה, אך כדי לבצע בדיקות מעמיקות יותר נדרשו כלים אשר מיועדים לניתוח תעבורת רשת: לצורך הבדיקות של תקשורת הרשת וההצפנה, נעשה שימוש בתוכנות כמו Wireshark. Wireshark הוא כלי פופולרי לניתוח תעבורת רשת בזמן אמת, אשר מאפשר צפייה מפורטת בכל החבילות המועברות בין השרת ללקוח. Wireshark סייע לוודא כי המידע מוצפן כהלכה וכי התקשורת אינה חשופה להאזנות.

כלי ניפוי השגיאות (Debugging) של Visual Studio שימש לבדיקת לוגיקה ותיקון באגים בקוד, מעקב אחרי זרימת התוכנית, ולאימות פעולת ההצפנה והפענוח (הצפנה היברידית).

מערכת CybAir עושה שימוש בפרוטוקול תקשורת מבוסס TCP ו־ UDP כדי לנהל את הקשר בין השרת ללקוחות. התקשורת מאורגנת כך שלכל הודעה יש מבנה מסוים. במצבים מסוימים מבנה ההודעות משתנה לפי הצורך.

בתחילת כל הודעה חייב להישלח פרוטוקול פעולה. פרוטוקול פעולה הוא מחרוזת, שכאשר השרת או הלקוח מקבלים אותה בהודעה, הם יודעים כיצד להתנהל ואיך לפעול. רשימת

פרוטוקולי הפעולה זהה עבור כל לקוח ושרת כדי ליצור הסכמה וסנכרון בכל המערכת. להלן רשימת הפרוטוקולים:

CANI,SIGNUP,LOGIN,FAILED,EMPTY,SUCCESSFULL,EXISTS, LOGOUT,
PROFILE_PICTURE, SEARCH,BOOK,RESERVATIONS,MAP,CHAT,MESSAGES,EXITCHAT

פרוטוקול תקשורת UDP משמש במערכת לבחירת התפקיד (לקוח/שרת) וקבלת המפתח הפומבי של הצפנת RSA מהשרת. מלבד המסר עצמו שנשלח בחבילת UDP יש עוד שני מרכיבים בעלי גודל קבוע שמרכיבים אותה:

- כותרת ה-IP: כותרת שמתווספת כדי שהחבילה תוכל להגיע ליעד ברשת. גודלה הוא 20 בתים
- כותרת ה-UDP: מכילה מידע טכני שדרוש כדי להעביר את החבילה ממחשב אחד לשני , בלי קשר לתוכן עצמו. גודלה הוא 8 בתים.

בפרוטוקול תקשורת זה קיימות שתי הודעות שונות:

1. **שם:** בקשה להיות שרת
   נשלחת מהלקוח לשרת
   מבנה השדות בהודעה: פרוטוקול פעולה "CANI", מחרוזת מפתח פומבי של RSA

2. **שם:** תגובה לבקשה להיות שרת
   נשלחת מהשרת ללקוח
   מבנה השדות בהודעה: פרוטוקול פעולה "FAILED", מחרוזת מפתח פומבי של RSA

שתי הפעולות שולחות תחילה מחרוזת בגודל 6/4 תווים ולכן גודלה היא 6/4 בתים. אך התגובה של השרת מוצפנת. גודלו של מפתח RSA הוגדר כ-256 בתים ולכן המחרוזת הופכת לגודל המפתח. כאשר מעבירים אותו למחרוזת מסוג XML גודלו הופך לכ-500 בתים. כלומר גודלו של המידע שנשלח בהודעה הראשונה הוא בקירוב 504 בתים. גודלו של המידע שנשלח בהודעה השנייה הוא בקירוב 1000 בתים. כלומר גודלו הכולל של כל חבילה בפרוטוקול תקשורת UDP במערכת הוא כ- 1028/532 בתים.

פרוטוקול תקשורת TCP משמש במערכת כחיבור יציב וקבוע בין הלקוח לשרת ששומר על סדר ההודעות ומוודא שאין שגיאות. דרך החיבור הזה ניתן לבצע את כל היכולות שיש לשרת וללקוח להציע במערכת כגון: הרשמה, התחברות, חיפוש טיסה ועוד. מלבד המסר עצמו שנשלח דרך חיבור ה-TCP יש עוד שני מרכיבים בעלי גודל קבוע שמאפיינים את החיבור:

- כותרת ה-IP: בדומה לפרוטוקול ה-UDP. גודלה הוא 20 בתים

- כותרת ה-TCP: בדומה לפרוטוקול ה-UDP אך מכילה יותר פרמטרים. גודלה הוא בין 20 ל-60 בתים.

לא ניתן לחשב את הגודל הקבוע של פרוטוקול ה-TCP במערכת CybAir (בניגוד ל-UDP) מכיוון שקיים מגוון רחב של הודעות שנשלחות בחיבור וגודלן שונה לחלוטין בהתאם לתוכן. למרות זאת, כדי למנוע שליחת הודעות גדולות מדי, הוצבה הגבלה על ההודעה מהסוג הגדול ביותר, שהיא הודעה הכוללת מידע על תמונה. הגודל המקסימלי של תמונה שהמשתמש יכול לבחור כתמונת פרופיל הוא 500KB. הגודל משפיע ישירות על הגודל המקסימלי של חיבור ה-TCP. כל הודעה אחרת שנשלחת מתקבלת בהתאם לגודל שלה על ידי אובייקט ה-StreamReader.

1. **שם:** הרשמה
   נשלחת מהלקוח לשרת
   מבנה השדות בהודעה: פרוטוקול פעולה "SIGNUP", אובייקט מסוג User שמכיל את נתוני המשתמש (שם, דוא"ל, סיסמה, תאריך לידה, עיר מגורים – לא חובה, תמונת פרופיל).

2. **שם:** תגובה להרשמה
   נשלחת מהשרת ללקוח
   מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם ההרשמה מוצלחת, אחרת פרוטוקול פעולה "FAILED".

3. **שם:** התחברות
   נשלחת מהלקוח לשרת
   מבנה השדות בהודעה: פרוטוקול פעולה "LOGIN", דוא"ל, סיסמה.

4. **שם:** תגובה להתחברות
   נשלחת מהשרת ללקוח
   מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם ההתחברות מוצלחת, אחרת פרוטוקול פעולה "FAILED".

5. **שם:** בקשת תמונת פרופיל
   נשלחת מהלקוח לשרת
   מבנה השדות בהודעה: פרוטוקול פעולה "PROFILE_PICTURE".

6. **שם:** תגובה לבקשת תמונת פרופיל
נשלחת מהשרת ללקוח
מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם התהליך מוצלח, אחרת פרוטוקול פעולה "FAILED", השם המלא של המשתמש, מחרוזת הבתים של תמונת הפרופיל.

7. **שם:** התנתקות
נשלחת מהלקוח לשרת
מבנה השדות בהודעה: פרוטוקול פעולה "LOGOUT"

8. **שם:** חיפוש
נשלחת מהשרת ללקוח
מבנה השדות בהודעה: פרוטוקול פעולה "SEARCH", אובייקט FlightSearchData שמכיל את נתוני החיפוש (תאריך יציאה, תאריך חזרה, מוצא, יעד, מספר נוסעים, מחלקה).

9. **שם:** תגובה לחיפוש
נשלחת מהלקוח לשרת
מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם התהליך מוצלח, אחרת פרוטוקול פעולה "FAILED", רשימת טיסות הלוך מסוג ActiveFlight, רשימת טיסות חזור מסוג ActiveFlight.

10. **שם:** הזמנת טיסה
נשלחת מהשרת ללקוח
מבנה השדות בהודעה: פרוטוקול פעולה "BOOK", אובייקט טיסה רצויה ActiveFlight, רשימת מושבים שנבחרו מסוג Seat.

11. **שם:** תגובה להזמנת טיסה
נשלחת מהלקוח לשרת
מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם התהליך מוצלח, אחרת פרוטוקול פעולה "FAILED".

12. **שם:** בקשת קבלת כל ההזמנות
נשלחת מהשרת ללקוח

מבנה השדות בהודעה: פרוטוקול פעולה "RESERVATIONS".

13. **שם:** תגובה לבקשת קבלת כל ההזמנות
נשלחת מהלקוח לשרת
מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם התהליך מוצלח, אחרת פרוטוקול פעולה "FAILED", רשימת טיסות מסוג ActiveFlight, רשימת הזמנות מסוג BookedFlight.

14. **שם:** בקשת מידע על המפה
נשלחת מהשרת ללקוח
מבנה השדות בהודעה: פרוטוקול פעולה "MAP", אובייקט טיסה רצויה ActiveFlight.

15. **שם:** תגובה לבקשת מידע על המפה
נשלחת מהלקוח לשרת
מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם התהליך מוצלח, אחרת פרוטוקול פעולה "FAILED". רשימת משתמשים בטיסה מסוג User.

16. **שם:** שליחת הודעה לשרת
נשלחת מהשרת ללקוח
מבנה השדות בהודעה: פרוטוקול פעולה "CHAT", הודעה.

17. **שם:** שליחת הודעה ללקוח
נשלחת מהלקוח לשרת
מבנה השדות בהודעה: פרוטוקול פעולה "CHAT", הודעה.

18. **שם:** התנתקות מהצ'אט
נשלחת מהשרת ללקוח
מבנה השדות בהודעה: פרוטוקול פעולה "EXITCHAT".

19. **שם:** בקשת קבלת כל ההודעות
נשלחת מהשרת ללקוח
מבנה השדות בהודעה: פרוטוקול פעולה "MESSAGES".

20. **שם:** תגובה לבקשת קבלת כל ההודעות
נשלחת מהלקוח לשרת
מבנה השדות בהודעה: פרוטוקול פעולה "SUCCESSFUL" אם התהליך מוצלח, אחרת פרוטוקול פעולה "FAILED". רשימת הודעות של במשתמש ובשרת מסוג ChatMessage.

**תיאור מסכי המערכת**

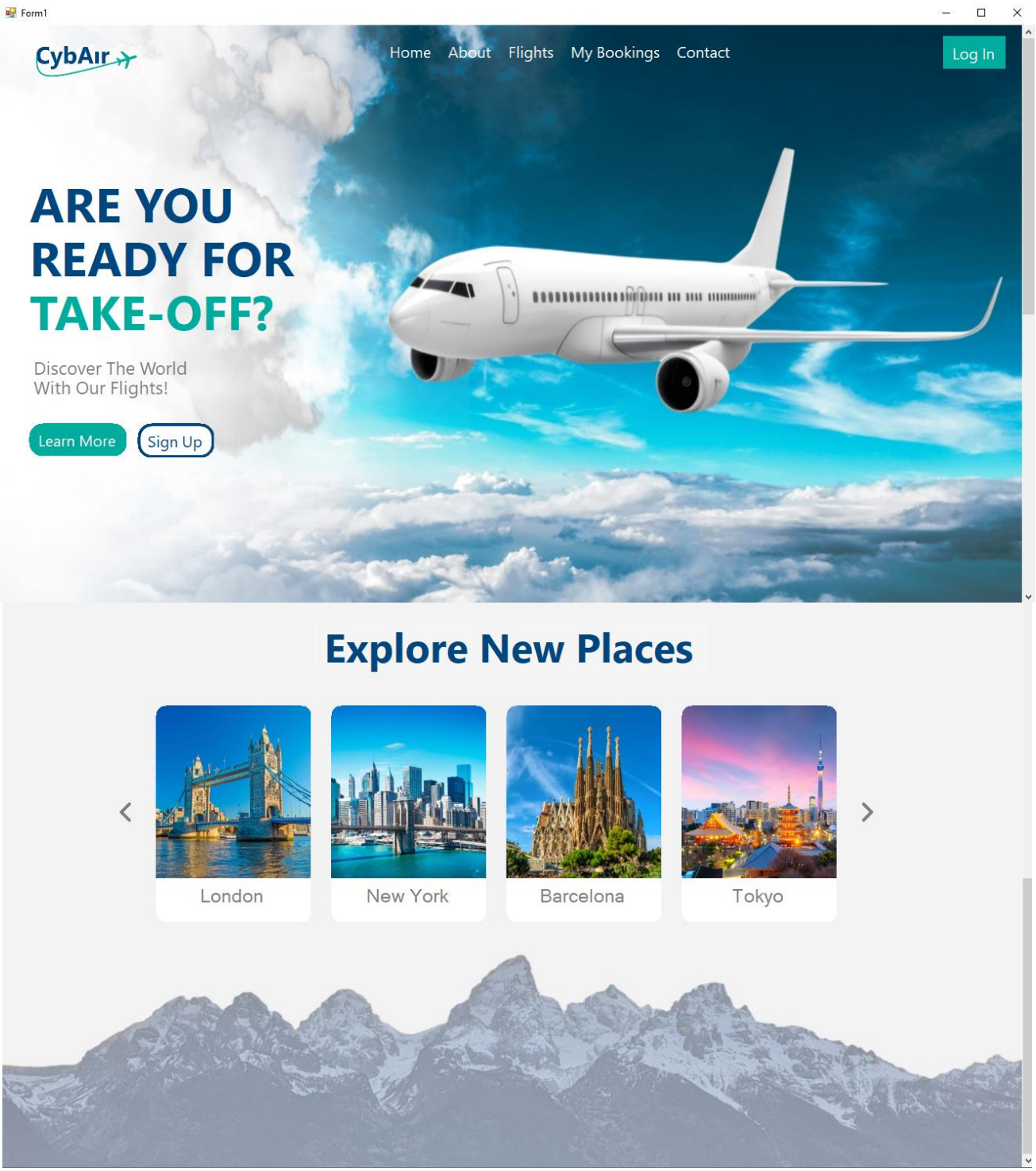
במערכת CybAir כל המסכים מקושרים זה לזה באמצעות תפריט ניווט אחד. תפריט הניווט נמצא בכל מסך במערכת ומאפשר להגיע לכל אחד מהמסכים האחרים באמצעות לחיצה על הכפתור. חלק מהמסכים מצריכים התחברות למערכת ולכן יפתחו רק בתנאי מקדים זה. תפריט הניווט כולל את הפריטים הבאים:

Log In – כפתור שמוביל למסך ההתחברות (במידה והמשתמש לא מחובר)
תמונת פרופיל ושם מלא (במידה והמשתמש מחובר)
Log Out – כפתור התנתקות שנפתח בלחיצה על תמונת הפרופיל (במידה והמשתמש מחובר)
Home – כפתור שמוביל למסך הבית
About – כפתור שמוביל למסך אודות
Flights – כפתור שמוביל למסך חיפוש הטיסות
My Bookings – כפתור שמוביל למסך ההזמנות שלי
Contact – כפתור שמוביל למסך יצירת קשר

כאשר בתיאור כל אחד מהמסכים יצוין תפריט הניווט, הכוונה תהיה למבנה שהוצג לעיל.

- **מסך בית**

מסך הבית מהווה את נקודת הכניסה הראשית למערכת CybAir. במסך זה מוצגים הלוגו ושם המערכת, וגם טקסט קצר שמסביר את מטרת המערכת. במסך מוצג תפריט הניווט שמאפשר למשתמש לעבור בין העמודים השונים. בנוסף לתפריט הניווט מוצגים עוד שני כפתורים שמפנים לעמודים אחרים:
Sign Up – כפתור שמוביל למסך ההרשמה
Learn More – כפתור שמוביל למסך האודות
בחלק השני של המסך מוצגות תמונות של יעדים ברחבי העולם שהחברה מציעה אליהם טיסות. ניתן להיעזר בכפתורי הניווט שלצד התמונות ועיין בכל התמונות.

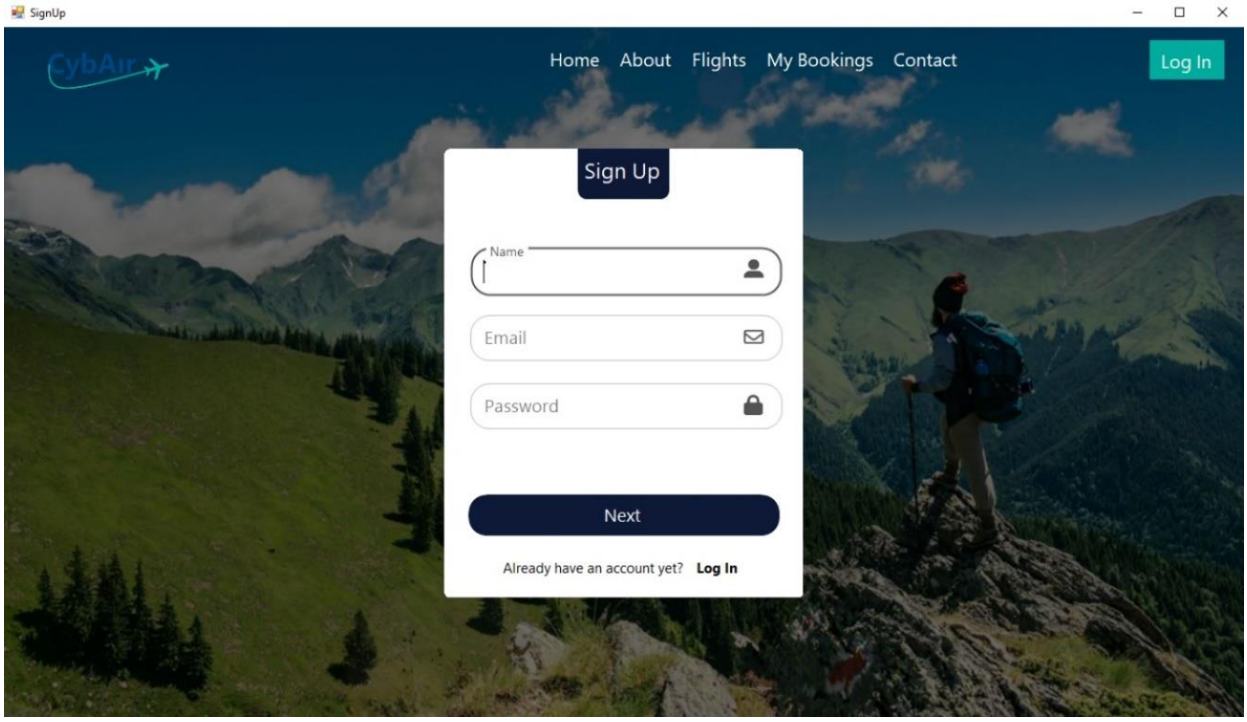


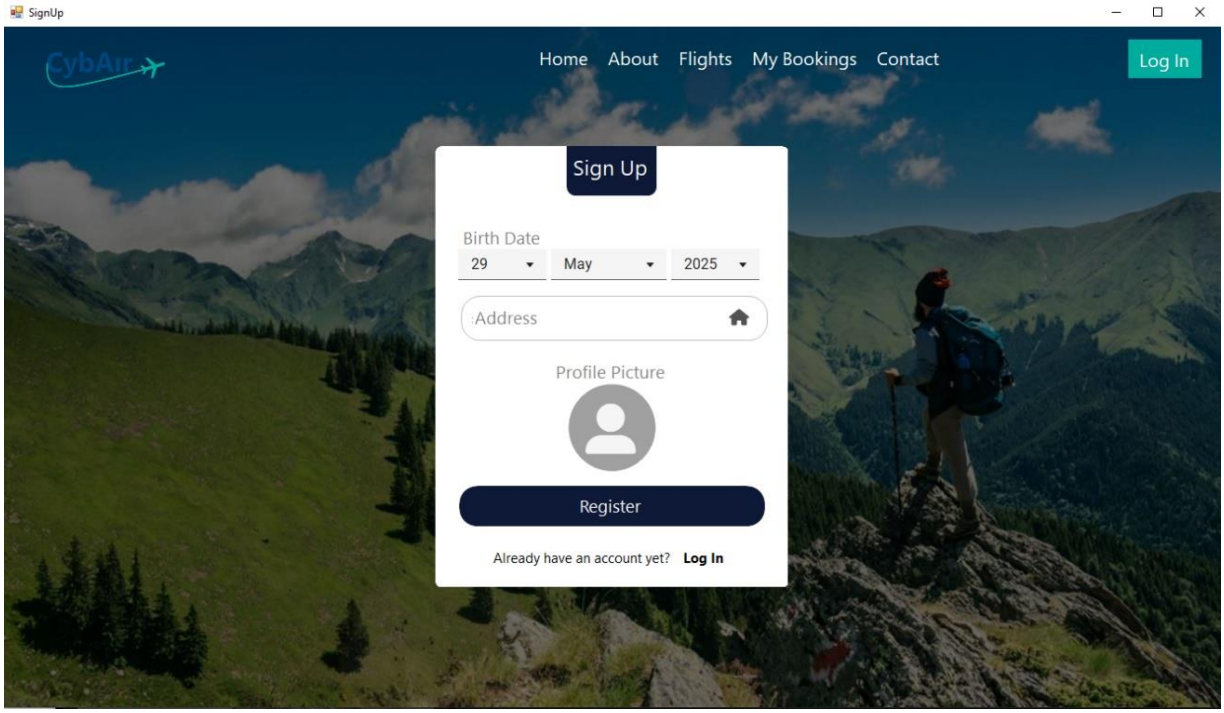

- **מסך אודות**

מסך "אודות" מציג מידע כללי על הפרויקט CybAir. במסך אפשר לראות את שם המערכת, את מספר הגרסה שבה המשתמש עובד, ואת ההסבר הכללי על המערכת, למה היא נועדה, מה היא מאפשרת לעשות, ומה מיוחד בה. מסך האודות נועד לעזור למי שמשתמש במערכת להבין את הרקע שלה.

- **מסך הרשמה**

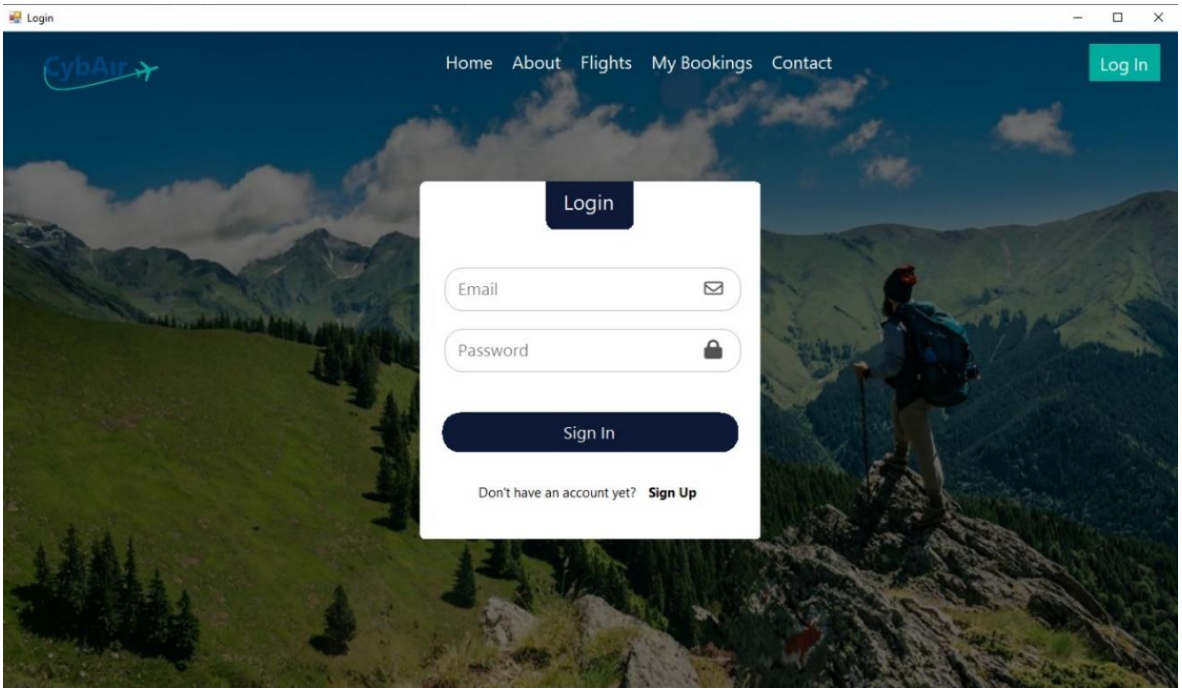
במסך ההרשמה מופיע תפריט הניווט והוא מחולק לשני חלקים. בחלק הראשון המשתמש נדרש למלא את שמו המלא, את כתובת הדוא"ל שלו ואת הסיסמה שתהיה לו בחשבון, בשלוש תיבות טקסט שונות. בתיבת הטקסט של הסיסמה ניתן ללחוץ על המנעול ולהעביר את הפורמט של הסיסמה מסמוי לגלוי. לאחר שהמשתמש ממלא את הפרטים הוא יכול ללחוץ על הכפתור Next שמעביר אותו לחלק השני של ההרשמה במידה והפרטים מולאו בהצלחה ובאופן תקין. בחלק השני המשתמש נדרש למלא את תאריך הלידה שלו ברשימות הנפתחות, ויש לו אפשרות לבחור למלא את עיר המגורים שלו בתיבת הטקסט ולהגדיר את תמונת הפרופיל. לבסוף המשתמש יכול לשלוח את הטופס לשרת באמצעות לחיצה על כפתור ה-Register. בנוסף ניתן לעבור לחלון ההתחברות בכל נקודה באמצעות הכפתור שבתחתית הטופס.


SignUp
CybAir
Home
About
Flights
My Bookings
Contact
Log In
Sign Up
Name
Email
Password
Next
Already have an account yet?
Log In


SignUp
CybAir
Home
About
Flights
My Bookings
Contact
Log In
Sign Up
Birth Date
29
May
2025
Address
Profile Picture
Register
Already have an account yet?
Log In

- **מסך התחברות**

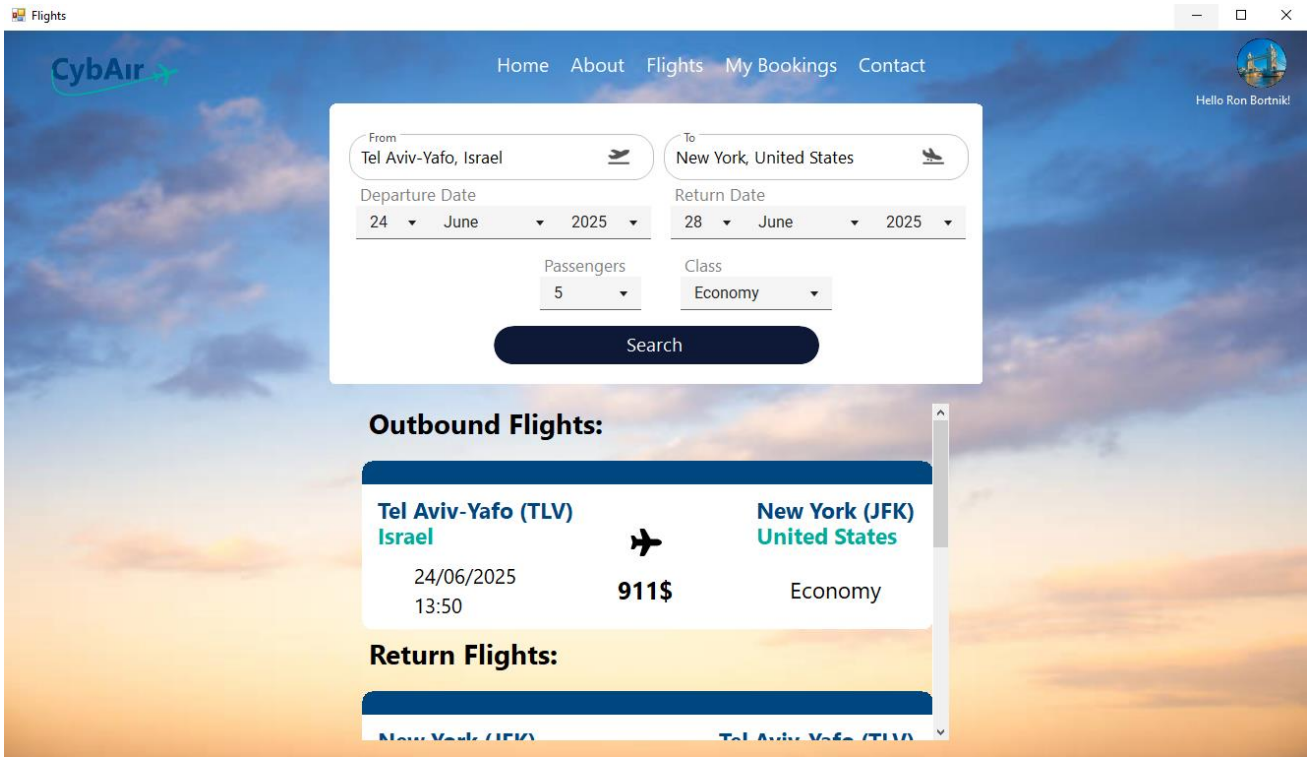
במסך ההתחברות מופיע תפריט הניווט. המשתמש נדרש למלא את כתובת הדוא"ל ואת הסיסמה של החשבון בתיבות הטקסט המיועדות לכך. בתיבת הטקסט של הסיסמה ניתן ללחוץ על המנעול ולהעביר את הפורמט של הסיסמה מסמוי לגלוי.
לבסוף המשתמש יכול לשלוח את הטופס לשרת באמצעות לחיצה על כפתור ה- Sign In. בנוסף ניתן לעבור לחלון ההרשמה בכל נקודה באמצעות לחיצה על הכפתור שבתחתית הטופס.

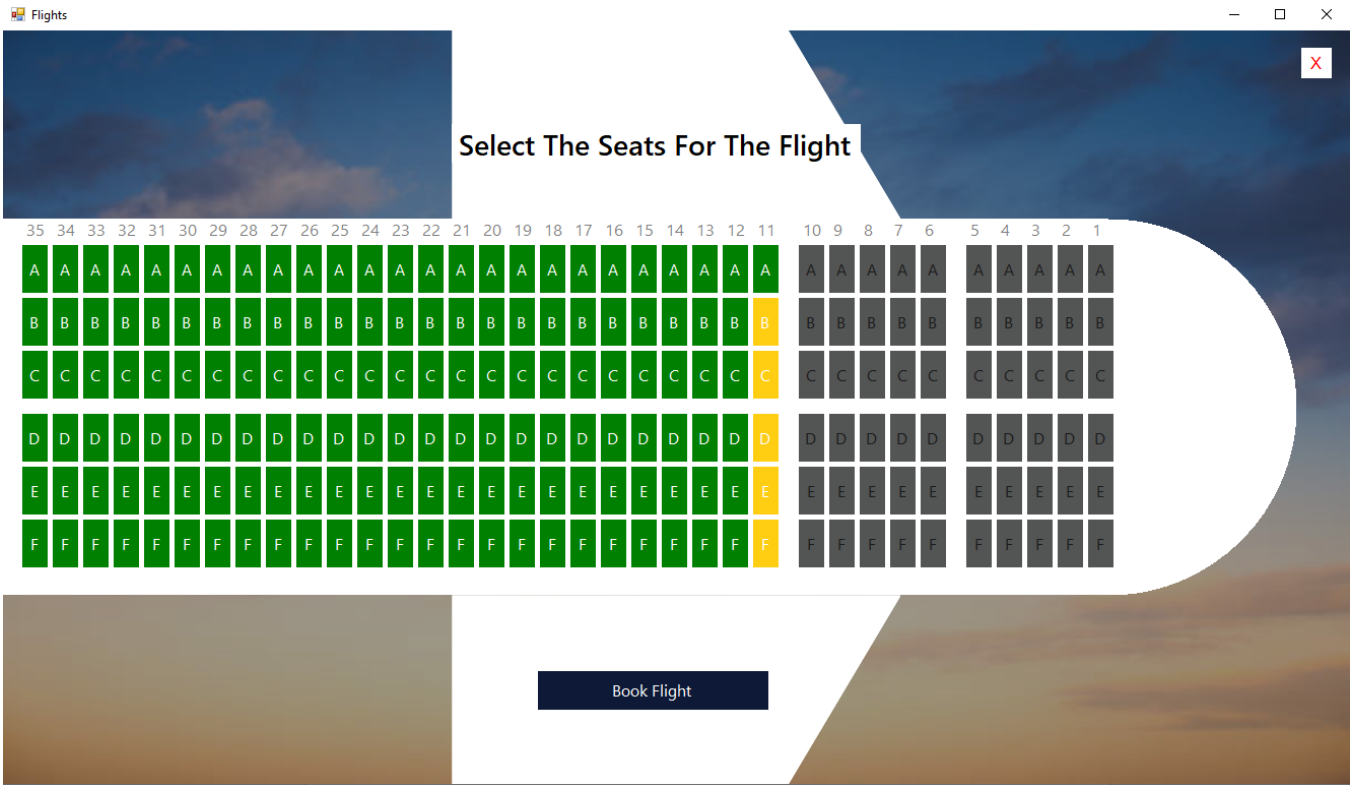


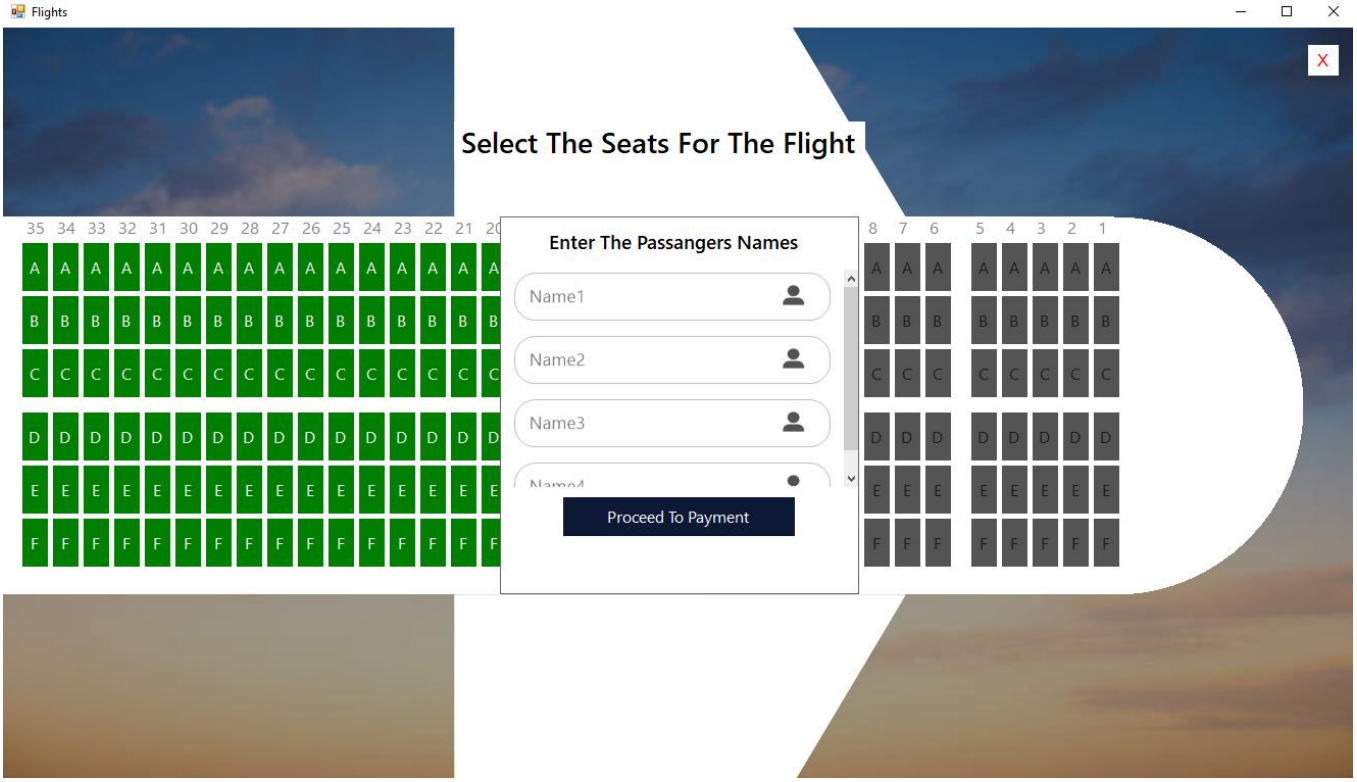

- **מסך חיפוש טיסה**

במסך חיפוש טיסה מופיע תפריט הניווט. המסך מחולק לשלושה חלקים. בחלק הראשון מופיע טופס חיפוש הטיסה שבו המשתמש נדרש למלא עיר המוצא, עיר היעד, תאריך היציאה, תאריך החזרה, מספר נוסעים ומחלקת טיסה. לאחר לחיצה על כפתור ה-Search יופיעו טיסות ההלוך וחזור שמתאימות לנתונים שמולאו בטופס.
בלחיצה על אחד מכרטיסי הטיסה שמוצגים, מופיע החלק השני של מסך חיפוש הטיסה. בחלק השני מופיע מיפוי של מושבי הטיסה בתוך המטוס. המושבים בצע ירוק פנויים והמושבים בצע אפור תפוסים או לא שייכים למחלקה שנבחרה.
ניתן ללחוץ על כל אחד מהמושבים הפנויים ובכך לסמן אותם בצבע צהוב. לבסוף ניתן ללחוץ על כפתור ה-Book Flight ולעבור לחלק השלישי. בחלק השלישי מופיע טופס נוסף שבו המשתמש צריך למלא את שמות הנוסעים האחרים שלהם הוא מזמין מקום. לאחר מילוי השמות בהצלחה המשתמש, מכניס את פרטי התשלום, לוחץ על כפתור ה-Complete Payment וכל המידע נשלח לשרת.

המשתמש יכול ללחוץ על כפתור ה-X כדי לחזור חזרה לחלק הראשון ולחפש טיסה אחרת.

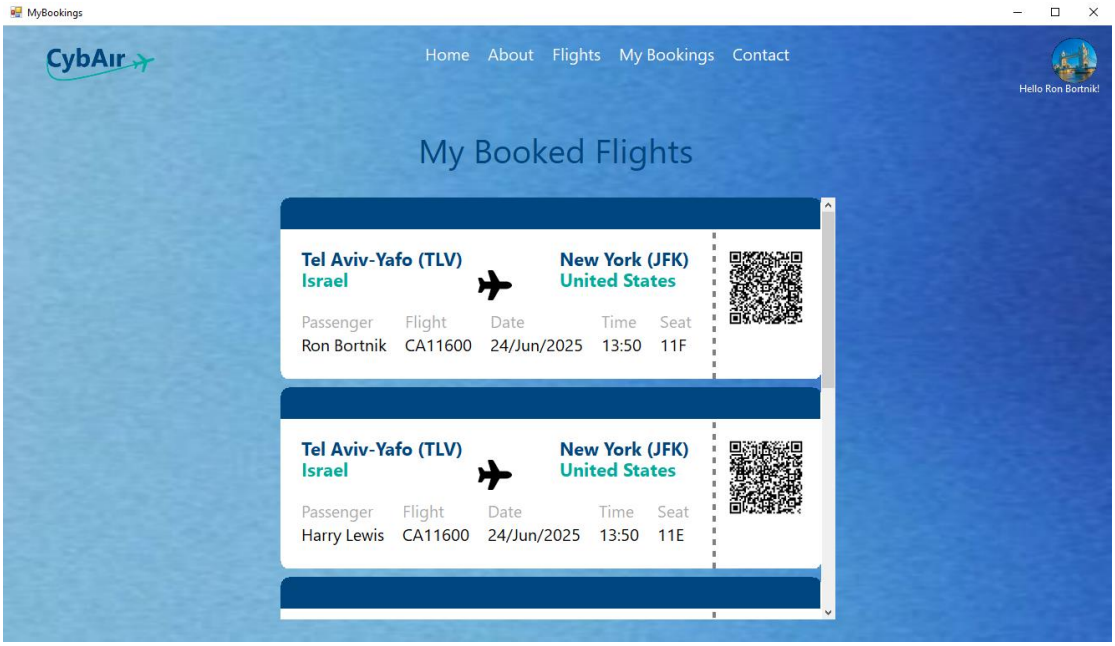


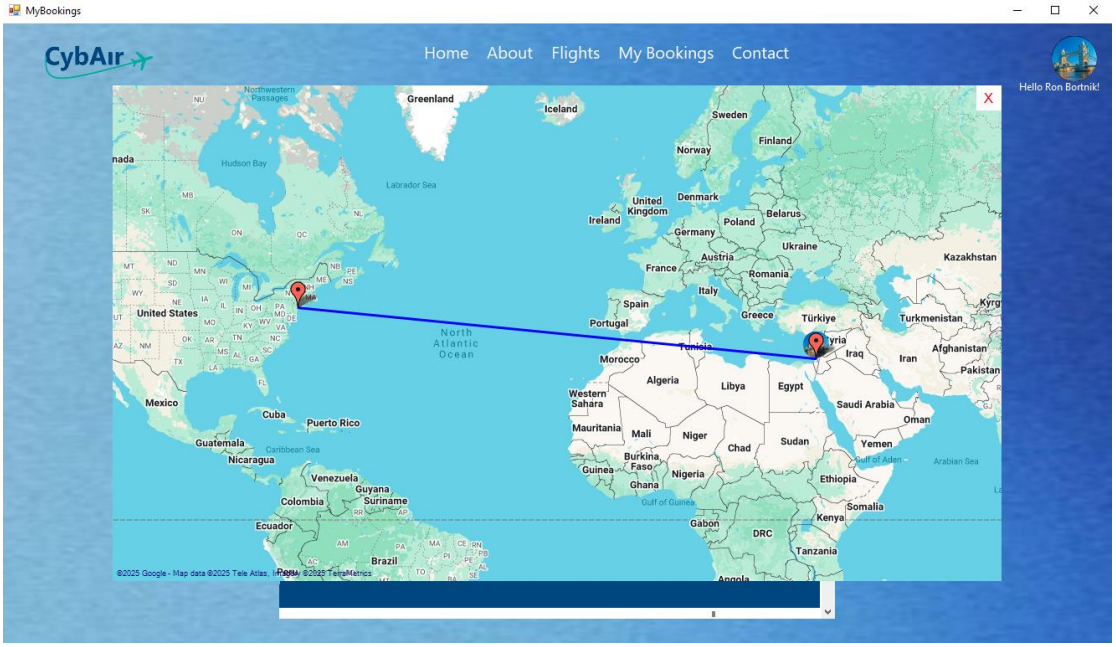






- **עמוד ההזמנות שלי**

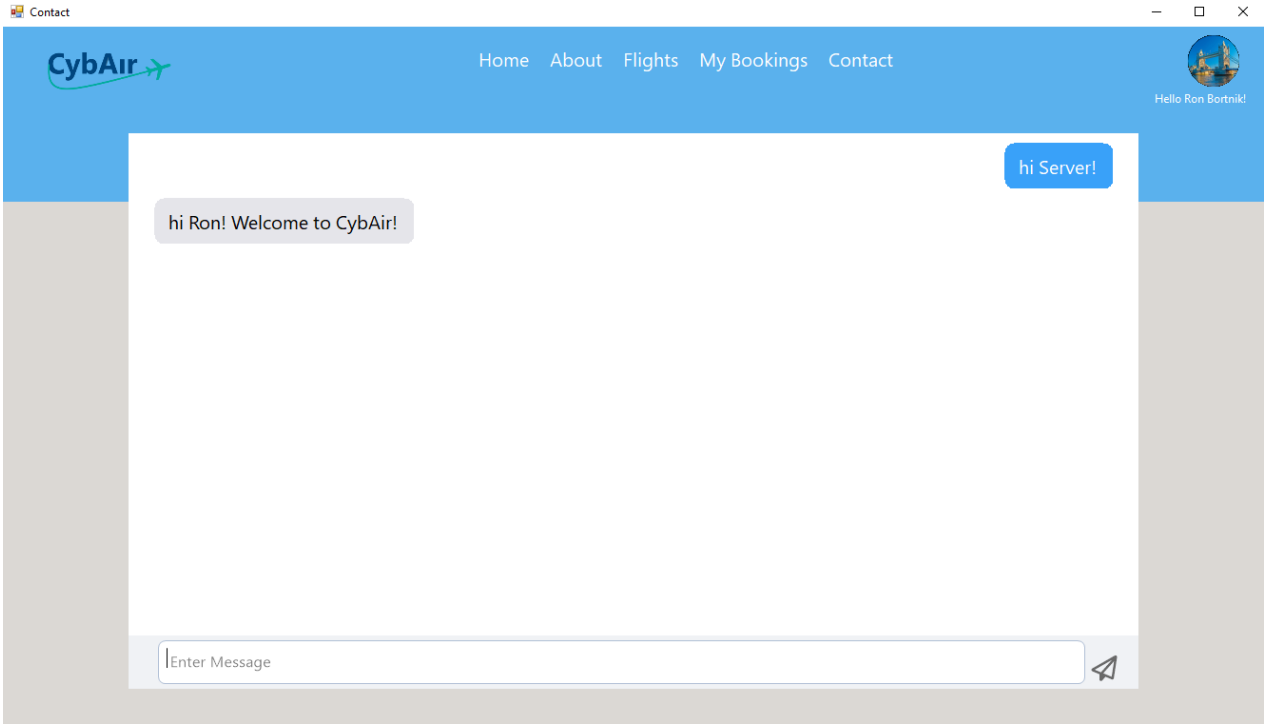
בעמוד ההזמנות שלי מופיע תפריט הניווט. בנוסף מופיעים כל כרטיסי הטיסה וההזמנות על שם המשתמש. בכל כרטיס טיסה מופיעים כל הפרטים הנחוצים בשביל כרטיס טיסה רשמי כולל קוד QR שבו מופיעים הפרטים על הטיסה ומאפשרים אימות של כרטיס הטיסה בבדיקות בשדה התעופה עצמו. לחיצה על כל כרטיס פותחת את מפת העולם האינטראקטיבית, מציגה את מסלול הטיסה והמשתמשים האחרים שהזמינו טיסה ומוכנים להיכרות.


MyBookings
CybAir
Home
About
Flights
My Bookings
Contact
Hello Ron Bortnik!
My Booked Flights
Tel Aviv-Yafo (TLV)
Israel
New York (JFK)
United States
Passenger
Flight
Date
Time
Seat
Ron Bortnik
CA11600
24/Jun/2025
13:50
11F
Tel Aviv-Yafo (TLV)
Israel
New York (JFK)
United States
Passenger
Flight
Date
Time
Seat
Harry Lewis
CA11600
24/Jun/2025
13:50
11E


MyBookings
CybAir
Home
About
Flights
My Bookings
Contact
Hello Ron Bortnik!
Greenland
Iceland
Sweden
Finland
Norway
United States
Mexico
Cuba
Puerto Rico
Guatemala
Nicaragua
Venezuela
Guyana
Suriname
Colombia
Ecuador
Brazil
North Atlantic Ocean
Ireland
United Kingdom
Denmark
Belarus
Poland
Germany
Ukraine
Austria
France
Romania
Italy
Spain
Portugal
Greece
Türkiye
Kazakhstan
Turkmenistan
Afghanistan
Iran
Pakistan
Iraq
Morocco
Algeria
Libya
Egypt
Western Sahara
Mauritania
Mali
Niger
Chad
Sudan
Saudi Arabia
Oman
Yemen
Ethiopia
Somalia
Kenya
Nigeria
Ghana
Guinea
Burkina Faso
Gabon
DRC
Tanzania

- **מסך הצ'אט**

במסך הצ'אט מופיע תפריט הניווט. במסך ניתן להקליד בתיבת הטקסט הודעות לשרת ולשוחח איתו. ההודעות שהמשתמש שולח הן בצבע כחול וההודעות שהשרת שולח הן בצבע אפור.

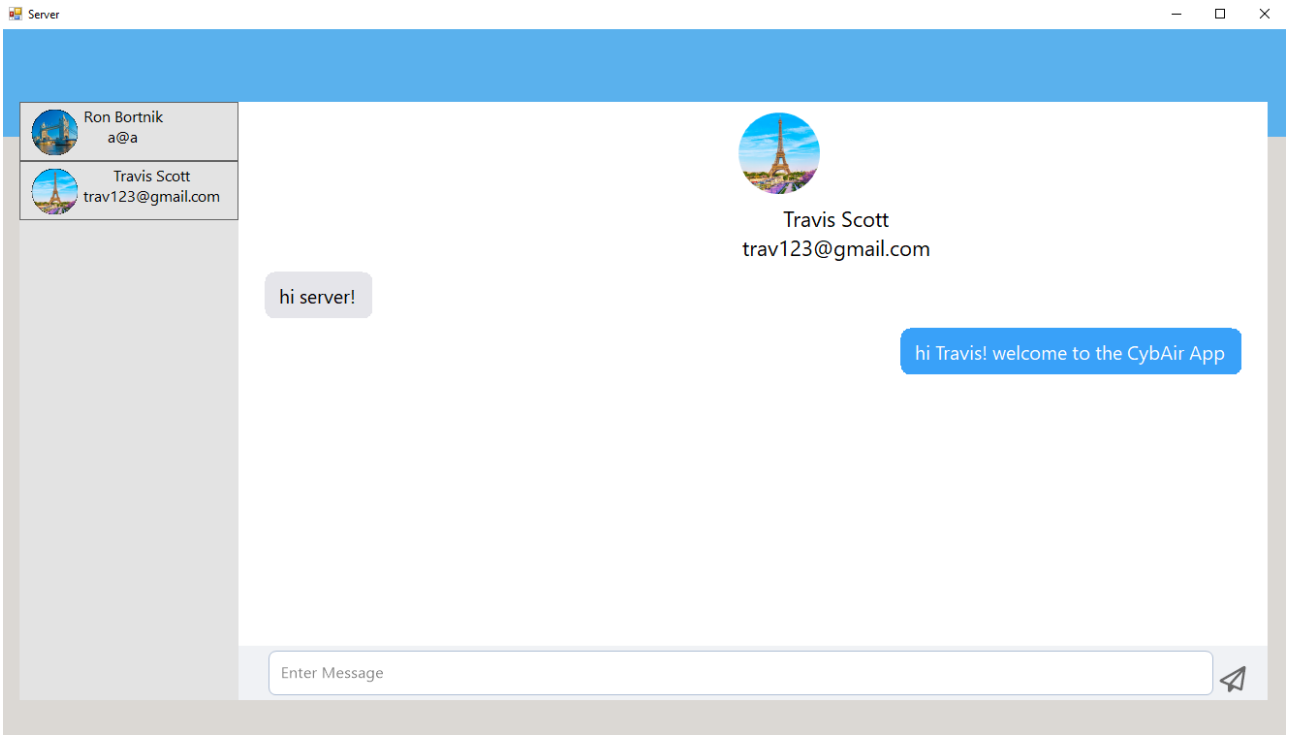



- **מסך השרת**

במסך השרת לא מופיע תפריט הניווט מכיוון שהוא לא משתמש רגיל. במסך זה השרת יכול לשוחח ולענות לכל המשתמשים. בצד שמאל מופיעים כל המשתמשים שיש להם חשבון במערכת ובלחיצה על כל אחד מהם נפתח הצ'אט עם כל ההודעות. השרת יכול לכתוב הודעה בתיבת הטקסט ולשלוח אותה למשתמש הנוכחי שעליו הוא לחץ, בדומה לאפליקציות כמו WhatsApp.

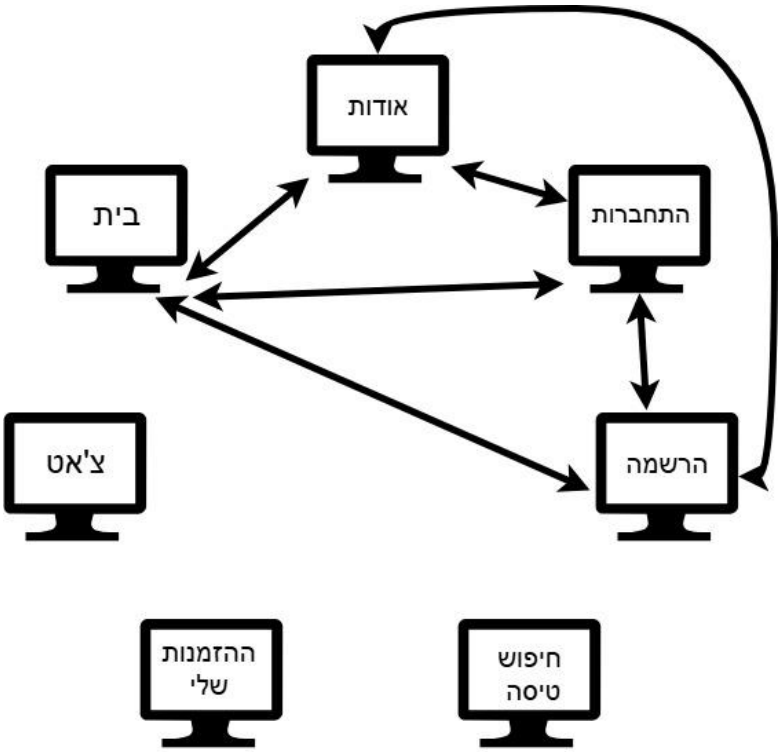
ההודעות שהשרת שולח הן בצבע כחול וההודעות שהלקוח שולח הן בצבע אפור.

במידה והשרת מקבל הודעה ממשתמש שהוא לא נמצא בשיחה איתו, השרת יקבל התראה ליד השם שלו.

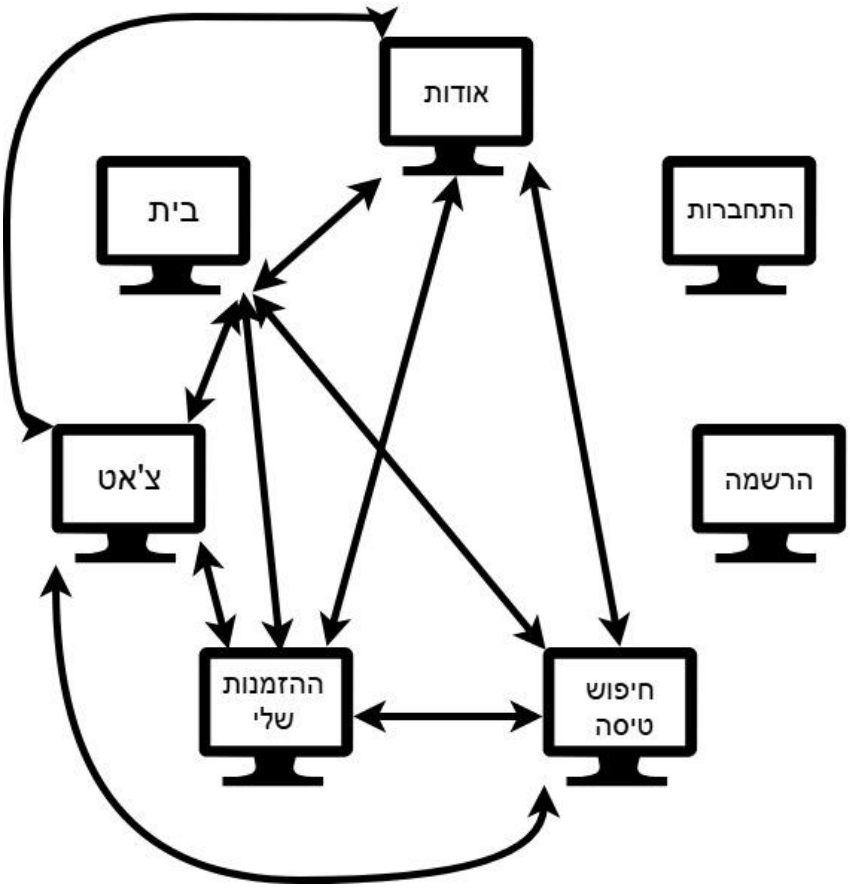

Server
Ron Bortnik
a@a
Travis Scott
trav123@gmail.com
Travis Scott
trav123@gmail.com
hi server!
hi Travis! welcome to the CybAir App
Enter Message

להלן תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם:

לפני התחברות למערכת:




אחרי התחברות למערכת:



**פירוט מבני הנתונים**

מסד הנתונים במערכת CybAir הוא אחד מהחלקים המרכזיים והחשובים ביותר כמו בכל מערכת הזמנת טיסות או חברת תעופה. מסד הנתונים מורכב מכמה קבצים שונים, חלקם מתעדכנים במהלך הרצת הפרויקט על ידי השרת וחלקם נשארים קבועים ומטרתם היא סיפוק מידע.

Worldcities.csv (מקור [4])– הקובץ הזה הוא קובץ מידע, שחשוף לכל המשתמשים ולשרת. הקובץ מכיל מידע על כ-48,000 ערים ברחבי העולם. המידע שבקובץ משמש כמקור השלמה אוטומטית בטפסים השונים שבהם נדרשת כתובת כגון: עיר מגורים בטופס ההרשמה, עיר המוצא ועיר היעד בטופס חיפוש הטיסה. המפה האינטראקטיבית שבמסך ההזמנות שלי משתמשת במידע שבקובץ כדי להציג את תמונת הפרופיל במיקום הגיאוגרפי של עיר המגורים. בנוסף המידע שבקובץ מהווה חלק ביצירת מאגר הטיסות.

| שדה | Iso3 | Iso2 | lng | lat | country | city |
|---|---|---|---|---|---|---|
| טיפוס | string | string | double | double | string | string |
| ערכים | JPN | JP | 139.6922 | 35.6897 | Japan | Tokyo |

Airports.csv (מקור [5]) – קובץ זה הוא קובץ מידע שחשוף לכל המשתמשים ולשרת. הקבוץ מכיל מידע על כ-7,700 שדות תעופה שונים ברחבי העולם במגוון גדלים. למערכת אין שימוש ישיר בקובץ אך הוא מהווה חלק ביצירת מאגר הטיסות.

| שדה | IATA | country | city | name |
|---|---|---|---|---|
| טיפוס | string | string | string | string |
| ערכים | TLV | Israel | Tel Aviv-Yafo | Ben Gurion International Airport |

מפתח ראשי: IATA

routes.csv (מקור [6]) – קובץ זה הוא קובץ מידע שחשוף לכל המשתמשים ולשרת. הקבוץ מכיל מידע על כ-67,000 טיסות ישירות שהתקיימו על ידי כ-600 חברות תעופה שונות. למערכת אין שימוש ישיר בקובץ אך הוא מהווה חלק ביצירת מאגר הטיסות.

| שדה | destination airport | source airport |
|---|---|---|
| טיפוס | string | string |
| ערכים | JFK | TLV |

Flights.csv - קובץ זה הוא קובץ מידע שחשוף לכל המשתמשים ולשרת. הקובץ מכיל את המידע על כ-4500 הטיסות השונות שמערכת CybAir מציעה. הוא משמש כקובץ המידע הראשי במערכת שעליו מבוסס כל תהליך החיפוש וההזמנה של הטיסות, התהליך המרכזי בפרויקט. הקובץ נבנה באמצעות שילוב וחיבור של הקבצים שצוינו לעיל תוך הוספת פרטים ושדות כדי להתאים את המידע למערכת ולהפוך אותו ליותר מפורט.

| id | origin_airport | Name_x | origin_city | origin_country | שדה |
|---|---|---|---|---|---|
| int | string | string | string | string | טיפוס |
| 1 | TLV | Ben Gurion International Airport | Tel Aviv-Yafo | Israel | ערכים |

| destination_airport | Name_y | destination_city | destination_country | שדה |
|---|---|---|---|---|
| string | string | string | string | טיפוס |
| JFK | John F Kennedy International Airport | New York | United States | ערכים |

| Price ($) | flights_per_day | active_days | times | שדה |
|---|---|---|---|---|
| int | int | string | string | טיפוס |
| 911 | 1 | [3 5 1 2 7] | [13:50] | ערכים |

מפתח ראשי:
id

המאגר נוצר באמצעות כמה פעולות:

- מיזוג של routes.csv ו-airports.csv לפי השדות source airport ו-IATA
- מיזוג של routes.csv ו-airports.csv לפי השדות destination airport ו-IATA
- יצירת עמודת price באמצעות חישוב המרחק בין שתי הערים לפי נוסחת האברסין וחלוקה ב-10
- יצירת עמודת flights_per_day באמצעות בחירת מספר אקראי של בין 1 ל-4 טיסות ביום
- יצירת עמודת active_days באמצעות בחירת מספר אקראי של ימים בשבוע ואת הימים עצמם
- יצירת עמודת times באמצעות בחירת זמנים אקראיים בין 6:00 ל-23:00 לפי מספר הטיסות ביום

Users.json – קובץ זה מתעדכן אך ורק על ידי השרת והוא היחיד שרואה את הנתונים העדכניים בקובץ. קובץ זה שומר את המידע על כל אחד מהמשתמשים במערכת והוא מתעדכן בכל פעם שהם מבצעים פעולות כמו הזמנה, ביטול טיסה ועבור כל הודעה שנשלחת בצ'אט.

| שדה | address | birth_date | salt | password | email | name |
|---|---|---|---|---|---|---|
| טיפוס | string | DateTime | string | string | string | string |
| ערכים | Tel Aviv - Yafo | 15/12/2007 | | Abcd123! | Ron.bortnik@gmail.com | Ron Bortnik |

| שדה | Chat_online | Chat_messages | Booked_flights | Loged_in | Profile_picture |
|---|---|---|---|---|---|
| טיפוס | bool | List<ChatMessage> | List<BookedFlight> | bool | byte[] |
| ערכים | true | | | true | |

מפתח ראשי: email

ActiveFligths.json - קובץ זה מתעדכן אך ורק על ידי השרת והוא היחיד שרואה את הנתונים העדכניים בקובץ. קובץ זה שומר את המידע על הטיסות הפעילות ומתעדכן כאשר תאריך הטיסה עבר, משתמש מזמין מושבים בטיסה או מבטל אותם. הקובץ הוא רשימה של אובייקטים מסוג ActiveFlight. אובייקט זה יורש מאובייקט Flight אשר מיוצג על ידי שורה בקובץ flights.csv, כלומר הוא מקבל את כל הערכים שלו. לאובייקט ActiveFlight יש כמה שדות בנוסף לאובייקט Flight:

| שדה | seats | Time | date | Flight_number |
|---|---|---|---|---|
| טיפוס | bool[] | string | DateTime | string |
| ערכים | | 14:50 | 25/6/2025 | CA1160 |

**סקירת האיומים שיתכנו למערכת**

- עבודה מול אתרי Web עשויה לכלול סיכונים פוטנציאליים למשל, שימוש בשירותי אינטרנט שאינם מוצפנים (HTTP) עלול לחשוף את המערכת לתקיפות מסוג Man-in-the-Middle (MITM), שבהן תוקף מאזין לתקשורת או משנה את המידע בדרכו. בנוסף, שימוש בשירותים חיצוניים שאינם עומדים בתקני אבטחה עלול להוביל לחשיפת מידע רגיש, טעינת קוד זדוני, או תלות במקורות שעלולים להיעלם או להשתנות ללא התרעה. בפרויקט CybAir קיימת עבודה מול אתרי Web בכך שנעשה שימוש בספריית GMap.NET לצורך שילוב מפות בזמן אמת בתוך המערכת. הספרייה עובדת מול שירותי מפות חיצוניים, ונבחר להשתמש ב־OpenStreetMapProvider, שהוא שירות קוד פתוח. ספק מפות זה תומך בתקשורת מאובטחת באמצעות פרוטוקול HTTPS, דבר המבטיח שהמידע המתקבל מהשרת מוצפן ואינו חשוף להאזנה או שינוי על ידי גורמים זדוניים. בפרויקט נעשה שימוש רק בפונקציות בסיסיות כמו הצגת סמנים (Markers) ו־קווים ישרים (Polylines) ביניהם, אשר מבוצעים על גבי שכבות נתונים מקומיות ואינם דורשים שליחה של מידע רגיש לשרתים חיצוניים. לא נעשה שימוש ביכולות מתקדמות כמו חישוב מסלול או מידע תחבורתי, ולכן אין תלות בשירותים רגישים או בעיבוד נתונים בענן. כתוצאה מכך העבודה מול אתרי Web בפרויקט נעשתה באופן מאובטח לחלוטין.
- תהליך האימות והווידוא בזמן תהליך ה-login במערכת נועד להבטיח שרק משתמשים מורשים יכולים להיכנס לחשבון שלהם ולהשתמש בכל האפשרויות שהיא מציעה. כאשר המשתמש מזין את שם המשתמש והסיסמה, מתבצעת תחילה בדיקה בצד הלקוח שמטרתה היא לוודא שהמבנה של הנתונים שהוזנו תקין. לאחר שהבדיקה הסתיימה וכל הנתונים תקינים, הם נשלחים לשרת, ששם הם נבדקים מחדש כדי לוודא שלא היה שינוי בזמן העברתם, סכנה שאכן קיימת. בנוסף השרת בודק את נכונותם מול הנתונים השמורים במסד הנתונים. במסד הנתונים הסיסמאות של המשתמש לא שמורות בצורה חשופה אפילו לשרת כדי שבמידה ומישהו מצליח לפרוץ למסד אז הסיסמאות לא יהיו גלויות. הסיסמאות שמורות בצורה מוצפנת באמצעות שימוש ב-Hash ו-Salt. ה-Salt שמוגדר לכל משתמש הוא שונה אך אלגוריתם ה-Hash זהה וזאת כדי לאפשר ווידוא ואימות בזמן ההתחברות. כאשר השרת בודק את נכונות הנתונים מול מסד הנתונים הכוונה היא שהוא מוצא את המשתמש על פי כתובת הדוא"ל שהתקבלה (מאפיין מייחד של כל משתמש), ומקבל את ערך ה-Salt השמור אצלו. לאחר מכן הוא משתמש בערך זה ובאלגוריתם ה-Hash כדי להגיע לאותה תוצאה ששמורה בתור הסיסמה במסד הנתונים. אם הפרטים תואמים, המשתמש מזוהה כתקין, והמערכת מוצאת שהוא לא מחובר, אז המערכת תאפשר למשתמש להיכנס לחשבון. במידה והפרטים שגויים או שהמשתמש כבר פעיל, נשלחת תגובה מתאימה שמונעת ממנו להיכנס.
- בפרויקט זה נבחר ליישם מנגנון הצפנה היברידית על גבי תקשורת ה־TCP, במטרה להגן על המידע המועבר בין הלקוח לשרת בצורה מאובטחת ולמנוע התקפות MITM (Man In The

(Middle. בהתקפה מסוג זה תוקף "מתמקם באמצע" בין שני צדדים שמתקשרים, למשל בין לקוח לשרת, מבלי שהם יודעים שהוא שם. התוקף מצליח להתערב בתקשורת ולהיחשף למידע שעובר בין הצדדים, ולעיתים גם לשנות אותו, לפני שהוא ממשיך ליעדו. כדי למנוע התקפה מסוג זה חשוב להצפין את התקשורת.

ההצפנה ההיברידית משלבת בין RSA (הצפנה אסימטרית) לבין AES (הצפנה סימטרית), כך שבתחילת התהליך השרת שולח ללקוח את המפתח הציבורי של RSA באמצעות UDP. הלקוח משתמש במפתח זה כדי להצפין מפתח סימטרי מסוג AES ושולח אותו לשרת דרך TCP. לאחר מכן, כל ההודעות בין הצדדים מוצפנות באמצעות המפתח הסימטרי. הבחירה בהצפנה היברידית נובעת מהרצון לשלב בין ביצועים גבוהים לבין רמת אבטחה גבוהה: RSA נחשב לאיטי להצפנת כמויות גדולות של מידע אך בטוח להעברת מפתחות, ואילו AES מאפשר הצפנה מהירה של תוכן ההודעות עצמן. בכך המערכת משיגה שכבת הגנה משמעותית מפני תקיפות רשת נפוצות כמו MITM.

- התקפות DOS/DDOS אכן יכולות לקרות בפרויקט בגלל התקשורת הקבועה בין השרת ללקוח. התקפות מסוג זה שולחות אלפי בקשות לשרת (או יותר) מכתובת IP אחת (DOS) או מהרבה כתובות IP שונות (DDOS). התקפות מסוג זה יוצרות עומס רב על השרת ופוגעות בתפקוד שלו עד למצב שבו הוא מפסיק לעבוד ונופל. כדי להתמודד עם התקפות מסוג זה חשוב להציב מגבלות על קריאה והתמודדות עם בקשות בצד של השרת כדי להתמודד עם תקיפות DOS. בנוסף יש להציב מגבלה על מספר החיבורים ללקוח באמצעות TCP כדי למנוע התקפות DDOS. בפרויקט הוחלט לקבוע מגבלה של 5 חיבורים בשנייה ו-5 הודעות בכל חיבור בשנייה. כך לא יכול להיווצר מצב של עומס על השרת שיביא לנפילתו.

## מימוש הפרויקט

**מודולים/מחלקות מיובאים**

**C#:**

- CsvHelper – מודל שמיועד לקריאה וכתיבת קבצים מסוג csv. המודול אפשר עיבוד הנתונים בקובצים flights.csv ו-worldcities.csv.
- FontAwesome.Sharp – מודול שמיועד להצגת סמלים ויזואליים שלא נמצאים על המקלדת. השימוש הבולט במודול זה הוא לעיצוב הטפסים השונים במטרה לשפר את חווית המשתמש.
- GMap.Net – מודול המיועד לניהול מפות, מיקומים ותצוגות גאוגרפיות. המודול אפשר הצגת מפת העולם ונתונים שונים עליה.
- Krypton.Toolkit.Lite – מודול המיועד לשיפור העיצוב הבסיסי שמוצע על ידי הממשק הגרפי WindowsForm.
- MaterialSkin.2 - מודול המיועד לשיפור העיצוב הבסיסי שמוצע על ידי הממשק הגרפי WindowsForm.
- QRCoder – מודול שמיועד ליצירת QR code ויזואלי שניתן לסריקה. המודול אפשר ליצור QR code לכל כרטיס טיסה למטרות אימות הכרטיס בשדה התעופה.

**Python:**

- Pandas – מודול המיועד לניהול ועיבוד נתונים, קריאה וכתיבת קבצי נתונים. המודול אפשר לעבד את נתוני הקבצים השונים במערכת שמכילים מידע על ערים, טיסות ושדות תעופה, ולמזג אותם למאגר נתונים מותאם למערכת.

**מחלקות/מודולים שפותחו במסגרת הפרויקט**

### About

תפקיד: מייצגת את חלון "אודות" בממשק המשתמש. אחראית על עיצוב והתאמה דינאמית של הרכיבים בחלון, כולל ניווט, תמונות, כיתוב, ותגובות לאירועים כמו שינוי גודל החלון או סגירתו.

תכונות: רכיבי UI בלבד

פעולות:

- **About()**

  **טענת כניסה**: אין

  **טענת יציאה**: מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

- **About_Load(object sender, EventArgs e)**

  **טענת כניסה:** אובייקט שולח ואירוע טעינה

  **טענת יציאה:** משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_components.

- **About_Shown(object sender, EventArgs e)**

  **טענת כניסה:** אובייקט שולח ואירוע הצגה

  **טענת יציאה:** יוצרת מילון של רכיבים בחלון ומפעילה את set_profile_picture בצורה אסינכרונית.

- **set_components()**

  **טענת כניסה:** אין

  **טענת יציאה:** מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **About_Resize(object sender, EventArgs e)**

  **טענת כניסה:** אובייקט שולח ואירוע שינוי גודל

  **טענת יציאה:** מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_components.

- **logout_btn_Click(object sender, EventArgs e)**

  **טענת כניסה:** אובייקט שולח ואירוע לחיצה

  **טענת יציאה:** מפעילה את פעולת ההתנתקות מהחשבון ע"י קריאה לפונקציה UI.logout.

- **About_FormClosed(object sender, FormClosedEventArgs e)**

  **טענת כניסה:** אובייקט שולח ואירוע סגירת חלון

  **טענת יציאה:** סוגרת את כל האפליקציה עם קריאה ל־Application.Exit().

### Contact

תפקיד: מציגה את חלון יצירת הקשר והצ'אט עם השרת. אחראית על הצגת ההודעות, שליחתן, האזנה להודעות חדשות, עיצוב הרכיבים הדינאמיים וניתוק בטוח.

תכונות: רכיבי UI בלבד

פעולות:

- **Contact()**

  **טענת כניסה**: אין

  **טענת יציאה:** מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

- **Contact_Load(object sender, EventArgs e)**

  **טענת כניסה:** אירוע טעינה של החלון

  **טענת יציאה:** משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_components.

- **set_components()**

  **טענת כניסה:** אין

  **טענת יציאה:** מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **Contact_FormClosed(object sender, FormClosedEventArgs e)**

  **טענת כניסה:** אירוע סגירת חלון

  **טענת יציאה:** שולח הודעת סיום האזנה לשרת, ממתין לעצירת ההאזנה ואז סוגר את החלון.

- **Contact_Resize(object sender, EventArgs e)**

  **טענת כניסה:** אירוע שינוי גודל

  **טענת יציאה:** מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_components.

- **logout_btn_Click(object sender, EventArgs e)**

  **טענת כניסה:** אירוע לחיצה על כפתור יציאה

  **טענת יציאה:** מפעילה את פעולת ההתנתקות מהחשבון ע"י קריאה לפונקציה UI.logout.

- **Contact_Shown(object sender, EventArgs e)**

  **טענת כניסה:** אירוע הצגה ראשונית של החלון

**טענת יציאה**: טוען את תמונת הפרופיל בצורה אסינכרונית (get_profile_picture), טוען את כל ההודעות מהשרת (get_all_messages) ומפעיל האזנה להודעות חדשות(listen_to_messages)

פעולות פנימיות:

- **get_profile_picture()**

**טענת כניסה:** אין

**טענת יציאה:** קוראת לפעולה UI.set_profile_picture ומציגה את תמונת הפרופיל בצורה אסינכרונית.

- **get_all_messages()**

**טענת כניסה**: אין

**טענת יציאה**: שולח בקשה לשרת לקבל את כל ההודעות מהשיחות הקודמות, מקבל אותן, ומציג כל אחת בחלון דרך create_message

- **textbox_TextChanged(object sender, EventArgs e)**

**טענת כניסה:** אירוע שינוי טקסט בתיבת ההקלדה

**טענת יציאה:** מתאים דינאמית את גובה תיבת הטקסט שמיועדת לכתיבת ההודעות, לפי כמות השורות בה.

- **create_message(string text, string role)**

**טענת כניסה:** טקסט של ההודעה, תפקיד (role) של השולח - לקוח או שרת

**טענת יציאה:** יוצרת כיתוב גרפי חדש של הודעה בצבע ועיצוב תואמים לתפקיד, ומציגה אותה בממשק במיקום המתאים.

- **listen_to_messages()**

**טענת כניסה**: אין

**טענת יציאה**: מתחיל האזנה ברקע להודעות נכנסות מ־TCP, מציג הודעות חדשות או סוגר על פי פקודות מהשרת. מאזינה בלולאה להודעות נכנסות, קוראת אותן ומציגה אותן עם create_message.

- **send_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: אירוע לחיצה על כפתור שליחה

**טענת יציאה**: שולחת את ההודה שבתיבת הטקסט, אם היא לא ריקה, לשרת ומציגה אותה בחלון שלו כלקוח.

## Flights

**תפקיד:** מחלקת Flights אחראית על הצגת חלון חיפוש, הצגה והזמנה של טיסות. היא מנהלת את כל ממשק המשתמש הקשור לבחירת טיסות: חיפוש לפי כתובות מוצא ויעד, תאריכי יציאה וחזרה, מספר נוסעים וסוג מחלקה.
לאחר קבלת הנתונים מהשרת, היא מציגה כרטיסי טיסה רלוונטיים, מאפשרת למשתמש לבחור בטיסה מסוימת ולעבור לחלון הצגת מבנה המטוס לבחירת מושבים. לאחר בחירת מושבים Flights ,מאפשרת הזנת פרטי תשלום וביצוע ההזמנה בפועל מול השרת.
המחלקה כוללת גם טיפול בכל הוויזואליות הדינמית של החלון.

תכונות: רכיבי UI בלבד

**פעולות:**

- **Flights()**

**טענת כניסה**: אין

**טענת יציאה**: מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

- **Flights_Load(object sender, EventArgs e)**

**טענת כניסה**: אובייקט שולח ואירוע טעינה
**טענת יציאה**: משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_components.

- **set_components()**

**טענת כניסה**: אין

**טענת יציאה**: מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **set_tickets(List<ActiveFlight> departure_flights, List<ActiveFlight> return_flights)**

**טענת כניסה**: רשימות של טיסות יוצאות ורשימה של טיסות חוזרות

**טענת יציאה**: מציג את כרטיסי הטיסה הזמינים עם הפרטים המתאימים עבור כל טיסה.

פעולה פנימית:

- **create_ticket(ActiveFlight flight)**

**טענת כניסה**: אין

**טענת יציאה**: מחזיר Panel עם פרטים על הטיסה שהתקבלה. כאשר לוחצים על ה-Panel הפעולה set_plane_cabin מזומנת יחד עם פרטי הטיסה.

- **formClosed(object sender, FormClosedEventArgs e)**

**טענת כניסה**: אירוע סגירה

**טענת יציאה**: סוגרת את האפליקציה וגורמת להתנתקות מהמערכת באמצעות UI.logout().

- **Flights_Resize(object sender, EventArgs e)**

**טענת כניסה**: אירוע שינוי גודל

**טענת יציאה**: מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_components.

- **update_dates(object sender, EventArgs e)**

**טענת כניסה**: אירוע שינוי תאריך

**טענת יציאה**: מוודאת שהתאריכים נבחרו כראוי ומתאימים לתנאים לוגיים למשל: מועד חזרה אחרי מועד יציאה.

- **textbox_focus(object sender, EventArgs e)**

**טענת כניסה**: כניסה לשדה טקסט

**טענת יציאה**: מייצרת תווית עם שם השדה ומזיזה אותה באמצעות פעולת move_labels_animation.

פעולה פנימית:

- **create_label()**

**טענת כניסה**: אין

**טענת יציאה**: יוצרת תווית עם שם השדה

- **move_labels_animation(Label label, Point finish_pos)**

**טענת כניסה**: תווית ומיקום סופי

**טענת יציאה**: מזיזה את התווית באנימציה חלקה מהמיקום הנוכחי למיקום הסופי.

- **textbox_leave(object sender, EventArgs e)**

**טענת כניסה**: עזיבת שדה טקסט

**טענת יציאה**: מחזירה את המצב הוויזואלי לשגרה, מסירה תוויות אם השדה ריק.

- **Flights_Shown(object sender, EventArgs e)**

**טענת כניסה**: אירוע הצגת החלון

**טענת יציאה**: יוצרת מילון של רכיבים בחלון ומפעילה את set_profile_picture בצורה אסינכרונית.

- **logout_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: אירוע לחיצה על כפתור יציאה

**טענת יציאה**: מפעילה את פעולת ההתנתקות מהחשבון ע"י קריאה לפונקציה UI.logout.

- **search_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: אירוע לחיצה על כפתור חיפוש

**טענת יציאה**: שולח בקשת חיפוש לשרת, מקבל את תוצאות החיפוש ומציג באמצעות פעולת set_tickets().

- **validate()**

**טענת כניסה**: אין

**טענת יציאה**: בודקת תקינות של כתובות מוצא ויעד, מחזירה true אם השדות תקינים, אחרת מחזירה false.

- **set_plane_cabin(ActiveFlight flight)**

**טענת כניסה**: טיסה נבחרת. הטיסה מתקבלת כתוצאה מלחיצה על Panel הטיסה אחרי חיפוש הטיסות.

**טענת יציאה**: מציגה את מבנה המטוס כולל המושבים הפנויים והתפוסים לפי סוג מחלקה ומנהלת את תהליך ההזמנת המושבים. במידה וכפתור סיום ההזמה נלחץ הפעולה complete_booking מזומנת.

פעולה פנימית:

- **create_seat(int j,int i,string flight_class)**

**טענת כניסה**: מספר עמודה של המושב, מספר שורה של המושב, מחלקת טיסה

**טענת יציאה**: יוצרת את כפתור המושב במיקום המתאים, מתאימה את הצבע לפי הזמינות שלו (ירוק – פנוי, אפור – תפוס) ומנהלת את הלחיצות על המושב כך שאם לוחצים על המושב הוא מתווסף לרשימת מושבים להזמנה, אך אם לוחצים עליו שוב, הוא יוצא מהרשימה.

- **exit_plane_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: לחיצה על כפתור חזרה

**טענת יציאה**: מסתירה את ממשק המטוס.

- **complete_booking(ActiveFlight flight, List<Seat> selected_seats)**

**טענת כניסה**: אובייקט טיסה ורשימת מושבים נבחרים

**טענת יציאה**: מציג טופס תשלום ובקשת שמות נוסעים נוספים. בלחיצה על כפתור התשלום, הפעולה payment_btn_click מזומנת.

- **payment_btn_click(ActiveFlight flight, List<Seat> selected_seats)**

**טענת כניסה**: טיסה שנבחרה ומושבים שנבחרו

**טענת יציאה**: מאמת פרטי תשלום ושמות הנוסעים הנוספים, ושולח בקשת הזמנה לשרת. מציג סטטוס הצלחה/כישלון בהתאם לתגובת השרת.

פעולה פנימית:

- **validate()**

**טענת כניסה**: אין

**טענת יציאה**: בודקת שהשדות בטופס תקינים – השמות של הנוסעים האחרים תקינים ופרטי התשלום תקינים. אם הפרטים תקינים מחזירה true אחרת מחזירה false.

- **hide_panel()**

**טענת כניסה**: אין

**טענת יציאה**: מסתירה את חלון המטוס באופן אסינכרוני לאחר 3 שניות מאז קבלת התשובה מהשרת ומחזירה לחלון החיפוש.

- **update_payment_dates(object sender, EventArgs e)**

**טענת כניסה**: שינוי בתאריך תפוגת כרטיס

**טענת יציאה**: בודק שהתאריך בתוקף, ואם לא מתקן אותו אוטומטית.

### Home

- **תפקיד**:

המחלקה Home מייצגת את חלון הבית הראשי במערכת CybAir. היא משמשת כעמוד הראשוני והמרכזי של האפליקציה, בו מוצגים מידע כללי, כפתורי קריאה לפעולה (כמו התחברות והרשמה), ואזור המציג יעדים פופולריים ככרטיסים גרפיים.
המחלקה מטפלת בעיצוב של הרכיבים בהתאם לגודל המסך, באנימציות גלילה של הכרטיסים ובהצגת תמונת הפרופיל של המשתמש.

- תכונות: רכיבי UI בלבד

- **פעולות**:
  - **Home()**

    **טענת כניסה**: אין

    **טענת יציאה**: מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

  - **Home_Load(object sender, EventArgs e)**

    **טענת כניסה**: אירוע טעינת חלון

    **טענת יציאה**: משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_controls.

  - **Home_Shown(object sender, EventArgs e)**

    **טענת כניסה**: אירוע הצגת החלון

    **טענת יציאה**: מציג את תמונת הפרופיל באמצעות UI.set_profile_picture, ויוצר את כרטיסי המקומות.

  - **create_places_cards()**

    **טענת כניסה**: אין

    **טענת יציאה**: יוצר כרטיסים גרפיים עם שם ותמונה עבור יעדים פופולריים.

    פעולות פנימיות:

    - **container_setup()**

      **טענת כניסה**: אין

      **טענת יציאה**: מייצרת Panel שמייצג כרטיס גרפי, מתאימה את הגודל שלו והמיקום שלו לפי אורך רשימת הכרטיסים.

    - **image_setup ()**

      **טענת כניסה**: אין

      **טענת יציאה**: מייצרת את התמונה שבתוך הכרטיס, מתאימה את הגודל והמיקום שלה.

    - **label_setup()**

**טענת כניסה**: אין

**טענת יציאה**: מייצרת את התווית שבתוך הכרטיס, מתאימה את הגודל והמיקום שלה והכיתוב שלה.

- **update_place_cards(int offset)**

**טענת כניסה**: מספר פיקסלים להזזה

**טענת יציאה**: משנה את המיקום של כרטיסי היעדים לפי מספר הפיקסלים שהתקבל.

- **set_controls(Control parent)**

**טענת כניסה**: פאנל האב

**טענת יציאה**: מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **Home_Resize(object sender, EventArgs e)**

**טענת כניסה**: אירוע שינוי גודל חלון

**טענת יציאה**: מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_controls.

- **formClosed(object sender, FormClosedEventArgs e)**

**טענת כניסה**: אירוע סגירה

**טענת יציאה**: מבצע Logout, סוגר את האפליקציה ומסיים את התהליך.

- **move_left_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: לחיצה על כפתור החלקה שמאלה

**טענת יציאה**: מזיז את כרטיסי היעדים שמאלה באנימציה ומעדכן את המיקום שלהם באמצעות הפעולה update_place_cards.

- **move_right_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: לחיצה על כפתור החלקה ימינה

**טענת יציאה**: מעביר את הכרטיס הראשון לסוף, מזיז את כרטיסי היעדים שמאלה באנימציה ומעדכן את המיקום שלהם באמצעות הפעולה update_place_cards.

- **logout_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: לחיצה על כפתור יציאה

**טענת יציאה**: מפעילה את פעולת ההתנתקות מהחשבון ע"י קריאה לפונקציה UI.logout.

## Login

- **תפקיד:**

חלון ההתחברות של משתמשים אחראית על קבלת פרטי התחברות, שליחתם לשרת, ניתוח התגובה, והעברה לחלונות הרלוונטיים לפי תוצאה. בנוסף, מטפלת בעיצוב אינטראקטיבי וידידותי למשתמש.

תכונות: רכיבי UI בלבד

פעולות:

- **Login()**

**טענת כניסה**: אין.

**טענת יציאה**: מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

- **Login_Load(object sender, EventArgs e)**

**טענת כניסה :**אובייקט ושולח אירוע טעינת חלון.

**טענת יציאה:** משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_components.

- **set_components()**

**טענת כניסה :**אין.

**טענת יציאה:** מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **textbox_focus(object sender, EventArgs e)**

**טענת כניסה**: כניסה לשדה טקסט

**טענת יציאה**: מייצרת תווית עם שם השדה ומזיזה אותה באמצעות פעולת move_labels_animation.

פעולה פנימית:

  - **create_label()**

**טענת כניסה**: אין

**טענת יציאה**: יוצרת תווית עם שם השדה .

- **move_labels_animation(Label label, Point finish_pos)**

**טענת כניסה**: תווית ומיקום סופי

**טענת יציאה**: מזיזה את התווית באנימציה חלקה מהמיקום הנוכחי למיקום הסופי.

- **textbox_leave(object sender, EventArgs e)**

**טענת כניסה**: עזיבת שדה טקסט

**טענת יציאה**: מחזירה את המצב הוויזואלי לשגרה, מסירה תוויות אם השדה ריק.

- **formClosed(object sender, FormClosedEventArgs e)**

**טענת כניסה :** אירוע סגירת החלון.

**טענת יציאה:** קורא לפעולה UI.logout() לצורך ניתוק תקשורת וסגירה תקינה של המערכת.

- **login_btn_Click(object sender, EventArgs e)**

**טענת כניסה :**אירוע לחיצה על כפתור ההתחברות.

**טענת יציאה:** בודק תקינות של שדות באמצעות פעולת validate. אם הנתונים תקינים שולח את הנתונים לשרת, מחכה לתשובה ומחליט האם להציג את חלון Home או להחזיר שגיאה.

- **validate()**

**טענת כניסה :** אין.

**טענת יציאה:** בודק אם שדות הדוא"ל והסיסמה תקינים ומלאים, אם כן מחזיר true אחרת מחזיר false.

- **Login_Shown(object sender, EventArgs e)**

**טענת כניסה :** אירוע הצגת החלון.

**טענת יציאה:** יוצרת מילון של רכיבים בחלון ומפעילה את set_profile_picture בצורה אסינכרונית.

- **logout_btn_Click(object sender, EventArgs e)**

**טענת כניסה :** אירוע לחיצה על כפתור יציאה.

**טענת יציאה:** מפעילה את פעולת ההתנתקות מהחשבון ע"י קריאה לפונקציה UI.logout.

- **Login_Resize(object sender, EventArgs e)**

**טענת כניסה :**אירוע שינוי גודל חלון.

**טענת יציאה**: מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_components.

- **password_icon_Click(object sender, EventArgs e)**

**טענת כניסה :** אירוע לחיצה על הסמל בשדה הסיסמה

**טענת יציאה:** אם הסיסמה נמצאת במצב סמוי היא הופכת למצב גלוי, אם הסיסמה נמצאת במצב גלוי היא הופכת למצב סמוי.

### Signup

- **תפקיד:**

חלון ההתחברות של משתמשים אחראית על קבלת פרטי התחברות, שליחתם לשרת, ניתוח התגובה, והעברה לחלונות הרלוונטיים לפי תוצאה. בנוסף, מטפלת בעיצוב אינטראקטיבי וידידותי למשתמש.

**תכונות:** רכיבי UI בלבד

**פעולות:**

- **Signup()**

**טענת כניסה**: אין.

**טענת יציאה**: מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

- **Signup_Load(object sender, EventArgs e)**

**טענת כניסה :**אובייקט ושולח אירוע טעינת חלון.

**טענת יציאה:** משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_components.

- **set_components()**

**טענת כניסה :**אין.

**טענת יציאה:** מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **textbox_focus(object sender, EventArgs e)**

**טענת כניסה**: כניסה לשדה טקסט

**טענת יציאה**: מייצרת תווית עם שם השדה ומזיזה אותה באמצעות פעולת move_labels_animation.

פעולה פנימית:

- **create_label()**

**טענת כניסה**: אין

**טענת יציאה**: יוצרת תווית עם שם השדה .

- **move_labels_animation(Label label, Point finish_pos)**

**טענת כניסה**: תווית ומיקום סופי

**טענת יציאה**: מזיזה את התווית באנימציה חלקה מהמיקום הנוכחי למיקום הסופי.

- **textbox_leave(object sender, EventArgs e)**

**טענת כניסה**: עזיבת שדה טקסט

**טענת יציאה**: מחזירה את המצב הוויזואלי לשגרה, מסירה תוויות אם השדה ריק.

- **formClosed(object sender, FormClosedEventArgs e)**

**טענת כניסה :** אירוע סגירת החלון.

**טענת יציאה:** קורא לפעולה ()UI.logout לצורך ניתוק תקשורת וסגירה תקינה של המערכת.

- **Signup_btn_Click(object sender, EventArgs e)**

**טענת כניסה :**אירוע לחיצה על כפתור ההרשמה.

**טענת יציאה:** בודק תקינות של שדות באמצעות פעולת validate_part1 ו-validate_part2. אם הנתונים תקינים שולח את הנתונים לשרת, מחכה לתשובה ומחליט האם להציג את חלון Home או להחזיר שגיאה.

- **Validate_part1()**

**טענת כניסה :** אין.

**טענת יציאה:** בודק אם שדות הדוא"ל, הסיסמה והשם המלא תקינים ומלאים, אם כן מחזיר true אחרת מחזיר false.

- **Validate_part2()**

**טענת כניסה :** אין.

**טענת יציאה:** בודק אם שדות תאריך הלידה, הכתובת ותמונת הפרופיל תקינים (לא בהכרח חייבים להיות מלאים), אם כן מחזיר true אחרת מחזיר false.

- **Signup_Shown(object sender, EventArgs e)**

**טענת כניסה :** אירוע הצגת החלון.

**טענת יציאה:** יוצרת מילון של רכיבים בחלון ומפעילה את set_profile_picture בצורה אסינכרונית.

- **logout_btn_Click(object sender, EventArgs e)**

**טענת כניסה :** אירוע לחיצה על כפתור יציאה.

**טענת יציאה:** מפעילה את פעולת ההתנתקות מהחשבון ע"י קריאה לפונקציה UI.logout.

- **Signup_Resize(object sender, EventArgs e)**

**טענת כניסה :**אירוע שינוי גודל חלון.

**טענת יציאה**: מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_components.

- **password_icon_Click(object sender, EventArgs e)**

**טענת כניסה :** אירוע לחיצה על הסמל בשדה הסיסמה

**טענת יציאה:** אם הסיסמה נמצאת במצב סמוי היא הופכת למצב גלוי, אם הסיסמה נמצאת במצב גלוי היא הופכת למצב סמוי.

**convert_image(Control control)**

**טענת כניסה :** אלמנט שמכיל תמונה

**טענת יציאה:** הופך את התמונה למערך בתים ומחזיר אותו.

## MyBookings

- **תפקיד:**

חלון ההתחברות של משתמשים אחראית על קבלת פרטי התחברות, שליחתם לשרת, ניתוח התגובה, והעברה לחלונות הרלוונטיים לפי תוצאה. בנוסף, מטפלת בעיצוב אינטראקטיבי וידידותי למשתמש.

תכונות: רכיבי UI בלבד

- פעולות:

- **MyBookings()**

**טענת כניסה**: אין.

**טענת יציאה**: מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

- **MyBookings_Load(object sender, EventArgs e)**

**טענת כניסה :**אובייקט ושולח אירוע טעינת חלון.

**טענת יציאה:** משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_components.

- **set_components()**

**טענת כניסה :**אין.

**טענת יציאה:** מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **formClosed(object sender, FormClosedEventArgs e)**

**טענת כניסה :** אירוע סגירת החלון.

**טענת יציאה:** קורא לפעולה ()UI.logout לצורך ניתוק תקשורת וסגירה תקינה של המערכת.

- **MyBookings_Shown(object sender, EventArgs e)**

**טענת כניסה :** אירוע הצגת החלון.

**טענת יציאה:** מזמן את הפעולה UI.set_profile_picture כדי להציג את תמונת הפרופיל של המשתמש ואת הפעולה get_bookings כדי לקבל את המידע על כל ההזמנות של המשתמש

- **get_bookings()**

**טענת כניסה :** אין.

**טענת יציאה:** שולח לשרת בקשה לקבל את ההזמנות של המשתמש ומחכה לתגובה מהשרת. אחרי שהמידע על ההזמנות מתקבל הפעולה show_bookings מזומנת.

- **show_bookings(List<ActiveFlight> flights, List<BookedFlight> booked_flights)**

**טענת כניסה :** רשימת טיסות שהוזמנו מטיפוס ActiveFlight ורשימת טיסות שהוזמנו מטיפוס BookedFlight .

**טענת יציאה:** מציג את כרטיסי הטיסה של המשתמש וקורא לפונקציה handle_option_click בלחיצה על אפשרות מפאנל האפשרויות.

**פעולה פנימית:**

- **create_ticket(ActiveFlight flight,Seat seat)**

**טענת כניסה :** אובייקט טיסה מטיפוס ActiveFlight ואובייקט מושב מסוג Seat .

**טענת יציאה:** מייצר Panel עם כל הנתונים הנחוצים על כרטיס הטיסה ומחזיר אותו.

- **handle_ticket_click(object sender, EventArgs e)**

**טענת כניסה :** אירוע לחיצה על כרטיס טיסה.

**טענת יציאה:** מעדכן את מיקום פאנל האפשרויות והופך אותו מחבוי לנראה.

- **handle_option_click(object sender, EventArgs e)**

**טענת כניסה :** אירוע לחיצה על אפשרות בפאנל האפשרויות.

**טענת יציאה:** מזמן את הפונקציה המתאימה לפי האפשרות שנבחרה ( view map, cancel ticket, save ticket).

- **save_ticket(Control ticket)**

**טענת כניסה:** אלמנט כרטיס טיסה

**טענת יציאה:** הופך את האלמנט לתמונה ופותח את חלון SaveDialog שמאפשר למשתמש לבחור את מקום השמירה של התמונה.

- **cancel_ticket(ActiveFlight flight,Seat seat,Control ticket)**

**טענת כניסה:** אלמנט כרטיס טיסה, מושב מטיפוס Seat, טיסה מטיפוס ActiveFlight

**טענת יציאה:** שולח בקשה לשרת לבטל את הכרטיס בטיסה והמיקום שהתקבלו. אם התגובה מהשרת מוצלחת, קורא לפעולה get_bookings כדי לעדכן את ממשק המשתמש על הביטול.

- **get_map_data(ActiveFlight flight)**

**טענת כניסה:** טיסה מטיפוס ActiveFlight

**טענת יציאה:** שולח בקשה לשרת לקבל מידע על המפה. אם התשובה של השרת מוצלחת הפעולה display_map מזומנת.

- **Gmap_OnMapClick(PointLatLng pointClick, System.Windows.Forms.MouseEventArgs e)**

**טענת כניסה:** אירוע לחיצה על המפה ונקודת הלחיצה

**טענת יציאה:** מסתיר את תווית המידע.

- **Gmap_OnMarkerClick(GMapMarker item, GMapControl gmap)**

**טענת כניסה:** סמן במפה והמפה עצמה

**טענת יציאה:** חושף את תווית המידע עם המידע המעודכן ומשנה את מיקומה באמצעות פעולה UpdateLabelPosition.

- **UpdateLabelPosition(GMapMarker item ,GMapControl gmap)**

**טענת כניסה:** סמן במפה והמפה עצמה

**טענת יציאה:** מעדכן את מיקום תווית המידע לפי הסמן במפה.

- **logout_btn_Click(object sender, EventArgs e)**

**טענת כניסה :** אירוע לחיצה על כפתור יציאה.

**טענת יציאה:** מפעילה את פעולת ההתנתקות מהחשבון ע"י קריאה לפונקציה UI.logout.

- **MyBookings_Resize(object sender, EventArgs e)**

**טענת כניסה :**אירוע שינוי גודל חלון.

**טענת יציאה**: מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_components.

## Server

**תפקיד:** מטרת המחלקה היא לנהל את כל החיבורים עם לקוחות אחרים דרך הרשת באמצעות חיבור TCP, לקבל בקשות ולשלוח הודעות. בנוסף המחלקה מאפשרת קבלה ושליחת הודעות ללקוחות אחרים שיוצרים קשר.

**תכונות:**

- רכיבי UI

תפקיד ושימוש: מהווים חלקים מתוך ממשק המשתמש

- port

תפקיד ושימוש: מאפשר ללקוחות לדעת לאיזה port להתחבר בעת יצירת חיבור ה-TCP

- IP

תפקיד ושימוש: מועבר לכל הלקוחות ומאפשר לדעת לאיזו כתובת IP להתחבר בעת יצירת חיבור ה-TCP

- is_running

תפקיד ושימוש: מהווה תנאי לפעילות של התהליכון הראשי שמאזין באופן אינסופי לחיבור לקוחות. ברגע שהוא מבוטל ההאזנה מפסיקה.

- users_file_path

תפקיד ושימוש: מקל על פעולות שדורשות שינוי או עיבוד המידע בקובץ

- flights_file_path

תפקיד ושימוש: מקל על פעולות שדורשות שינוי או עיבוד המידע בקובץ

- rsa

תפקיד ושימוש: מועבר לכל הלקוחות ומאפשר להם להשתמש בו להצפנת המפתח הסימטרי האישי של כל לקוח.

- current_chat_user

תפקיד ושימוש: אחראי על המשתמש הנוכחי שהמשתמש בשרת לוחץ עליו בצ'אט ויקבל את ההודעה שתישלח

- **פעולות**:

- **Server()**

**טענת כניסה**: אין

**טענת יציאה**: מאתחלת את הרכיבים הגרפיים של החלון, ומפעילה הגדרות לייעול ציור החלון.

- **Server_Load(object sender, EventArgs e)**

**טענת כניסה**: אירוע טעינת חלון

**טענת יציאה**: משנה את גודל החלון בהתאם לגודל המסך, ומפעילה את פונקציית set_components. בנוסף מזמנת את הפעולה UDP.deny_others שמונעת מאחרים להפוך לשרת, ומתחילה את התהליכון הראשי שמאזין לחיבור לקוחות לשרת.

- **Server_Shown(object sender, EventArgs e)**

**טענת כניסה**: אירוע הצגת החלון

**טענת יציאה**: עבור כל משתמש רשום, נוצר צ'אט חדש באמצעות הפעולה create_chat_user

- **set_components()**

**טענת כניסה**: אין

**טענת יציאה**: מגדירה מיקומים וגדלים של כל הרכיבים בחלון בהתאם לגודל החלון. משתמשת בפונקציות עזר ממחלקת UI.

- **create_chat_user(User u ,Server form)**

**טענת כניסה**: אובייקט משתמש מסוג User, טופס מטיפוס Server

**טענת יציאה**: מייצר Panel שבו פרטי המשתמש כמו: תמונת פרופיל, שם וכתובת דוא"ל. בלחיצה על ה-Panel עדכון תכונת current_chat_user, הצגת כל ההודעות באמצעות פעולת create_message.

- **create_message(string text, string role,Server form)**

**טענת כניסה**: מחרוזת ההודעה, תפקיד (לקוח/שרת) וטופס מטיפוס Server

**טענת יציאה**: מייצרת תווית הודעה בצבע והמיקום המתאימים לפי התפקיד.

- **send_btn_Click(object sender, EventArgs e)**

**טענת כניסה**: אירוע לחיצה על כפתור השליחה

**טענת יציאה**: בדיקה שההודעה שנשלחה לא ריקה ושהמשתמש בשרת אכן לחץ על אחד המשתמשים. הוספת ההודעה למאגר ההודעות של המשתמש. הצגת ההודעה על מסך השרת. במידה והמשתמש מחובר, מוצא את החיבור TCP ומפתח ההצפנה הסימטרי ושולח את ההודעה ללקוח כדי שהיא תוצג בזמן גם אצלו.

- **add_notification(Server form,User user)**

**טענת כניסה**: טופס מטיפוס Server ואובייקט משתמש מטיפוס User

**טענת יציאה**: מייצר תווית התראה בחלון של השרת, ליד ה-Panel של המשתמש, אם היא לא קיימת אחרת, מוסיף אחד לערך שבתוכה.

- **textbox_TextChanged(object sender, EventArgs e)**

  **טענת כניסה**: אירוע שינוי כיתוב

  **טענת יציאה**: עדכון הגובה של תיבת הטקסט הראשית בהתאם לכמות הכיתוב בהודעה
- **Home_Resize(object sender, EventArgs e)**

  **טענת כניסה**: אירוע שינוי גודל חלון

  **טענת יציאה**: מבצעת התאמה מחודשת של רכיבי הממשק במקרה של שינוי גודל חלון (אלא אם מוקטן למינימום) ומזמנת את פעולת set_controls.
- **formClosed(object sender, FormClosedEventArgs e)**

  **טענת כניסה**: אירוע סגירה

  **טענת יציאה**: סוגר את האפליקציה ומסיים את התהליך.

### ClientHandler

תפקיד:

המחלקה מטפלת בכל התקשורת בין השרת לבין לקוח יחיד במערכת. היא מקבלת חיבור TCP מהלקוח, יוצרת ערוצי קריאה, כתיבה והצפנה, ומתחילה להאזין ברקע לבקשות מוצפנות מהלקוח. כל בקשה מזוהה לפי פרוטוקול פעולה ומועברת לפעולה המתאימה, כמו התחברות, הרשמה, חיפוש טיסות, הזמנות, הודעות צ'אט, ביטולים ועוד. לאחר ביצוע הפעולה, נשלחת תגובת שרת מוצפנת ללקוח. המחלקה אחראית גם על עדכון קבצי המערכת, ניהול מצב המשתמש, סגירת תקשורת תקינה, והצגת מידע ל־UI של חלון השרת.

**תכונות:**

- tcpClient

  תפקיד ושימוש: מייצג את חיבור ה־TCP הפעיל מול הלקוח
- writer

  תפקיד ושימוש: מאפשר כתיבה של טקסט על גבי החיבור
- reader

  תפקיד ושימוש: מאפשר קריאה של טקסט על גבי החיבור
- stream

  תפקיד ושימוש: מייצג את זרם הנתונים מהלקוח לצורך הצפנה

- form

תפקיד ושימוש: מייצג את חלון השרת לצורך עדכון UI

- user

תפקיד ושימוש: מייצג את המשתמש שמחובר לחיבור הנוכחי

- Is_user_active

תפקיד ושימוש: מאפשר הפסקת Thread ההאזנה בצורה בטוחה

- security

תפקיד ושימוש: מאפשר לבצע פענוח וכתיבה של הודעות מוצפנות על גבי החיבור עם המשתמש הנוכחי

**פעולות:**

- **ClientHandler(TcpClient tcpClient, Server form)**

**טענת כניסה:** אובייקט חיבור TCP ואובייקט חלון השרת

**טענת יציאה:** מאתחלת את כל רכיבי התקשורת וההצפנה, רושמת את אירועי הסגירה של החלון ומפעילה Thread שמאזין באופן קבוע להודעות מהלקוח

- **main_socket_thread()**

**טענת כניסה:** אין

**טענת יציאה:** קוראת הודעות מוצפנות מהלקוח בלולאה אינסופית, מזהה את סוג הבקשה ושולחת את ההודעה לפעולה המתאימה, ולאחר העיבוד מחזירה ללקוח תגובה מוצפנת עם תוצאה מתאימה

- **handle_login(ref string message)**

**טענת כניסה:** מחרוזת JSON עם אימייל וסיסמה של המשתמש

**טענת יציאה:** בודקת אם המשתמש קיים ומורשה, ואם אינו מחובר כבר, מוסיפה אותו לרשימת המשתמשים המחוברים ושולחת חיווי הצלחה או שגיאה לפי התוצאה

- **handle_signup(ref string message)**

**טענת כניסה:** מחרוזת JSON המכילה את כל פרטי המשתמש החדש

**טענת יציאה:** אם כל הנתונים תקינים והמשתמש לא קיים כבר במערכת, הוא נוסף לקובץ המשתמשים והמחוברים ומקבל חיווי הצלחה, אחרת מוחזרת הודעת כישלון או קיום מוקדם

- **handle_logout(ref string message)**

**טענת כניסה:** מחרוזת פעולה כללית

**טענת יציאה:** במידה והמשתמש מחובר, הוא מוסר מרשימת המשתמשים הפעילים, Thread ההאזנה נעצר, והמשתמש מאופס, אחרת מוחזרת שגיאה

- **handle_search(ref string message)**

**טענת כניסה:** JSON עם פרטי החיפוש כולל תאריכים, יעדים, מספר נוסעים וסוג מחלקה

**טענת יציאה:** מתבצע חיפוש טיסות הלוך וחזור התואמות לפרמטרים, בהתאם למועדים ותנאים, והתוצאות מוחזרות ללקוח בפורמט JSON, או שגיאה במקרה הצורך

- **handle_booking(ref string message)**

**טענת כניסה:** מחרוזת הכוללת את פרטי הטיסה המבוקשת ואת רשימת המושבים שנבחרו

**טענת יציאה:** במידה והמושבים זמינים בטיסה, הם נשמרים למשתמש והטיסה מתעדכנת, אחרת מוחזרת הודעה מתאימה של כישלון או קיום מוקדם

- **handle_cancelation(ref string message)**

**טענת כניסה:** מחרוזת עם מזהי טיסה ומושב לביטול

**טענת יציאה:** המושב מוסר מהזמנת המשתמש ומהטיסה הרלוונטית, ואם הטיסה נותרת פעילה היא נשמרת מחדש, אחרת התהליך נכשל ומוחזרת הודעת שגיאה

- **send_bookings(ref string message)**

**טענת כניסה:** מחרוזת כללית

**טענת יציאה:** כל ההזמנות של המשתמש מאותרות, פרטי הטיסות מתאימים נשלפים, והתוצאה מוחזרת בפורמט JSON הכולל גם את ההזמנות וגם את פרטי הטיסות

- **send_map_data(ref string message)**

**טענת כניסה:** JSON עם פרטי טיסה

**טענת יציאה:** מאתרים את כל המשתמשים שטסים באותה טיסה וכוללים כתובת תקפה, מוחקים מידע רגיש כמו מייל וסיסמה, ושולחים רשימה נקייה ללקוח

- **handle_chat(string message)**

**טענת כניסה:** JSON של הודעת צ'אט שנשלחה ע"י הלקוח

**טענת יציאה:** ההודעה מתווספת לרשימת ההודעות של המשתמש, ואם הצ'אט פתוח מוצגת לשרת, ואם לא נוצרת התראה עבור המשתמש, הכל מתבצע דרך ה־UI של השרת

- **send_messages(ref string message)**

**טענת כניסה:** מחרוזת כללית

**טענת יציאה:** מחזיר את כל הודעות הצ'אט של המשתמש ללקוח, מעדכן את סטטוס המשתמש כ"מחובר בצ'אט", ואם מתרחשת תקלה מוחזרת שגיאה

- **getProfilePicture(ref string message)**

**טענת כניסה:** מחרוזת כללית

**טענת יציאה:** אם המשתמש מחובר והנתונים תקינים, מוחזר שמו עם או בלי תמונת פרופיל מקודדת ב־Base64, ואם יש בעיה מוחזרת שגיאה

## City

- **תפקיד:**

המחלקה מייצגת עיר בעולם וכוללת מידע גאוגרפי בסיסי שנדרש להצגת מיקומים, חיפוש טיסות, בניית מפה, ושימושים שונים במערכת CybAir.

- **תכונות:**
  - city

תפקיד ושימוש: שם העיר, משמש לזיהוי העיר בתהליכים כמו חיפוש טיסות או הצגת מיקום

  - country

תפקיד ושימוש: שם המדינה, נלווה לשם העיר ליצירת מחרוזת מלאה כמו " Paris, France"

  - lat

תפקיד ושימוש: קו רוחב, מציין את המיקום הצפוני־דרומי של העיר על פני כדור הארץ, מאפשר למקם את העיר על מפת העולם

  - lng

תפקיד ושימוש: קו אורך, מציין את המיקום המזרחי־מערבי של העיר על פני כדור הארץ, מאפשר למקם את העיר על מפת העולם

  - iso2

תפקיד ושימוש: קוד המדינה באורך 2 תווים

  - iso3

תפקיד ושימוש: קוד המדינה באורך 3 תווים

- **פעולות:**

אין פעולות במחלקה זו, כל השדות הם לצורך אחסון ושליפה של מידע

## Seat

**תפקיד:**

Seat מייצגת מושב יחיד במטוס בתוך מערכת ההזמנות. כל מופע מתאר את מיקום המושב לפי שורה ועמודה, לאיזו מחלקה הוא שייך (רגילה, ביזנס, ראשונה) ואם הוזמן, גם את שם הנוסע שיושב בו. המחלקה משמשת לבניית מבנה המטוס, ניהול זמינות מושבים, ושיוך מושבים להזמנות משתמש.

- **תכונות:**
  - row

  תפקיד ושימוש: מספר השורה של המושב, משמש למיקום ויזואלי ולאימות כפילויות
  - column

  תפקיד ושימוש: מספר העמודה של המושב, יחד עם השורה מזהה את מיקומו המדויק
  - name

  תפקיד ושימוש: שם הנוסע שישב במושב, משמש בעת הצגת שמות בהזמנה
  - flight_class

  תפקיד ושימוש: סוג המחלקה של המושב, יכול להיות Economy, Business או First

- **פעולות:**
- **Seat()**

  **טענת כניסה:** אין

  **טענת יציאה:** יוצר אובייקט מושב ריק ללא הגדרת שורה, עמודה או מחלקה, מיועד לאתחול ראשוני

- **Seat(int row, int column, string flight_class)**

  **טענת כניסה:** מספר שורה, מספר עמודה, סוג מחלקה

  **טענת יציאה:** יוצר מושב עם מיקום מוגדר וסוג מחלקה, ללא נוסע משויך, ומגדיר את name כמחרוזת ריקה

- **Seat(int row, int column, string flight_class, string name)**

**טענת כניסה:** מספר שורה, מספר עמודה, סוג מחלקה ושם הנוסע

**טענת יציאה:** יוצר מושב מוגדר ומלא, עם מיקום, סוג מחלקה ושם נוסע שישב בו בפועל

### BookedFlight

**תפקיד:**

המחלקה מייצגת טיסה שהוזמנה בפועל על ידי משתמש. היא כוללת את מזהה הטיסה (שמשלב מספר טיסה ותאריך) ואת רשימת המושבים שהוזמנו במסגרת אותה טיסה. מחלקה זו משמשת לאחסון הנתונים של הזמנות המשתמש, להצגת היסטוריית הזמנות ולביצוע פעולות כמו ביטול או הצגת המושבים. המחלקה לא מציגה את כל נתוני הטיסה אך היא מאפשרת למצוא אותם באמצעות הנתונים המזהים שהיא מכילה.

- **תכונות:**
- flight

תפקיד ושימוש: מזהה טיסה בפורמט "flight_number|date", משמש לזיהוי ייחודי של טיסה מסוימת

- seats

תפקיד ושימוש: רשימת מושבים שהוזמנו בטיסה, כולל פרטים כמו שורה, עמודה וסוג מחלקה

- **פעולות:**
- **BookedFlight ()**

**טענת כניסה:** אין

**טענת יציאה:** יוצר אובייקט הזמנה ריק ללא טיסה או מושבים, מיועד לאתחול כללי או קריאה מקובץ

- **BookedFlight(string flight, List<Seat> seat)**

**טענת כניסה:** מזהה טיסה ורשימת מושבים

**טענת יציאה:** יוצר אובייקט הזמנה הכולל טיסה ספציפית ומושבים משויכים, כפי שנבחרו על ידי המשתמש בעת ההזמנה

## User

**תפקיד:**

מייצגת משתמש רשום. כולל פרטים אישיים, נתוני התחברות, הזמנות, מצב חיבור לצ'אט ותמונת פרופיל. מחלקה זו היא חלק מרכזי במערכת ומשמשת לאימות משתמשים, ניהול הזמנות, צ'אט בין נוסעים לשרת ועבודה עם כל המידע של המשתמש.

**תכונות:**

- name

תפקיד ושימוש: שם המשתמש, משמש לצורך תצוגה בפרופיל ובצ'אט

- email

תפקיד ושימוש: כתובת הדוא"ל של המשתמש, משמשת לזיהוי ומאפיין מייחד של כל משתמש

- password

תפקיד ושימוש: הסיסמה של המשתמש, נשמרת כמחרוזת מוצפנת(Base64)

- salt

תפקיד ושימוש: ערך ה־ salt ששימש להצפנת הסיסמה, לצורך אימות עתידי

- birth_date

תפקיד ושימוש: תאריך הלידה של המשתמש, משמש לבדיקת גיל מינימלי והצגה לצורך היכרות במפה

- address

תפקיד ושימוש: כתובת מלאה בפורמט "עיר, מדינה", נבחרת מרשימת ערים קיימות

- profile_picture

תפקיד ושימוש: תמונת פרופיל בפורמט בינארי, מוצגת בפרופיל ובצ'אט

- loged_in

תפקיד ושימוש: מציין אם המשתמש מחובר כעת למערכת

- booked_flights

תפקיד ושימוש: רשימת טיסות שהוזמנו על ידי המשתמש, משומש להצגה בחלון ההזמנות

- chat_messages

תפקיד ושימוש: היסטוריית ההודעות שנשלחו/התקבלו בצ'אט, מוצג בחלון יצירת הקשר

- chat_online

תפקיד ושימוש: מציין אם המשתמש נמצא כעת במצב צ'אט פעיל, מאפשר לשרת לדעת איך לשלוח את ההודעה.

**פעולות:**

- **User ()**

**טענת כניסה:** אין

**טענת יציאה:** יוצר אובייקט משתמש ריק עם רשימות ריקות של הזמנות והודעות, ומגדיר את המשתמש כלא מחובר וללא salt

- **User(string name, string email, string password, DateTime birth_date, string address, byte[] profile_picture)**

**טענת כניסה:** שם, כתובת דוא"ל, סיסמה, תאריך לידה, כתובת ותמונת פרופיל

**טענת יציאה:** יוצר משתמש מלא עם פרטים אישיים, רשימות ריקות להזמנות והודעות, ומסמן אותו כלא מחובר למערכת ולצ'אט

- **flight_booked(ActiveFlight flight)**

**טענת כניסה:** אובייקט טיסה מסוגActiveFlight

**טענת יציאה:** בודק אם למשתמש קיימת כבר הזמנה עבור הטיסה שנשלחה, על ידי השוואה בין מזהה הטיסה והתאריך, ומחזיר true אם קיימת או false אם לא.

### Flight

**תפקיד:**

המחלקה מייצגת טיסה כללית לפני שמוזמנים לה מושבים בפועל. היא כוללת את פרטי הטיסה הקבועים כמו שדות תעופה, ערים, מדינות, מחיר, זמינות בימים מסוימים ושעות פעילות. מחלקה זו משמשת לבניית טיסות פעילות, להצגת נתונים בחיפוש טיסות. המחלקה משמשת כהמרה של המידע בקובץ flights.csv לאובייקט טיסה שניתן להשתמש בו במערכת.

**תכונות:**

- id

תפקיד ושימוש: מזהה ייחודי של הטיסה, משמש לקישור בין טיסות פעילה לתוכנית

- origin_airport

תפקיד ושימוש: שם שדה התעופה שממנו יוצאת הטיסה

- origin_city

תפקיד ושימוש: העיר שממנה יוצאת הטיסה

- origin_country

תפקיד ושימוש: המדינה שממנה יוצאת הטיסה

- destination_airport

תפקיד ושימוש: שם שדה התעופה שאליו מגיעה הטיסה

- destination_city

תפקיד ושימוש: העיר שאליה מגיעה הטיסה

- destination_country

תפקיד ושימוש: המדינה שאליה מגיעה הטיסה

- price

תפקיד ושימוש: המחיר הבסיסי של כרטיס טיסה בטיסה זו, מחלקת Economy

- flights_per_day

תפקיד ושימוש: מספר הפעמים שטיסה זו מתקיימת ביום אחד

- active_days

תפקיד ושימוש: מחרוזת שמציינת באילו ימים בשבוע הטיסה פועלת, בפורמט של מספרים (1–7)

- times

תפקיד ושימוש: מחרוזת עם רשימת השעות שבהן מתקיימת הטיסה במהלך היום

- **פעולות:**

אין פעולות במחלקה זו, כל השדות הם לצורך אחסון ושליפה של מידע

## ActiveFlight

**תפקיד:**

המחלקה מייצגת טיסה פעילה שמתרחשת בפועל בתאריך ושעה מסוימים, בניגוד למחלקת Flight שמתארת טיסה תיאורטית. המחלקה יורשת את כל המאפיינים הכלליים ממחלקת Flight ומוסיפה מידע ספציפי למופע טיסה יחיד, כולל תאריך, שעת יציאה, מספר טיסה, ומבנה של מושבים מחולקים לפי מחלקות. היא משמשת לניהול הזמנות מושבים, בדיקת זמינות, ביטולים ועדכון מבנה הטיסה לפי השינויים.

**תכונות:**

- flight_number

תפקיד ושימוש: מזהה ייחודי לטיסה לפי מספר טיסה ושעת יציאה, מחושב אוטומטית

- date

תפקיד ושימוש: תאריך שבו הטיסה מתקיימת בפועל

- time

תפקיד ושימוש: שעת יציאה של הטיסה באותו יום

- first_class_rows

תפקיד ושימוש: מספר שורות במחלקה ראשונה, קבוע על 5. משמש למיפוי והצגת המושבים

- buissness_class_rows

תפקיד ושימוש: מספר שורות במחלקת עסקים, קבוע על 5. משמש למיפוי והצגת המושבים

- economy_class_rows

תפקיד ושימוש: מספר שורות במחלקת תיירים, קבוע על 25. משמש למיפוי והצגת המושבים

- seats_per_row

תפקיד ושימוש: מספר מושבים בכל שורה, קבוע על 6. משמש למיפוי והצגת המושבים

- seats

תפקיד ושימוש: מילון שמכיל רשימות של מערכי bool, שמייצגים זמינות של מושבים לפי מחלקה, משמש כמיפוי מקומות הישיבה וזמינותם.

**פעולות:**

- **copy(Flight baseFlight, DateTime date, string time)**

  **טענת כניסה:** אובייקט Flight, תאריך, ושעת יציאה

  **טענת יציאה:** מעתיק את כל הנתונים מהטיסה הבסיסית, יוצר מזהה טיסה ייחודי, מאתחל מבנה מושבים ריק לפי מחלקות ומגדיר אותו לפי התכונות שעוסקות במאפייני המושבים

- **empty_flight()**

  **טענת כניסה:** אין

  **טענת יציאה:** מחזיר true אם כל המושבים בטיסה פנויים, כלומר אף מושב לא הוזמן, אחרת מחזירfalse

- **available(int passengers, string flight_class)**

  **טענת כניסה:** מספר נוסעים וסוג מחלקה

  **טענת יציאה:** סופר את מספר המושבים הפנויים במחלקה שצוינה ובודק אם הכמות מספיקה לכל הנוסעים, אם כן מחזיר true אחרת מחזיר false

- **update_seats(List<Seat> selected_seats)**

  **טענת כניסה:** רשימת מושבים שנבחרו להזמנה

  **טענת יציאה:** בודק אם כל המושבים ברשימה עדיין פנויים, ואם כן מסמן אותם כתפוסים ומעדכן את מבנה המושבים של הטיסה, מחזיר true אם הצליח או false אם חלק מהמושבים כבר תפוסים

- **cancel_seat(Seat s)**

  **טענת כניסה:** מושב בודד מסוג Seat

  **טענת יציאה:** מאתר את המושב בטיסה לפי שורה ועמודה, ומעדכן אותו כלא תפוס במבנה המושבים של הטיסה

### FlightSearchData

**תפקיד:**

המחלקה מייצגת את כל נתוני החיפוש שטופלו ונשלחו מהמשתמש כאשר הוא מחפש טיסות. היא כוללת את מקור ויעד הטיסה, תאריכים להמראה וחזרה, מספר נוסעים וסוג

מחלקה נדרש. מחלקה זו משמשת להעברת בקשת חיפוש מסודרת לשרת ולמציאת הטיסות המתאימות בזמן אמת בהתאם לקריטריונים.

**תכונות:**

- origin

תפקיד ושימוש: מחרוזת המציינת את העיר והמדינה שמהן תצא הטיסה

- destination

תפקיד ושימוש: מחרוזת המציינת את העיר והמדינה אליהן תגיע הטיסה

- departure_date

תפקיד ושימוש: תאריך ההמראה של הטיסה המבוקשת

- return_date

תפקיד ושימוש: תאריך החזרה של טיסת החזור

- passengers

תפקיד ושימוש: מספר הנוסעים הכלולים בבקשת החיפוש

- flight_class

תפקיד ושימוש: סוג המחלקה שהמשתמש מעוניין להזמין בה, לדוגמה Economy

**פעולות:**

- **FlightSearchData(string origin, string destination, DateTime departure_date, DateTime return_date, int passengers, string flight_class)**

**טענת כניסה:** מחרוזות מקור ויעד, תאריך יציאה, תאריך חזרה, מספר נוסעים וסוג מחלקה

**טענת יציאה:** יוצר אובייקט חיפוש טיסה מלא עם כל הנתונים הדרושים לביצוע חיפוש מותאם אישית, המאוחסן ומועבר לשרת לצורך עיבוד והחזרת תוצאות מתאימות

## ChatMessage

**תפקיד:**

המחלקה מייצגת הודעת צ'אט בודדת בין שני השרת והמשתמש. כל הודעה כוללת את תוכן ההודעה ואת התפקיד של השולח (למשל לקוח או שרת). המחלקה משמשת לבניית היסטוריית צ'אט, להצגה ב־ UI של הצ'אט, ולשמירה בקבצי המשתמשים לצורך שליפה עתידית.

**תכונות:**

- message

תפקיד ושימוש: מחרוזת המייצגת את תוכן ההודעה שנשלחה בשיחה

- role

תפקיד ושימוש: תפקיד השולח של ההודעה לדוגמה client או server

**פעולות:**

- **ChatMessage ()**

**טענת כניסה:** אין

**טענת יציאה:** יוצר אובייקט הודעה ריק ללא תוכן או תפקיד, משמש לאתחול בסיסי

- **ChatMessage (string message, string role)**

**טענת כניסה:** מחרוזת תוכן ההודעה ותפקיד השולח

**טענת יציאה:** יוצר הודעת צ'אט מלאה עם טקסט ותפקיד, מוכנה לשמירה, הצגה או שליחה דרך מערכת הצ'אט של האפליקציה

### Users

**תפקיד:**

מחלקה שמנהלת את קובץ המשתמשים. היא כוללת פעולות לקריאת כל המשתמשים, הוספת משתמש, בדיקת קיום משתמש, הסרת משתמש, ועדכון הזמנות בטיסה. המחלקה פועלת ישירות על קובץ JSON שבו מאוחסנים כל המשתמשים הרשומים, ומשמשת כבסיס לכל פעולת גישה או עדכון בנתוני המשתמשים מצד השרת.

**פעולות:**

- **get_users(string file_path)**

**טענת כניסה:** נתיב לקובץ JSON המכיל את המשתמשים

**טענת יציאה:** קורא את הקובץ, מפענח אותו לרשימת משתמשים ומחזיר אותה, זורק חריגה אם הקובץ לא נמצא

- **add_user(User user, string file_path)**

**טענת כניסה:** אובייקט משתמש חדש ונתיב לקובץ JSON

**טענת יציאה:** קורא את רשימת המשתמשים הקיימת, מוסיף את המשתמש החדש, ומעדכן את הקובץ לאחר סידור הפורמט

- **user_found(List<User> users, string email, string password, bool is_login_check)**

**טענת כניסה: רשימת משתמשים,** דוא"ל, סיסמה, ודגל בדיקת התחברות

**טענת יציאה:** עובר על רשימת המשתמשים ובודק אם הדוא"ל של אחד מהם תואם את הדוא"ל שהתקבל. אם הערך is_loging_check שווה true אז מתבצעת השוואת סיסמאות, אחרת מוחזר הערך true. אם הסיסמאות זהות מוחזר הערך true בכל מצב אחר מוחזר false.

- **get_user(List<User> users, string email)**

**טענת כניסה:** רשימת משתמשים ואימייל לחיפוש

**טענת יציאה:** מחפש את המשתמש לפי דוא"ל ומחזיר את האובייקט המתאים, או null אם לא נמצא או התרחשה שגיאה

- **remove_user(string file_path, User user)**

**טענת כניסה:** נתיב לקובץ JSON ואובייקט משתמש למחיקה

**טענת יציאה:** מסיר את המשתמש שמזוהה לפי כתובת אימייל, ומעדכן את הקובץ לאחר המחיקה

- **update_flight(ActiveFlight flight, string file_path)**

**טענת כניסה: טיסה פעילה ונתיב לקובץ המשתמשים**

**טענת יציאה:** מאתר את כל המשתמשים שהזמינו את הטיסה, ועבור כל משתמש מסיר את כל ההזמנות הרלוונטיות, ואז שומר את הקובץ את הנתונים המעודכנים

**ActiveFlightsManager**

**תפקיד:**

מחלקה שמנהלת את רשימת הטיסות הפעילות בפועל. היא מספקת ממשק לקריאה, הוספה, הסרה, חיפוש ועדכון טיסות על בסיס קובץ JSON . המחלקה משמשת לניהול

של כל טיסה פעילה, שומרת את הנתונים המעודכנים מול המשתמשים ומסייעת לביצוע פעולות כמו חיפוש טיסה פנויה או סינון טיסות ישנות.

**פעולות:**

- **get_flights(string file_path)**
  **טענת כניסה:** נתיב לקובץ JSON של טיסות פעילות
  **טענת יציאה:** קורא את הקובץ, מפענח לרשימת טיסות פעילות ומחזיר אותה, אם הקובץ לא קיים נזרקת חריגה

- **add_flight(ActiveFlight flight, string file_path)**
  **טענת כניסה:** טיסה פעילה ונתיב לקובץ JSON
  **טענת יציאה:** מוסיף את הטיסה לרשימת הטיסות הקיימת ומעדכן את הקובץ בצורה מסודרת, זורק שגיאה אם מתרחשת שגיאה

- **found_flight(string flight_number, DateTime date, string file_path)**
  **טענת כניסה:** מספר טיסה, תאריך ונתיב קובץ
  **טענת יציאה:** מחפש טיסה לפי מספר טיסה ותאריך, ומחזיר את הטיסה אם נמצאה או null אם לא קיימת

- **update_flights(string flights_file_path, string users_file_path)**
  **טענת כניסה:** נתיבים לקבצי טיסות ומשתמשים
  **טענת יציאה:** מאתר טיסות שהסתיימו (לפני יום) ומסיר אותן מהמערכת, תוך עדכון קובץ המשתמשים להסרת ההזמנות הקשורות, לאחר מכן שומר מחדש את הקובץ המעודכן

- **remove_flight(ActiveFlight flight, string file_path)**
  **טענת כניסה:** טיסה פעילה ונתיב לקובץ טיסות
  **טענת יציאה:** מאתר את הטיסה בקובץ לפי מזהה, תאריך ושעה, ומסיר אותה אם קיימת, לאחר מכן מעדכן את הקובץ החדש ללא הטיסה שנמחקה

## ConnectionManager

**תפקיד:** מחלקה שמנהלת את החיבור TCP של הלקוח אל השרת. היא שומרת את כל הרכיבים הדרושים לתקשורת: כתובת IP, port, זרמים, קורא, כותב, אובייקט הצפנה ומצב הקשבה. המחלקה מספקת כלים ליצירת תקשורת מוצפנת עם השרת, סגירת החיבור באופן בטוח והפקת זרמי קריאה וכתיבה מאובטחים מתוך החיבור.

**תכונות:**

- server_ip

תפקיד ושימוש: כתובת ה־IP של השרת שאליו הלקוח יתחבר

- port

תפקיד ושימוש: מספר הפורט המשמש לתקשורת עם השרת, מתואם עם השרת - 6000

- tcp_client

תפקיד ושימוש: האובייקט המרכזי שמייצג את החיבור TCP

- tcp_stream

תפקיד ושימוש: הזרם הבינארי שנוצר מתוך החיבור ומשמש להצפנה

- reader

תפקיד ושימוש: אובייקט שמאפשר קריאה על גבי החיבור

- writer

תפקיד ושימוש: אובייקט שמאפשר כתיבה על גבי החיבור

- listening_for_messages

תפקיד ושימוש: משתנה שמציין האם התקשורת כרגע מאזינה להודעות

- security

תפקיד ושימוש: מופע של אובייקט SecurityManager שמבצע את ההצפנה, הפענוח והניהול של העברת הנתונים על גבי חיבור ה-TCP עם השרת

**פעולות:**

- **log_out()**

**טענת כניסה:** אין

**טענת יציאה:** סוגר ומנקה את כל רכיבי החיבור: זרמים, קורא, כותב, אובייקט TCP ואובייקט ההצפנה, מאפס את המשתנים הגלובליים של התקשורת, ומחזיר true אם התהליך הצליח או false במקרה של חריגה

- **get_reader_writer(TcpClient client)**

**טענת כניסה:** אובייקט TcpClient פתוח ופעיל
**טענת יציאה:** מפיק את זרם הרשת מתוך החיבור, ויוצר ממנו StreamReader ו־StreamWriter, מחזיר את שניהם כ-tupple, ואם החיבור לא תקין זורק שגיאה מתאימה

## OnlineUsersManager

**תפקיד:**

מחלקה שמנהלת את כל המשתמשים שמחוברים כרגע לשרת. היא שומרת מילון שבו כל משתמש משויך לחיבור TCP ואובייקט ההצפנה שלו. המחלקה מאפשרת הוספה, הסרה, בדיקת קיום, שליפת חיבור לפי אימייל והחזרת רשימת כל המשתמשים המחוברים. המחלקה היא זו שמאפשרת לשרת לשלוח ללקוח הודעות מיוזמתו בחלון הצ'אט.

**תכונות:**

- loged_in_users
תפקיד ושימוש: מילון שמאחסן את המשתמשים המחוברים, אובייקט TcpClient שמייצג את החיבור בין השרת ללקוח ואובייקט SecurityManager שאחראי על ההצפנה באותו חיבור

**פעולות:**

- **Add(User user, TcpClient client, SecurityManager security)**
**טענת כניסה:** אובייקט משתמש, חיבור TCP ואובייקט הצפנה
**טענת יציאה:** מוסיף את המשתמש לרשימת המשתמשים המחוברים תוך קישור לחיבור שלו ולמערכת ההצפנה שבשימוש

- **Contains(string email)**
**טענת כניסה:** כתובת אימייל של משתמש

**טענת יציאה:** סורק את רשימת המשתמשים המחוברים ובודק האם יש משתמש עם האימייל הזה, מחזיר true אם נמצא ו־ false אם לא

- **getConnection(string email)**

**טענת כניסה:** כתובת אימייל של משתמש

**טענת יציאה:** מאתר את המשתמש לפי האימייל ומחזיר את החיבור וההצפנה שלו כזוג ערכים, אם המשתמש לא נמצא מחזיר null בשני השדות

- **RemoveUser(User user)**

**טענת כניסה:** אובייקט משתמש למחיקה

**טענת יציאה:** מחפש את המשתמש ברשימה לפי כתובת האימייל ומסיר אותו אם נמצא, לא מבצע פעולה אם המשתמש לא קיים

- **GetOnlineUsers()**

**טענת כניסה:** אין

**טענת יציאה:** מחזיר רשימה לקריאה בלבד של כל המשתמשים שמחוברים כרגע לשרת, בלי אפשרות לשנות אותה מבחוץ

## SecurityManager

**תפקיד:**

המחלקה אחראית על כל ניהול ההצפנה במערכת, הן בצד השרת והן בצד הלקוח. היא משלבת בין הצפנת RSA (אסימטרית) להצפנת AES (סימטרית), תוך שימוש בגישה של הצפנה היברידית. המחלקה מאפשרת קריאה וכתיבה של הודעות מוצפנות, ניהול מפתחות AES ו־ RSA, המרה של מפתחות, hash לסיסמאות, ופענוח מידע רגיש במהלך התקשורת בין המשתמש לשרת.

**תכונות:**

- rsa

תפקיד ושימוש: מייצג את מפתח ההצפנה האסימטרי מסוג RSA לצורך הצפנת מפתח הצפנה סימטרית AES

- aes

תפקיד ושימוש: מופע של הצפנה סימטרית מסוג AES שבו נעשה שימוש להצפנת ההודעות

- writer

תפקיד ושימוש: אובייקט שמאפשר קריאה על גבי חיבור TCP

- reader

תפקיד ושימוש: אובייקט שמאפשר קריאה על גבי החיבור TCP

**פעולות:**

- **writeEncrypted(string msg)**

**טענת כניסה:** מחרוזת טקסט לשליחה

**טענת יציאה:** מצפין את ההודעה באמצעות מפתח AES , יוצר IV חדש ואם זהו המסר הראשון מצרף גם את המפתח המוצפן ב־RSA לבסוף כותב את המידע בצורה בטוחה לשרת.

- **readEncrypted()**

**טענת כניסה:** אין

**טענת יציאה:** קורא שורת טקסט מוצפנת באמצעות reader, מפענח אותה לפי פורמט הנתונים, בונה את מפתח ה־ AES אם זו ההודעה הראשונה שמתקבלת על ידי השרת, ומחזיר את תוכן ההודעה כפלט טקסט רגיל

- **get_public_key()**

**טענת כניסה:** אין

**טענת יציאה:** מחזיר את המפתח הציבורי של השרת בפורמט XML ללא המפתח הפרטי, לצורך שיתוף מאובטח עם הלקוח

- **rsa_from_public_key(string publicKeyXml)**

**טענת כניסה:** מחרוזת XML של מפתח ציבורי

**טענת יציאה:** יוצר אובייקט RSA חדש מתוך מפתח XML ציבורי שהתקבל מהשרת, לצורך הצפנת מפתחות AES בצד הלקוח

- **create_aes_details()**

  **טענת כניסה:** אין

  **טענת יציאה:** מייצר או משתמש במפתח AES קיים, יוצר IV חדש, ומחזיר את שניהם כצמד לצורך הצפנת הודעות חדשות

- **encrypt_aes_key(byte[] aesKey)**

  **טענת כניסה:** מערך בתים של מפתח AES

  **טענת יציאה:** מצפין את מפתח ה־ AES באמצעות מפתח RSA ציבורי לצורך שידור מאובטח לשרת

- **aes_encryption(byte[] msg_bytes, byte[] key, byte[] iv)**

  **טענת כניסה:** תוכן ההודעה כמערך בתים, מפתח AES ו־IV

  **טענת יציאה:** מצפין את התוכן באמצעות AES במצב CBC עם ריפוד PKCS7 ומחזיר את התוצאה כמערך בתים מוצפן

- **aes_decryption(byte[] msg_bytes, byte[] key, byte[] iv)**

  **טענת כניסה:** הודעה מוצפנת כמערך בתים, מפתח AES ו־IV

  **טענת יציאה:** מפענח את ההודעה המוצפנת באמצעות AES לפי המפתח וה־ IV שנמסרו, ומחזיר את הנתונים המקוריים כמערך בתים

- **hash_password(string password, byte[] salt)**

  **טענת כניסה:** סיסמה רגילה ו־ saltאקראי

  **טענת יציאה:** מחבר את הסיסמה עם ה-salt מבצע עליה פעולת hash באמצעות SHA256ומחזיר את הפלט לצורך שמירה והשוואה מאובטחת

- **generate_salt()**

**טענת כניסה**: אין

**טענת יציאה**: יוצר מערך אקראי בגודל 32 בתים שישמש כ־ salt להצפנת סיסמאות ומחזיר אותו.

## UDP

**תפקיד:**

מחלקה שמאפשרת ללקוח לבדוק האם הוא יכול לשמש כשרת בתוך הרשת המקומית. המחלקה משתמשת בשידור UDP לצורך גילוי שרת פעיל, ומנהלת תקשורת הצפנה מאובטחת באמצעותRSA . היא אחראית גם למנוע מלקוחות אחרים להפוך לשרתים נוספים, כדי להבטיח שרק שרת אחד פעיל ברשת בכל רגע נתון.

**פעולות:**

- **canIBeServer()**

**טענת כניסה**: אין

**טענת יציאה**: משדרת בקשה לכל הרשת לבדוק האם כבר יש שרת אחר. אם מתקבלת תשובה שלילית מאובטחת מהשרת הכוללת IP ומפתח ציבורי, מגדירה את המשתנים המתאימים ומחזירהfalse . אם לא מתקבלת תשובה, מחזירה true ומאפשרת ללקוח להפוך לשרת

**פעולות פנימיות:**

- **request()**

**טענת כניסה**: אין

**טענת יציאה**: מבצעת את שידור ההודעה לכל הרשת. מגדירה גם timeout לקבלת תשובה

- **negative_response()**

**טענת כניסה**: אין

**טענת יציאה**: בודקת האם התקבלה תגובת דחייה מהשרת הקיים. אם ההודעה תקינה, מפענחת את כתובת השרת מהמפתח המוצפן בעזרת RSA שומרת את ה־ IP וה־ public key של השרת, ומחזירהtrue , אחרת מחזירה false

- **denyOthers()**

**טענת כניסה**: אין

**טענת יציאה**: מאזינה ברקע לבקשות מלקוחות שמנסים להפוך לשרתים. כשמתקבלת בקשה כזו, מזהה אותה, מצפינה הודעת דחייה עם כתובת ה־ IP של השרת, ושולחת בחזרה ללקוח. הריצה נמשכת כל עוד השרת לא נסגר

**פעולות פנימיות:**

- **someone_requested()**

**טענת כניסה**: אין

**טענת יציאה**: מאזין לבקשות ברשת, ואם מתקבלת בקשה מלקוח מייבא את המפתח הציבורי של הלקוח ושומר אותו להצפנה חוזרת. אם כן, מחזיר true , אחרת false

- **deny()**

**טענת כניסה**: אין

**טענת יציאה**: בונה תגובת דחייה מוצפנת בעזרת המפתח של הלקוח המכילה את כתובת ה־ IP של השרת. מוסיף גם את המפתח הציבורי של השרת בתשובה, ואז שולח את ההודעה ללקוח המבקש

## UI

**תפקיד:**

מחלקה שאחראית על ניהול כל ממשק המשתמש (GUI) במערכת. היא מרכזת פעולות נפוצות לעיצוב והפעלה של ממשקים, כולל עיגול פינות לרכיבים, סידור כפתורים, טיפול ב־ hover, הצגת תמונת פרופיל, ניווט בין חלונות, ויציאה.

**פעולות:**

- **startThread(Action act)**

**טענת כניסה:** פעולה מסוג Action

**טענת יציאה**: מפעיל את הפעולה ברקע כ־ Thread (תהליכון) חדש שהוגדר כפועל ברקע.

- **RoundCorners(Control control, int radius, Color color, int width)**

**טענת כניסה**: רכיב, רדיוס פינות, צבע קו, ועובי קו

**טענת יציאה**: מצייר פינות מעוגלות לרכיב

- **calculate_size(int num, double value)**

**טענת כניסה**: מספר וגודל יחסי

**טענת יציאה**: מחזיר את המכפלה בין num ל־ value כשלם, שימושי לחישוב יחסי של מיקומים וגדלים בממשק הגרפי

- **set_navbar(Panel navbar, Size clientSize, Font btn_font)**

**טענת כניסה**: פאנל ניווט, גודל חלון, גופן כפתורים

**טענת יציאה**: ממקם את כפתורי הניווט בצורה יחסית ומוסיף להם אירועים ל־ hover כולל סידור מיקום וגודל יחסיים.

- **set_profile_picture(Dictionary<string, Control> controls_dict)**

**טענת כניסה**: מילון של שם רכיב לרכיב

**טענת יציאה**: מנהל שליחת בקשת תמונת פרופיל לשרת, מקבל את המידע, ואם יש מציג תמונה ונותן התנהגות לכפתורים בהתאם להתחברות. אם אין חיבור מגדיר לחצנים לפתיחת Login

- **display_profile_picture(IconButton profile_picture, Image img, Label name_label, string name, IconButton logout_btn)**

**טענת כניסה**: כפתור תמונת פרופיל, תמונה, תווית שם משתמש, שם, וכפתור יציאה

**טענת יציאה**: מציג את תמונת הפרופיל והשם של המשתמש בפינה העליונה, תומך ב־ toggle לכפתור ה־logout

- **nav_buttons_hover(object sender, EventArgs e)**

**טענת כניסה**: אובייקט הכפתור שעליו עבר הסמן

**טענת יציאה**: מבליט את הכפתור עם קו תחתי בעת ריחוף

- **nav_buttons_stop_hover(object sender, EventArgs e)**

  **טענת כניסה**: אובייקט הכפתור שממנו יצא הסמן

  **טענת יציאה**: מבטל את הקו התחתי מהכפתור ומחזיר אותו למצב רגיל

- **AttachClickHandlers(Control.ControlCollection controls)**

  **טענת כניסה**: אוסף רכיבים

  **טענת יציאה**: מוסיף לכל רכיב ולתתי־הרכיבים בו אירוע click שגורם לו לקבל פוקוס או מעביר את הפוקוס לכפתור דמה, ומסתיר את תפריט האופציות אם קיים

- **registerControls(Control.ControlCollection controls, Dictionary<string, Control> dict)**

  **טענת כניסה**: אוסף רכיבי ממשק ומילון יעד

  **טענת יציאה**: מוסיף כל רכיב בממשק (כולל תת־רכיבים) למילון לפי שמו, לטובת גישה נוחה בעתיד

- **logout(Form form)**

  **טענת כניסה**: חלון הפעיל הנוכחי

  **טענת יציאה**: שולח בקשת יציאה מוצפנת לשרת, ממתין לתשובה, ואם התקבלה הצלחה, סוגר את החיבור, מחזיר את הממשק למסך הבית, ומסתיר את החלון הנוכחי

**קטעי קוד של האלגוריתמים**

1. **בחירת השרת ומניעת לקוחות אחרים מלקבל זכאות לתפקיד**

מאפשר רק למשתמש הראשון שמפעיל את המערכת להפוך להיות השרת ולאחר מכן מונע מכל האחרים להפוך לשרת

```
static class UDP
 {

     const int port = 3500;
     /// <summary>
     /// asks if the client can become the server
     /// </summary>
     /// <returns></returns>
     public static bool canIBeServer()
     {
```

```
        UdpClient UDP_client = new UdpClient() { EnableBroadcast = true };
        IPEndPoint all_points = new IPEndPoint(IPAddress.Broadcast, port);
        SecurityManager securityManager = new SecurityManager();

        byte[] request_msg =
Encoding.UTF8.GetBytes($"{all_messages.CANI.ToString()}|{securityManager.get_public_ke
y()}");
        IPEndPoint response_points = new IPEndPoint(IPAddress.Any, 0);
        //sends an encrypted request with the udpclient to all the devices on the
network on the port
        //sets timeout on the tcpclient
        void request()
        {
            UDP_client.Send(request_msg, request_msg.Length, all_points);
            UDP_client.Client.ReceiveTimeout = 1000;
        }
        //if the timeout runs out or any other error occurs returns false
        //reads the message with the udpclient, decrypts it and sets the ip of the
server and the public rsa key of the server
        //returns true
        bool negative_response()
        {
            try
            {
                byte[] reply = UDP_client.Receive(ref response_points);

                string msg = Encoding.UTF8.GetString(reply);
                byte[] decrypted_bytes = null;

                decrypted_bytes =
Server.rsa.Decrypt(Convert.FromBase64String(msg.Split('~')[0]),
RSAEncryptionPadding.OaepSHA1);


                string decrypted_string = Encoding.UTF8.GetString(decrypted_bytes);

                if (decrypted_string.StartsWith(all_messages.FAILED.ToString()))
                {

                    decrypted_string =
decrypted_string.Substring(all_messages.FAILED.ToString().Length);

                    ConnectionManager.server_ip = decrypted_string;
                    ConnectionManager.security = new SecurityManager();

ConnectionManager.security.rsa_from_public_key(msg.Split('~')[1]);
                    return true;
                }
                else
                {
                    return false;
                }

            }
            catch (SocketException ex)
            when (ex.SocketErrorCode == SocketError.TimedOut)
            {

                return false;
            }
            catch
            {
```

```
                return false;
            }


        }
        request();
        if (negative_response())
        {

            UDP_client.Close();
            return false;
        }
        UDP_client.Close();
        return true;
    }
    /// <summary>
    /// activates a thread that runs as long as the server is active
    /// denies all the clients from becoming the server
    /// </summary>
    public static void denyOthers()
    {
        SecurityManager security = new SecurityManager();
        UdpClient socket = new UdpClient();
        IPEndPoint point = new IPEndPoint(IPAddress.Any, port);
        socket.Client.SetSocketOption(SocketOptionLevel.Socket,
SocketOptionName.ReuseAddress, true);
        socket.Client.Bind(point);
        socket.Client.ReceiveTimeout = 1000;
        //if the timeout runs out or any other error occurs returns false
        //recieves a request message from the tcpclient and sets the public key of
the client rsa instance
        //returns true
        bool someone_requested()
        {
            try
            {
                byte[] request_msg = socket.Receive(ref point);
                string request_msg_str = Encoding.UTF8.GetString(request_msg);
                byte[] msg = Encoding.UTF8.GetBytes(request_msg_str.Split('|')[0]);
                string public_key = request_msg_str.Split('|')[1];

                if
(msg.SequenceEqual(Encoding.UTF8.GetBytes(all_messages.CANI.ToString())))
                {

                    security.rsa_from_public_key(public_key);
                    return true;
                }
                return false;
            }
            catch (SocketException ex)
            when (ex.SocketErrorCode == SocketError.TimedOut)
            {

                return false;
            }
            catch { return false; }

        }

        //sends a response to the client with the IP of the server and the public rsa
key of the server
```

```
        void deny()
        {
            string IP = Dns.GetHostEntry(Dns.GetHostName()).AddressList.First(ip =>
ip.AddressFamily == AddressFamily.InterNetwork).ToString();

            byte[] encrypted =
security.rsa.Encrypt(Encoding.UTF8.GetBytes(all_messages.FAILED.ToString() + IP),
RSAEncryptionPadding.OaepSHA1);
            byte[] response =
                Encoding.UTF8.GetBytes(
                    Convert.ToBase64String(encrypted) +
                    "~"
                    + Server.rsa.ToXmlString(false));

            socket.Send(response, response.Length, point);
        }

        UI.startThread(() =>
        {

            while (!Server.is_running.IsCancellationRequested)
            {
                if (someone_requested())
                    deny();
                Thread.Sleep(10);
            }
        });
    }
}
```

**2. יצירת חיבור מאובטח להעברת הודעות בתקשורת מוצפנת בין הלקוח לשרת**

מאפשר תקשורת בטוחה ומאובטחת בין הלקוח לשרת על גבי חיבור TCP ברשת

```
public class SecurityManager
{

    public RSA rsa { get; set; }
    public Aes aes { get; set; }
    public StreamWriter writer { get; set; }
    public StreamReader reader { get; set; }
    /// <summary>
    /// creates the aes details and encrypts the message with it
    /// if the message is the first one sent, the rsa key encrypts the aes key and
         sends the message with it and the IV value
    /// otherwise it sends the encrypted message and the IV value
    /// </summary>
    /// <param name="msg"></param>
    public void writeEncrypted(string msg)
    {
        try
        {

            if (writer == null && ConnectionManager.writer != null)
            {
                writer = ConnectionManager.writer;
```

```
            reader = ConnectionManager.reader;
        }
        byte[] msg_bytes = Encoding.UTF8.GetBytes(msg);

        bool first = this.aes == null;
        (byte[] aes_key, byte[] IV) = this.create_aes_details();

        byte[] encrypted_msg_bytes = this.aes_encryption(msg_bytes, aes_key,
    IV);

        string IV_string = Convert.ToBase64String(IV);
        string encrypted_msg_string =
    Convert.ToBase64String(encrypted_msg_bytes);

        if (first)
        {
            byte[] encrypted_key_bytes = this.encrypt_aes_key(aes_key);
            string encrypted_key_string =
    Convert.ToBase64String(encrypted_key_bytes);
            writer.WriteLine(IV_string + ":" + encrypted_key_string + ":" +
    encrypted_msg_string);
        }
        else
        {
            writer.WriteLine(IV_string + ":" + encrypted_msg_string);
        }
    }
    catch
    {
        throw;
    }
}
/// <summary>
/// reads the message over the tcp connection
/// if the message has 3 parts it means that the server recieved the first
     message
/// and it initializes the symmetric encryption with the client
/// takes the IV and the aes key and decryptes the message
/// returns the message
/// </summary>
/// <returns></returns>
public string readEncrypted()
{
    try
    {
        if (writer == null && ConnectionManager.writer != null)
        {
            writer = ConnectionManager.writer;
            reader = ConnectionManager.reader;
        }
        string msg = reader.ReadLine();

        if (msg == null) return null;
        if (msg.Split(':').Length < 2 || msg.Split(':').Length > 3) return null;
        string IV_string = msg.Split(':')[0];
        string encrypted_msg_string;
        string encrypted_key_string;
        byte[] decrypted_msg_bytes;

        if (msg.Split(':').Length == 3)
        {
            encrypted_key_string = msg.Split(':')[1];
            encrypted_msg_string = msg.Split(':')[2];
```

```
            byte[] decrypted_aes_key =
     Server.rsa.Decrypt(Convert.FromBase64String(encrypted_key_string),
     RSAEncryptionPadding.OaepSHA1);
            decrypted_msg_bytes =
     this.aes_decryption(Convert.FromBase64String(encrypted_msg_string),
     decrypted_aes_key, Convert.FromBase64String(IV_string));
            this.aes = Aes.Create();
            this.aes.Mode = CipherMode.CBC;
            this.aes.Padding = PaddingMode.PKCS7;
            this.aes.Key = decrypted_aes_key;
            this.aes.IV = Convert.FromBase64String(IV_string);
        }
        else
        {
            encrypted_msg_string = msg.Split(':')[1];
            decrypted_msg_bytes =
     this.aes_decryption(Convert.FromBase64String(encrypted_msg_string),
     this.aes.Key, Convert.FromBase64String(IV_string));
            this.aes.IV = Convert.FromBase64String(IV_string);
            this.aes.Mode = CipherMode.CBC;
            this.aes.Padding = PaddingMode.PKCS7;
        }
        return Encoding.UTF8.GetString(decrypted_msg_bytes);
    }
    catch
    {

        return null;
        throw;
    }

}
/// <summary>
/// returns the server rsa as a xml string
/// </summary>
/// <returns></returns>
public string get_public_key()
{
    return Server.rsa.ToXmlString(false);
}
/// <summary>
/// creates an rsa instance from a public key
/// </summary>
/// <param name="publicKeyXml"></param>
public void rsa_from_public_key(string publicKeyXml)
{
    rsa = RSA.Create();
    rsa.FromXmlString(publicKeyXml);
}
/// <summary>
/// generates new aes instance and key if dont exist
/// generates new IV value
/// returns a tuple of the key and the IV value
/// </summary>
/// <returns></returns>
public (byte[] Key, byte[] IV) create_aes_details()
{

    if (aes == null)
    {
        aes = Aes.Create();
        aes.GenerateKey();
    }
```

```
    aes.GenerateIV();
    aes.Mode = CipherMode.CBC;
    aes.Padding = PaddingMode.PKCS7;
    return (aes.Key, aes.IV);
}
/// <summary>
/// uses the public rsa key to encrypt the aes key
/// </summary>
/// <param name="aesKey"></param>
/// <returns></returns>
public byte[] encrypt_aes_key(byte[] aesKey)
{
    return rsa.Encrypt(aesKey, RSAEncryptionPadding.OaepSHA1);
}
/// <summary>
/// encrypts the given message using the given aes key and IV value
/// </summary>
/// <param name="msg_bytes"></param>
/// <param name="key"></param>
/// <param name="iv"></param>
/// <returns></returns>
public byte[] aes_encryption(byte[] msg_bytes, byte[] key, byte[] iv)
{
    using (Aes aes1 = Aes.Create())
    {
        aes1.Key = key;
        aes1.IV = iv;
        aes1.Mode = CipherMode.CBC;
        aes1.Padding = PaddingMode.PKCS7;
        ICryptoTransform encryptor = aes1.CreateEncryptor();
        return encryptor.TransformFinalBlock(msg_bytes, 0, msg_bytes.Length);
    }
}
/// <summary>
/// decrypts the given message using the given aes key and IV value
/// </summary>
/// <param name="msg_bytes"></param>
/// <param name="key"></param>
/// <param name="iv"></param>
/// <returns></returns>
public byte[] aes_decryption(byte[] msg_bytes, byte[] key, byte[] iv)
{
    using (Aes aes1 = Aes.Create())
    {
        aes1.Key = key;
        aes1.IV = iv;
        aes1.Mode = CipherMode.CBC;
        aes1.Padding = PaddingMode.PKCS7;
        ICryptoTransform decryptor = aes1.CreateDecryptor();
        return decryptor.TransformFinalBlock(msg_bytes, 0, msg_bytes.Length);
    }
}
/// <summary>
/// does the hash operation on the password and salt and returns it
/// </summary>
/// <param name="password"></param>
/// <param name="salt"></param>
/// <returns></returns>
public static byte[] hash_password(string password, byte[] salt)
{
    using (var sha256 = SHA256.Create())
    {
```

```
            byte[] combined =
        Encoding.UTF8.GetBytes(password).Concat(salt).ToArray();
            return sha256.ComputeHash(combined);
        }
    }
    /// <summary>
    /// generates a 32 byte array of random characters and returns it
    /// </summary>
    /// <returns></returns>
    public static byte[] generate_salt()
    {
        int size = 32;
        var salt = new byte[size];
        using (var rng = new RNGCryptoServiceProvider())
        {
            rng.GetBytes(salt);
        }
        return salt;
    }
     }
```

### 3. אלגוריתם חיפוש הטיסה

מאפשר למשתמש לקבל טיסות הלוך וחזור בהתאם לפרטים שהוא הזין בטופס

```
/// <summary>
/// takes the search data and finds all the active flights that match it
/// it creates a list of outbound flights and return flights that have enough space
/// for the amount of passengers selected and sends the list
/// </summary>
/// <param name="message"></param>

private void handle_search(ref string message)
{
    if (user == null || this.tcpClient == null)
    {
        message = all_messages.FAILED.ToString();
        return;
    }

    try
    {
        FlightSearchData search_data =
         JsonSerializer.Deserialize<FlightSearchData>(message);
        if (search_data.return_date < search_data.departure_date ||
         search_data.departure_date < DateTime.Now)
        {
            message = all_messages.FAILED.ToString();
        }

        List<Flight> getAllFlights()
        {
            try
            {
                List<Flight> flights = new List<Flight>();
                string filePath =
         System.IO.Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Data",
         "flights.csv");

                if (!File.Exists(filePath))
                {
                    throw new FileNotFoundException($"{filePath}: is missing");
                }
```

```
        using (var streamReader = new StreamReader(filePath))
        using (var csvReader = new CsvReader(streamReader,
 CultureInfo.InvariantCulture))
        {
            flights = csvReader.GetRecords<Flight>().ToList();
        }
        return flights;
    }
    catch (Exception ex)
    {
        throw ex;
    }
}

List<Flight> all_flights = getAllFlights();
List<ActiveFlight> departure_flights = new List<ActiveFlight>();
List<ActiveFlight> return_flights = new List<ActiveFlight>();

foreach (Flight flight in all_flights)
{
    if (search_data.origin.Equals($"{flight.origin_city},
 {flight.origin_country}", StringComparison.OrdinalIgnoreCase) &&
        search_data.destination.Equals($"{flight.destination_city},
 {flight.destination_country}", StringComparison.OrdinalIgnoreCase))
    {
        addFlights(flight, departure_flights, search_data.departure_date,
 search_data.flight_class);
    }

    if (search_data.destination.Equals($"{flight.origin_city},
 {flight.origin_country}", StringComparison.OrdinalIgnoreCase) &&
        search_data.origin.Equals($"{flight.destination_city},
 {flight.destination_country}", StringComparison.OrdinalIgnoreCase))
    {
        addFlights(flight, return_flights, search_data.return_date,
 search_data.flight_class);
    }
}

void addFlights(Flight flight, List<ActiveFlight> flights, DateTime date,
 string flight_class)
{
    List<string> active_days = flight.active_days.Trim('[', ']').Split('
 ').ToList();
    List<string> times = flight.times.Trim('[', ']').Replace(" ",
 "").Split(',').ToList();
    int dayofweek = (int)date.DayOfWeek + 1;
    if (active_days.Contains(dayofweek.ToString()))
    {
        for (int i = 0; i < flight.flights_per_day; i++)
        {
            ActiveFlight f = ActiveFlightsManager.found_flight(flight.id,
 times[i], date, active_flights_path);
            if (f != null)
            {
                if (f.available(search_data.passengers,
 search_data.flight_class))
                {
                    flights.Add(f);
                }
            }
            else
            {
```

```
                    ActiveFlight activeFlight = new ActiveFlight();
                    activeFlight.copy(flight, date, times[i]);
                    flights.Add(activeFlight);
                }
            }
        }
    }

    message = all_messages.SUCCESSFULL.ToString() +
     JsonSerializer.Serialize(departure_flights) + "|" +
     JsonSerializer.Serialize(return_flights);
}
catch
{
    message = all_messages.FAILED.ToString();
}
}
```

**מסמך הבדיקות:**

1. בדיקת זכאות להיות שרת- מטרת הבדיקה היא לוודא שלא יכול להיווצר מצב שבו שני משתמשים הם השרת. מה שבוצע בפועל זה שהיו מספר ניסיונות להפעיל את המערכת, גם לאחר זמן קצר וגם לאחר זמן ארוך, באותו הזמן, כדי לדמות מצב של כמה משתמשים פעילים ולוודא שהמערכת מאפשרת רק שרת אחד.
2. בדיקת תקינות רישום – הבדיקה מתבצעת גם בלקוח וגם בשרת כדי לוודא שמדובר בנתונים תקינים. מה שבוצע בפועל זה שבצד לקוח הבדיקה הייתה למלא את הטופס במגוון דרכים כך שבכל פעם שדה אחד אינו תקין, רק לאחר מכן הטופס מולא באופן תקין והבדיקה עברה לצד שרת. כדי לבדוק את צד השרת הטופס מולא מספר פעמים עם נתונים של משתמש זהה ופעם אחת עם פרטים של משתמש חדש.
3. בדיקת תקינות התחברות – מטרת הבדיקה היא לוודא שמשתמש שמזין פרטי משתמש תקינים מצליח להתחבר רק אם הוא לא מחובר כבר ממחשב אחר. אם הפרטים שהמשתמש מזין אינם מתאימים לאף משתמש קיים הוא לא יוכל להתחבר. מה שבוצע בפועל זה שבמספר הרצות של המערכת הוזנו פרטים שגויים, בהרצה אחת הייתה הזנה של פרטים נכונים שאפשרה התחברות ואז הוזנו אותם פרטים במספר הרצות כדי לוודא שהמערכת לא מאפשרת לאותו משתמש להתחבר יותר מפעם אחת.
4. בדיקת תקינות התנתקות מהחשבון – מטרת הבדיקה היא לוודא שלאחר התנתקות מהחשבון, החיבור TCP לא פעיל יותר וניתן להירשם ולהתחבר מחדש. מה שבוצע בפועל זה שהיו מספר התחברויות והתנתקויות ממספר הרצות שונות וחלונות שונים במערכת. ההתנתקויות היו יכולות להיות יפות (סגירת הטופס או לחיצה על כפתור ההתנתקות) או לא יפות (סגירת המערכת מבחוץ). לאחר מכן הייתה בדיקה שניתן לבצע את כל הפעולות במערכת כמו שצריך ושההתנתקות לא גרמה לתקלה. הבעיות העיקריות שנגרמו הם threads שלא נסגרו או טופלו כמו שצריך, והן נפתרו בהצבת תנאים על המשך הפעולה של ה-thread וכך הוא לא ממשיך כאשר יש התנתקות או סגירה של החלון.
5. בדיקת תקינות הזמנת טיסה – מטרת הבדיקה היא לוודא שמשתמש מצליח להזמין את המקומות שבחר והם מתעדכנים במערכת ושמשתמש לא יכול להזמין את אותה הטיסה אם כבר הזמין אותה. מה שבוצע בפועל זה שהתרחשו מספר הזמנות של טיסות שונות ואז הזמנות של אותן טיסות, כל הזמנה הייתה מפוקחת ונבדקה בקובץ ההזמנות.
6. בדיקת תקינות עמוד יצירת הקשר – מטרת הבדיקה היא לוודא שאין תקלות ביצירת הקשר עם השרת, ההודעות נשמרות, המידע מועבר בזמן אמת, הלקוח יכול לעבור בין חלונות ולחזור לצ'אט ושמספר משתמשים יכולים לתקשר עם השרת בו זמנית. מה שבוצע בפועל זה שבמספר הרצות שונות היו ניסיונות לשלוח לשרת הודעות, לסגור את החלון, להתנתק, לעבור חלון, לחזור לחלון הצ'אט, להתחבר מחדש. הבעיות העיקריות שהיו הן סגירת ה-thread המאזין להודעות בצד הלקוח ולאפשר מעבר בין החלונות וחזה לחלון יצירת הקשר. הבעיות

נפתרו באמצעות שליחת הודעת עזיבת הצ'אט לשרת ועדכון התנאי שאחראי על המשך פעולת ה-thread .

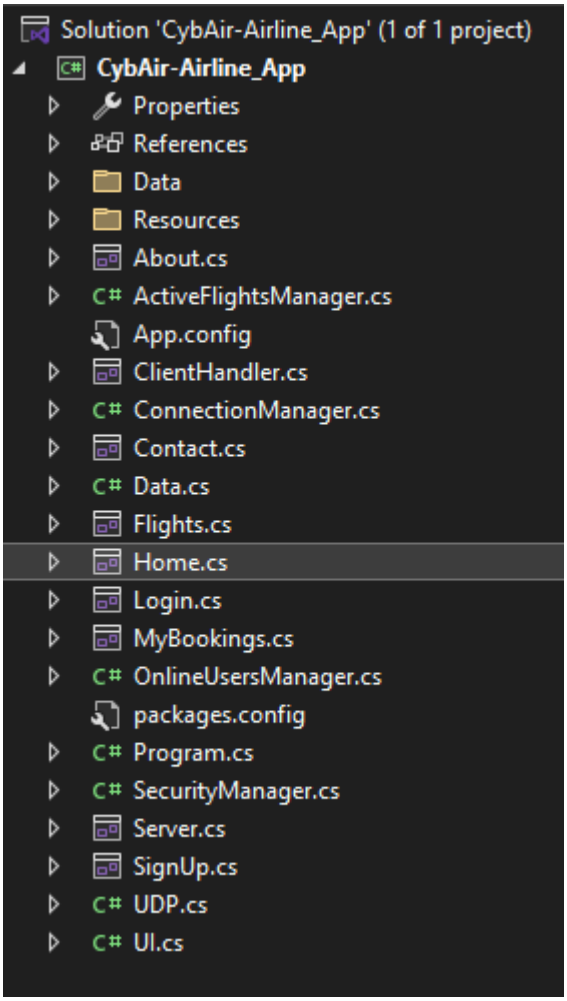
7. בדיקת תקינות ההצפנה – מטרת הבדיקה היא לוודא שהמידע שעובר ברשת בחיבור TCP אכן מוצפן ולא ניתן לראות את המידע החשוף מה שבוצע בפועל זה שבזמן הרצה ההודעות הודפסו לפני הצפנה ואחרי פענוח כדי לראות שיש תיאום והיה שימוש בתוכנת WireShark שמאפשרת לראות את המידע שעובר ברשת.

## מדריך למשתמש

**פירוט כלל קבצי המערכת:**

**תיקיות:**
Properties
References
Data
Resources

**קבצים:**
About.cs
ActiveFlightsManager.cs
App.config
ClientHandler.cs
ConnectionManager.cs
Contact.cs
Data.cs
Flights.cs
Home.cs
Login.cs
MyBookings.cs
OnlineUsersManager.cs
packages.config
Program.cs
SecurityManager.cs
Server.cs
SignUp.cs
UDP.cs
UI.cs




- **סביבה נדרשת:**

על מנת להריץ את המערכת נדרשת הסביבה הבאה:

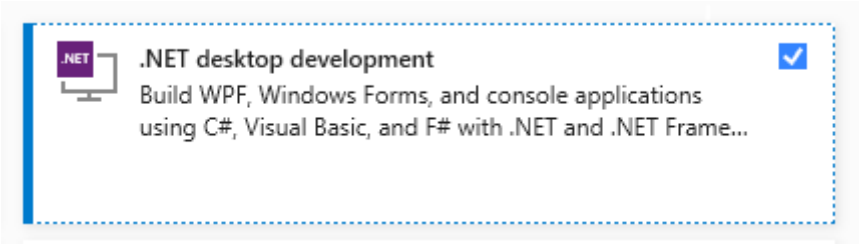
מערכת הפעלה: Windows

סביבת פיתוח: Visual Studio רצוי בגרסת 2022

ספריית .NET.

יש להוריד את סביבת הפיתוח בקישור: https://visualstudio.microsoft.com/downloads/

ולהיכנס ל-Visual Studio Installer שבו ניתן לבחור את התחום שבו עוסק הפרויקט ולהוריד את כל הכלים בצורה אוטומטית. המשבצת שיש לסמן בעת ההתקנה:




- **כלים נדרשים:**

הסביבה הנדרשת

כל התיקיות והקבצים בעץ הקבצים

המודולים החיצוניים המיובאים:

- CsvHelper – מודל שמיועד לקריאה וכתיבת קבצים מסוג csv. המודול אפשר עיבוד הנתונים בקובצים flights.csv ו-worldcities.csv.
- FontAwesome.Sharp – מודול שמיועד להצגת סמלים ויזואליים שלא נמצאים על המקלדת. השימוש הבולט במודול זה הוא לעיצוב הטפסים השונים במטרה לשפר את חווית המשתמש.
- GMap.Net – מודול המיועד לניהול מפות, מיקומים ותצוגות גאוגרפיות. המודול אפשר הצגת מפת העולם ונתונים שונים עליה.
- Krypton.Toolkit.Lite – מודול המיועד לשיפור העיצוב הבסיסי שמוצע על ידי הממשק הגרפי WindowsForm.
- MaterialSkin.2 - מודול המיועד לשיפור העיצוב הבסיסי שמוצע על ידי הממשק הגרפי WindowsForm.
- QRCoder – מודול שמיועד ליצירת QR code ויזואלי שניתן לסריקה. המודול אפשר ליצור QR code לכל כרטיס טיסה למטרות אימות הכרטיס בשדה התעופה.

כדי להתקין אותם ניתן להשתמש ב-NuGet Package Manager ולחפש כל מודול בנפרד ולהוריד אותו. דרך נוספת להוריד את המודולים היא להשתמש ב- Package Manager Console באמצעות השורה הבאה: Install-Package package_name

- **מיקומי הקבצים:**

כל הקבצים ממוקמים בתיקיית הפרויקט. התיקיה מכילה גם את התמונות שבהם נעשה שימוש, קבצי ההתקנה השונים, המחלקות, קבצי העיצוב וקבצי הפעלת המערכת.

- **רשת:**

על מנת שהמערכת תעבודה מכמה מחשבים שונים צריכה להיות מעורב רשת שדרכה כל המשתמשים והשרת יוכלו לתקשר. המערכת תומכת אך ורק בתקשורת ברשת מקומית (lan) ולכן חשוב שכל המשתמשים יהיו על אותה רשת מקומית כדי שהמערכת תפעל כראוי.

- **משתמשי המערכת:**

במערכת קיימים שני סוגי משתמשים: שרת – מנהל המערכת, לקוח – משתמש קצה
שני סוגי המשתמשים מפעילים את המערכת באותה צורה שהיא פתיחת הפרויקט מתוך Visual Studio ולחיצה על כפתור ההרצה. זאת מכיוון שהתפקידים נקבעים בזמן הרצת המערכת.

מסכי המערכת עבור כל משתמש מוצגים בפרק – "מבנה / ארכיטקטורה של הפרויקט", סעיף "תיאור מסכי המערכת"

## סיכום אישי / רפלקציה

לאורך השנה עבדתי על הפרויקט, עברתי תהליך ארוך, מעניין ומלמד במהלך הפיתוח. האתגר העיקרי בפיתוח הפרויקט הוא ניהול הזמן ושילוב שלו עם שאר המקצועות שנלמדים בבית הספר. אתגר נוסף הוא התכנון של כל השלבים והחלקים בפרויקט לפני העבודה המעשית. לאורך השנה השקעתי שעות רבות בפיתוח הפרויקט, למדתי חומר חדש ופיתחתי מיומנויות חדשות. העבודה הקשה היא זו שעזרה לי לפתור את הבעיות והאתגרים. ההצלחות הגדולות ביותר שלי הם שפיתחתי מערכת גדולה ומורכבת בעצמי, חשבתי על רעיונות מקוריים ויצירתיים, ביצעתי למידה עצמאית של המון חומר והצלחתי לנהל את הזמן בצורה יעילה. למדתי שפת תכנות חדשה לחלוטין בשבילי #C, למדתי על עקרונות תקשורת ועל הרשת שדרכה מחשבים מתקשרים, למדתי איך לנהל מאגרי נתונים בצורה יעילה, למדתי על אבטחה ועל הצפנה בסוגים שונים. כל הדברים החדשים שלמדתי הם כלים שאני אקח איתי להמשך וילוו אותי לאורך החיים, מכיוון שהם מאוד עניינו אותי ואני אשמח להעמיק בהם יותר בהמשך. התהליך גרם לי להבין כמה סבלנות ומנוחה הם גם חלקים חשובים בדיוק כמו העבודה הממושכת, מכיוון שבעיות ותקלות הם חלק בלתי נמנע בפיתוח פרויקט בסדר גודל כזה, ולפעמים הפסקה מאפשרת להסתכל על הדברים מנקודת מבט שונה ולמצוא פתרון יצירתי לבעיות. בנוסף התהליך עזר לי להבין שלמידת עמיתים, שאילת שאלות מתוך סקרנות, דיונים ולעזור לחברים שצריכים עזרה הם דרך מצוינת להתקדם וללמוד דברים חדשים. בהסתכלות לאחור הייתי יוצר מאגר נתונים מקצועי יותר כלומר Database רשמי שבו מאוחסנים כל הקבצים, שניתן להתחבר אליו והוא מיועד לאחסון מידע בפרויקטים מסוג זה. במידה והיו ברשותי משאבים נוספים הייתי משפר את העיצוב באמצעות ספריות חיצוניות מקצועיות יותר, הייתי מתחבר למקור מידע פתוח API של חברת תעופה רשמית שהטיסות שלה אמיתיות ויוצאות בזמן אמת כך שגם הטיסות שלי היו רשמיות. בנוסף הייתי מוסיף מערכת תשלום ומערכת שליחת אימיילים.

שאלות חקר שאני שואל את עצמי הן:

כיצד הבחירה שלי לשימוש בכל פרוטוקול תקשורת לביצוע הפעולות שהוא מבצע השפיעה על האבטחה וביצועי המערכת?

כיצד סדר החלקים בפרויקט שבחרתי לפתח השפיע על המאמץ הכולל שהשקעתי בפיתוח הפרויקט?

האם היה ניתן לעבוד באופן מסודר ויעיל יותר?

האם המערכת שלי יכולה באמת להיות שימושית בחיים האמיתיים במידה והיא תפעל באינטרנט?

אני רוצה להודות לכל מי שליווה אותי לאורך התהליך, למשפחה שלי, החברים שלי, והמורה שלי.

## ביבליוגרפיה:

Chandrajith, D., (אין שנה), Hybrid Encryption: Utilizing Symmetric and Asymmetric Encryption for Enhanced Protection
https://codimite.ai/blog/hybrid-encryption-utilizing-symmetric-and-asymmetric-encryption-for-enhanced-protection/

Fajwlewicz, F., (2016), What is TLS, what does TLS, and how does it impact Salesforce?
https://news.pwc.be/tls-impact-salesforce/

Gills, A. S., (2022), Diffie-Hellman key exchange (exponential key exchange)
https://www.techtarget.com/searchsecurity/definition/Diffie-Hellman-key-exchange

Ghann, J. & Mtsweni, N. & Kashti, O., (2024), 5 Best Practices for Site Search UX
https://www.fastsimon.com/ecommerce-wiki/site-search/best-practices-for-site-search-ux/

מאגר שדות תעופה
https://www.kaggle.com/datasets/thoudamyoihenba/airports

מאגר מסלולי טיסה
https://www.kaggle.com/datasets/open-flights/flight-route-database

מאגר הערים
https://simplemaps.com/data/world-cities